สยามชำนาญกิจ
SIAM CHAMNAN KIT

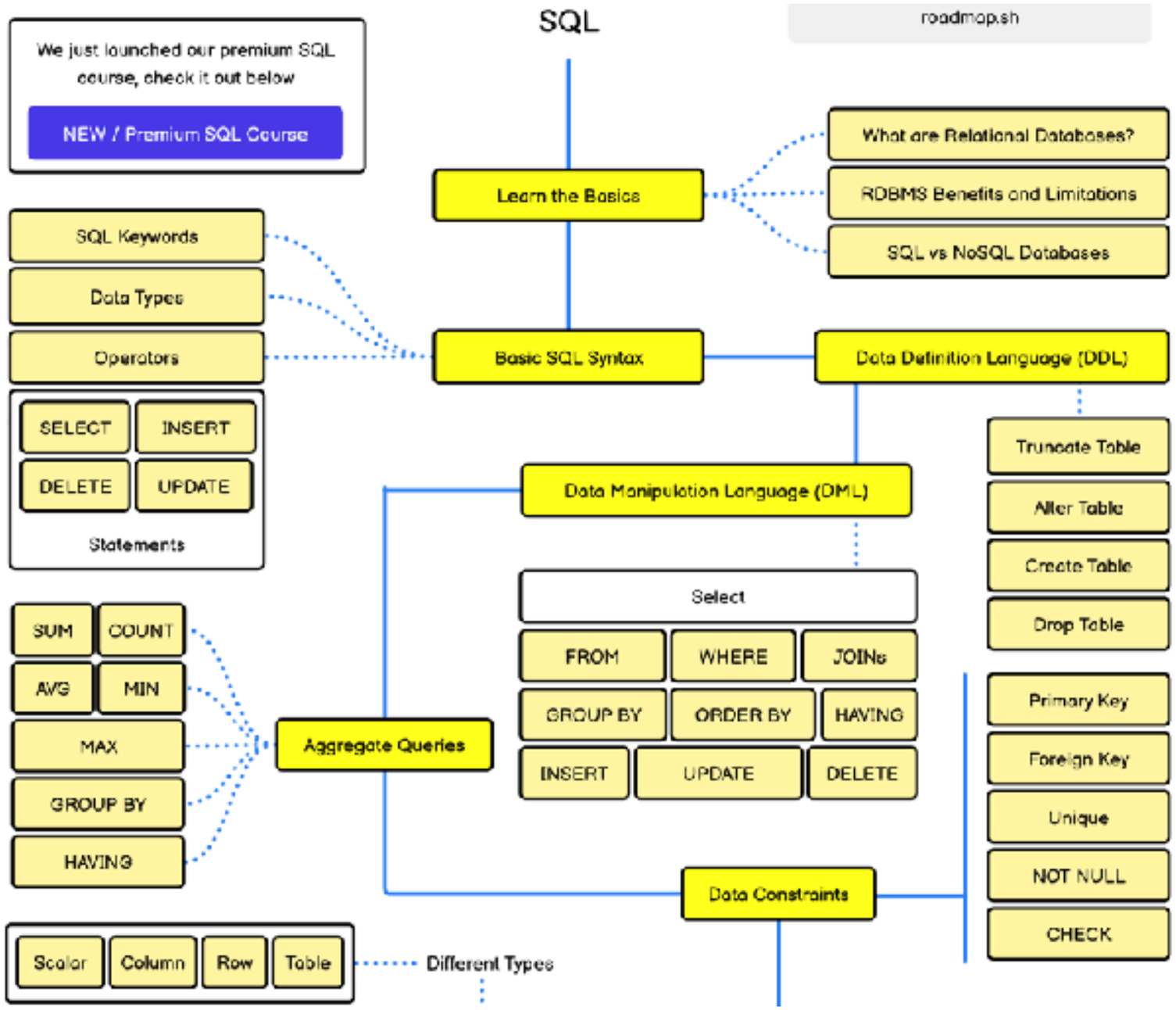
Database
SQL (Structure Query Language)

https://github.com/up1/workshop-database

Topics

Introduction to Database management
Database modeling
Relational Database vs NoSQL
Basic of Relational Database
SQL (Structure Query Language)
Workshop with SQL and PostgreSQL

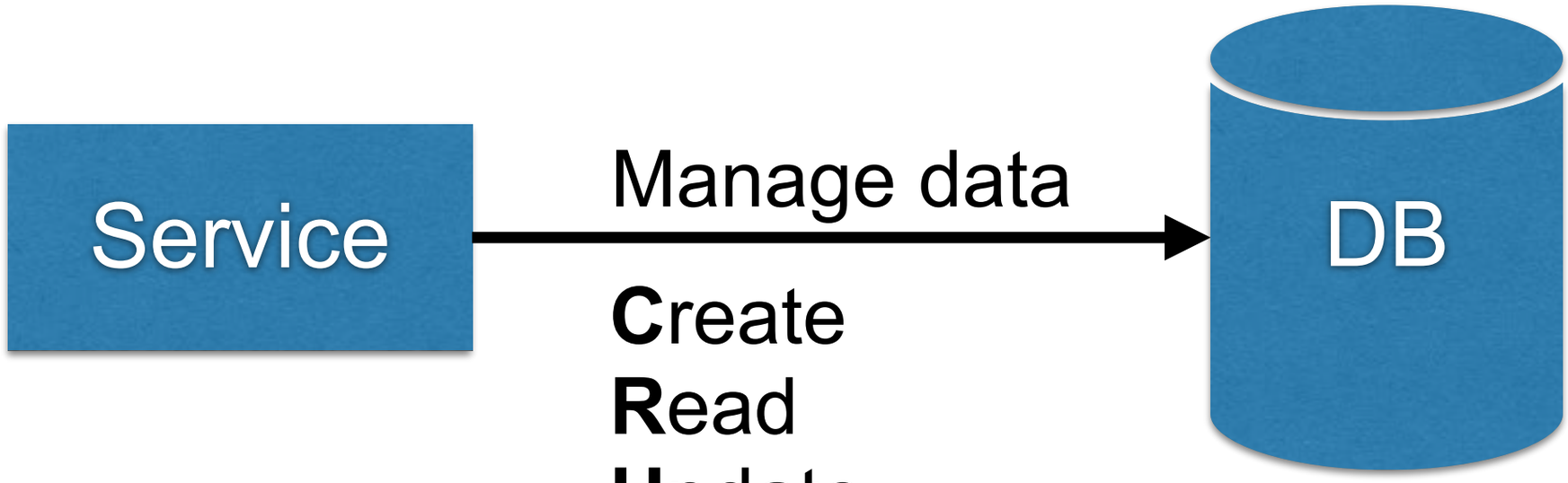
SQL Roadmap



https://roadmap.sh/sql

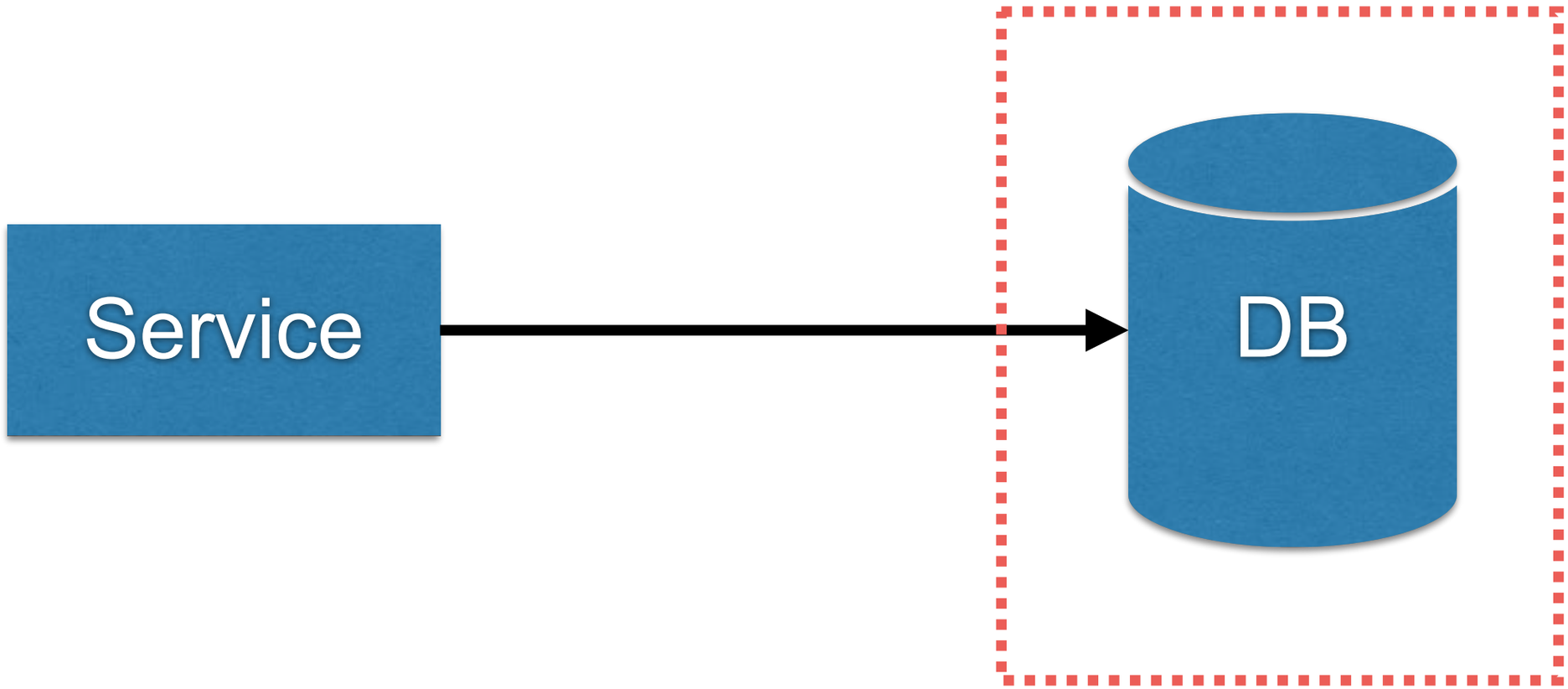
Database

Database



Working with Database

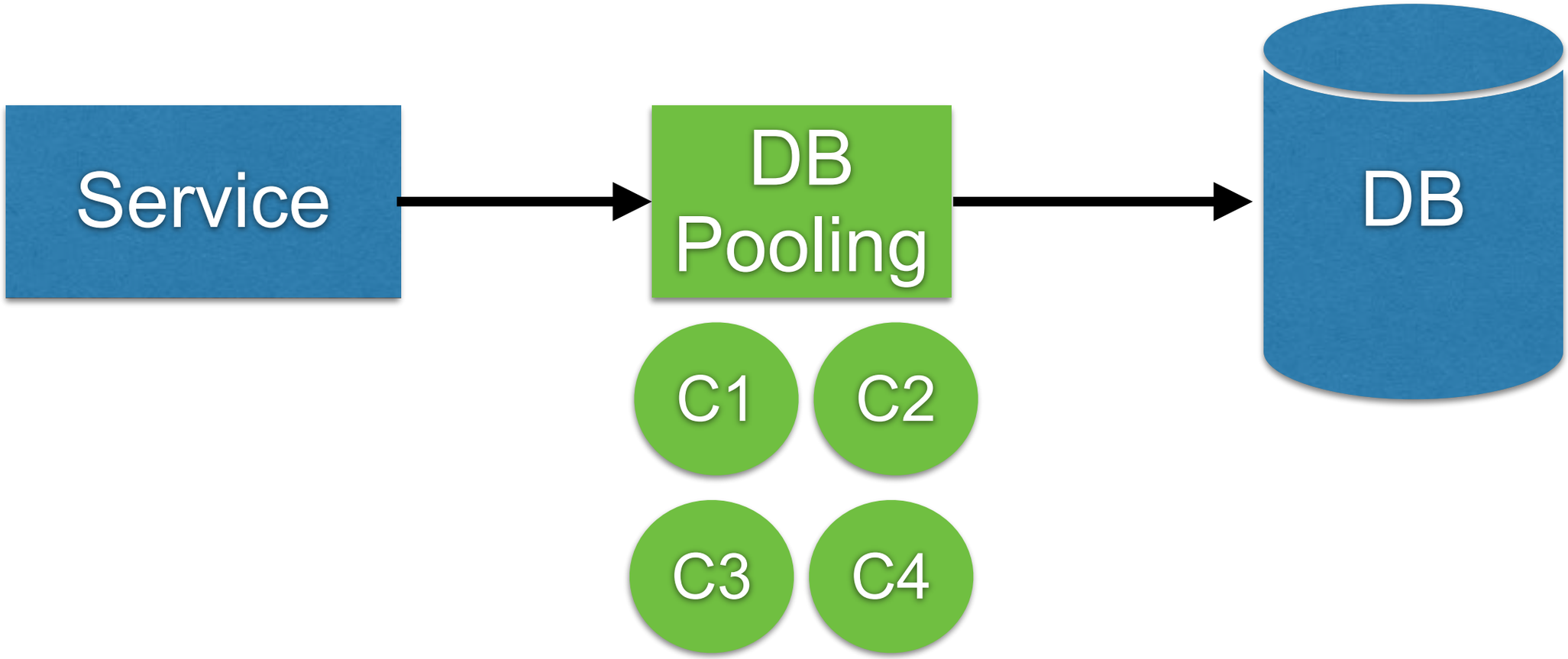
Directly to database



Data modeling
Database architecture

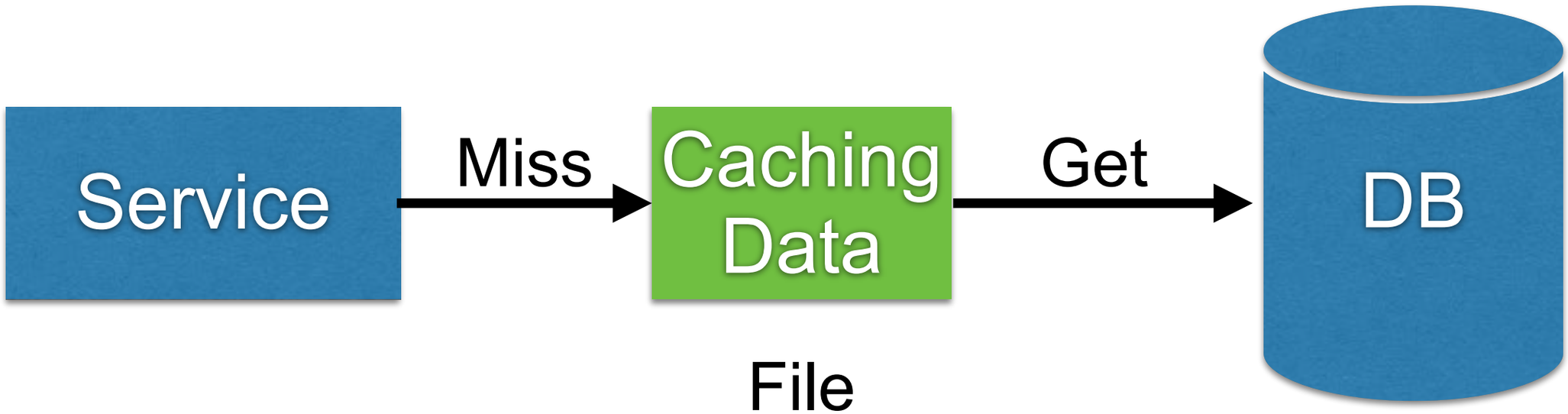
Working with Database

Connection pool



Working with Database

Caching data layer



File
Memory
Other database model

Scalability metrics

Throughput (transactions per second)
Latency (response time)
Availability (uptime)
Durability (data safety)

Database ?

Structured data collection
Records
Relationships

Database Management System ?

Software systems for storing and querying databases

User's perspective

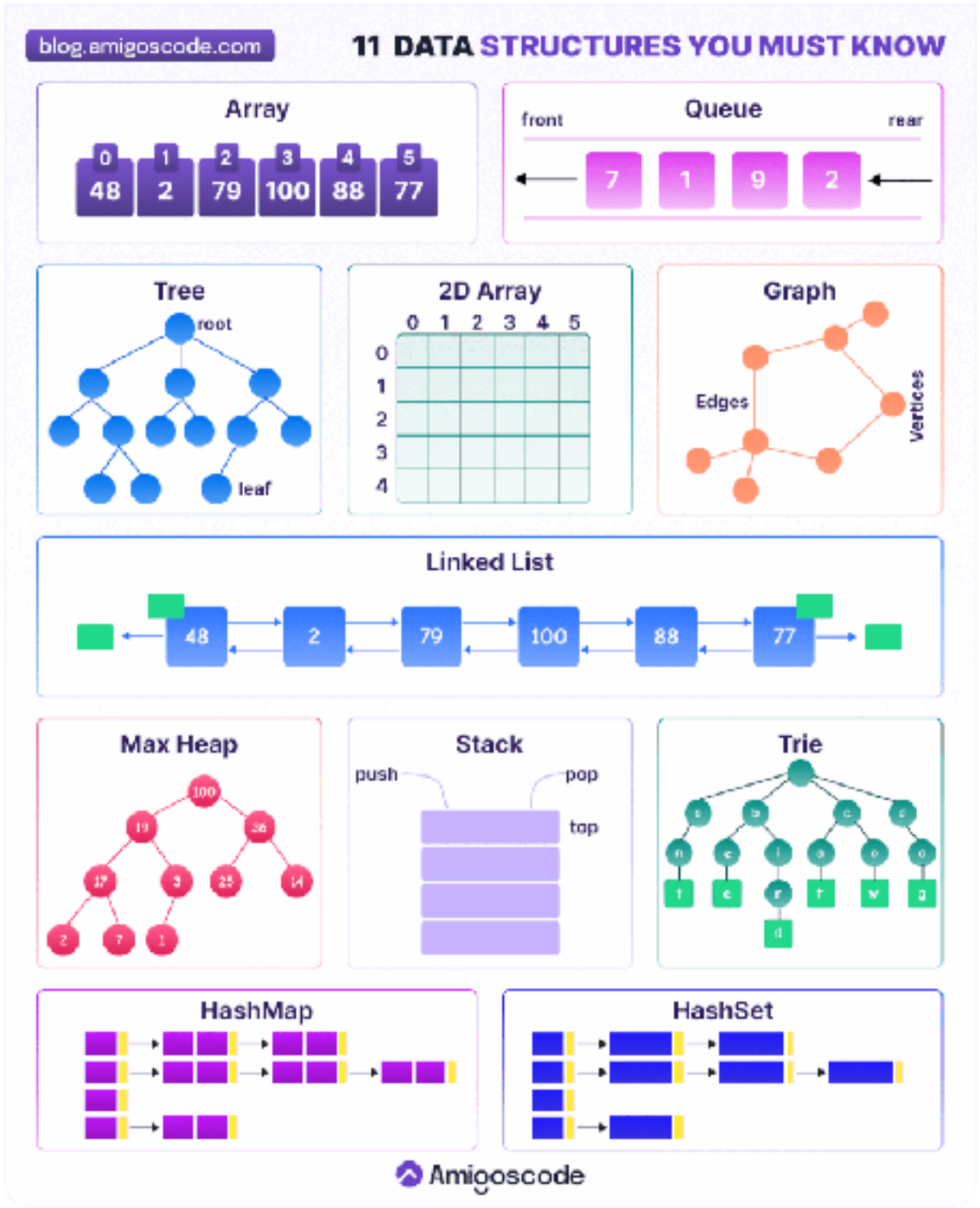
Data modeling
Querying data
Store data

Read

Write

Database Models ?

Data Structure



https://blog.amigoscode.com/p/11-data-structures-every-developer

Database Models ?

424 systems in ranking, July 2025

Rank Jul 2025	Rank Jun 2025	Rank Jul 2024	DBMS	Database Model	Score Jul 2025	Score Jun 2025	Score Jul 2024
1.	1.	1.	Oracle	Relational, Multi-model	1217.05	-13.33	-23.31
2.	2.	2.	MySQL	Relational, Multi-model	940.73	-12.85	-98.73
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	771.14	-5.61	-36.51
4.	4.	4.	PostgreSQL	Relational, Multi-model	680.89	+0.23	+41.93
5.	5.	5.	MongoDB	Document, Multi-model	403.83	+0.99	-25.99
6.	6.	↑ 7.	Snowflake	Relational	176.17	+1.68	+39.64
7.	7.	↓ 6.	Redis	Key-value, Multi-model	149.72	-2.01	-7.05
8.	8.	↑ 9.	IBM Db2	Relational, Multi-model	127.51	+2.38	+3.11
9.	9.	↓ 8.	Elasticsearch	Multi-model	118.83	-2.45	-12.00
10.	10.	10.	SQLite	Relational	115.44	-1.60	+5.49
11.	11.	↑ 12.	Apache Cassandra	Wide column, Multi-model	108.76	+0.49	+9.63
12.	12.	↑ 15.	Databricks	Multi-model	108.04	+3.36	+24.74
13.	13.	↑ 14.	MariaDB	Relational, Multi-model	95.44	+0.91	+4.85
14.	14.	↓ 11.	Microsoft Access	Relational	90.47	+2.19	-10.17
15.	15.	↑ 17.	Amazon DynamoDB	Multi-model	84.15	+0.81	+13.20

https://db-engines.com/en/ranking

Database Model

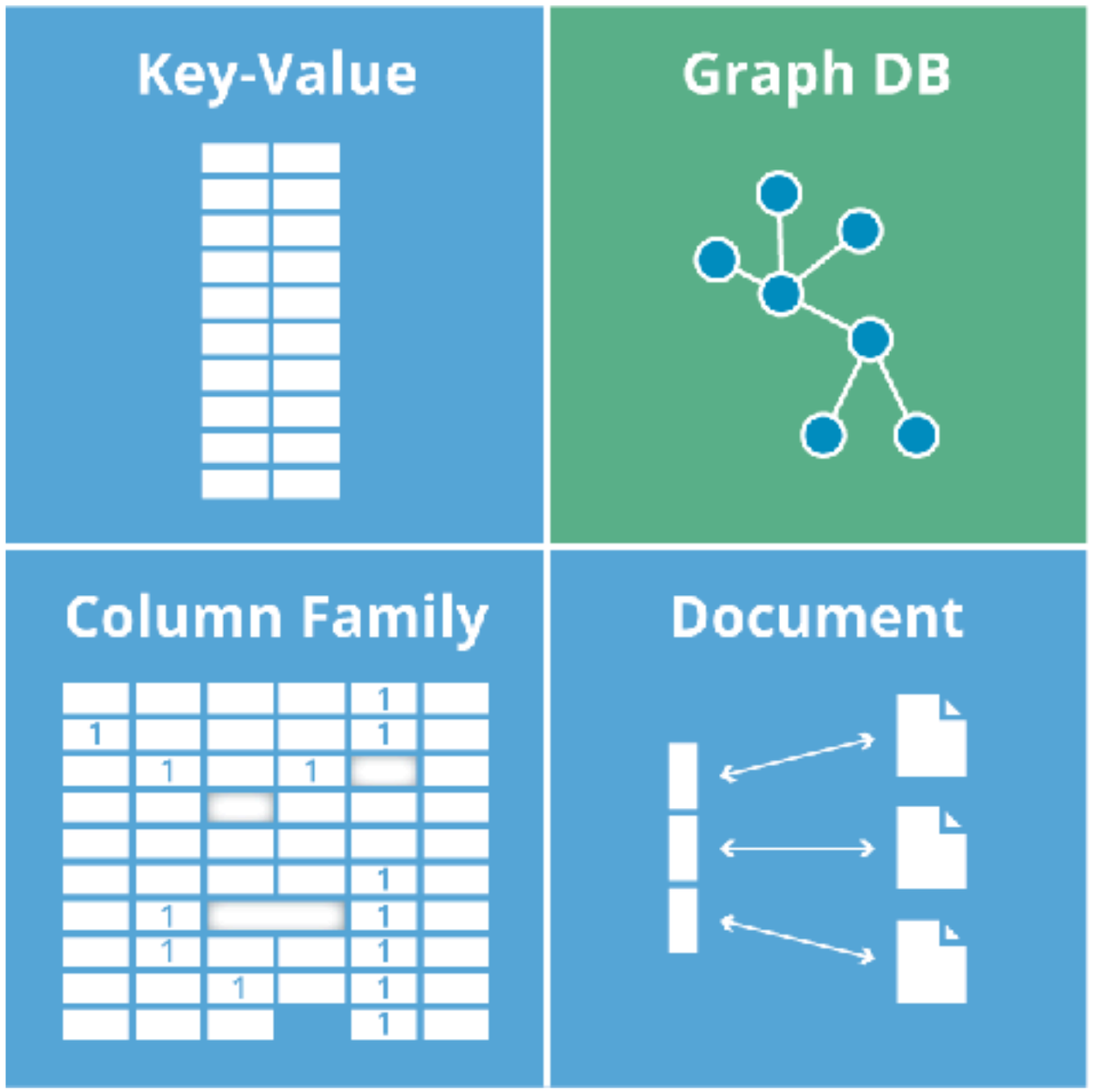
Relational

Document
Key-value
Search engine
Wide column
Graph
Time series
Vector

Non-Relational

NoSQL

NoSQL

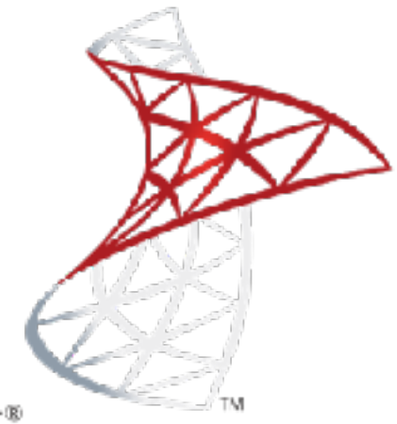


Relational Database

PostgreSQL

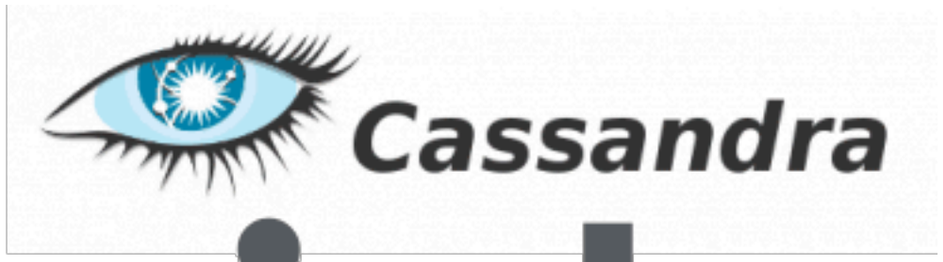
ORACLE®

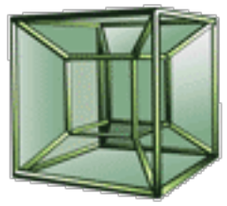
DATABASE

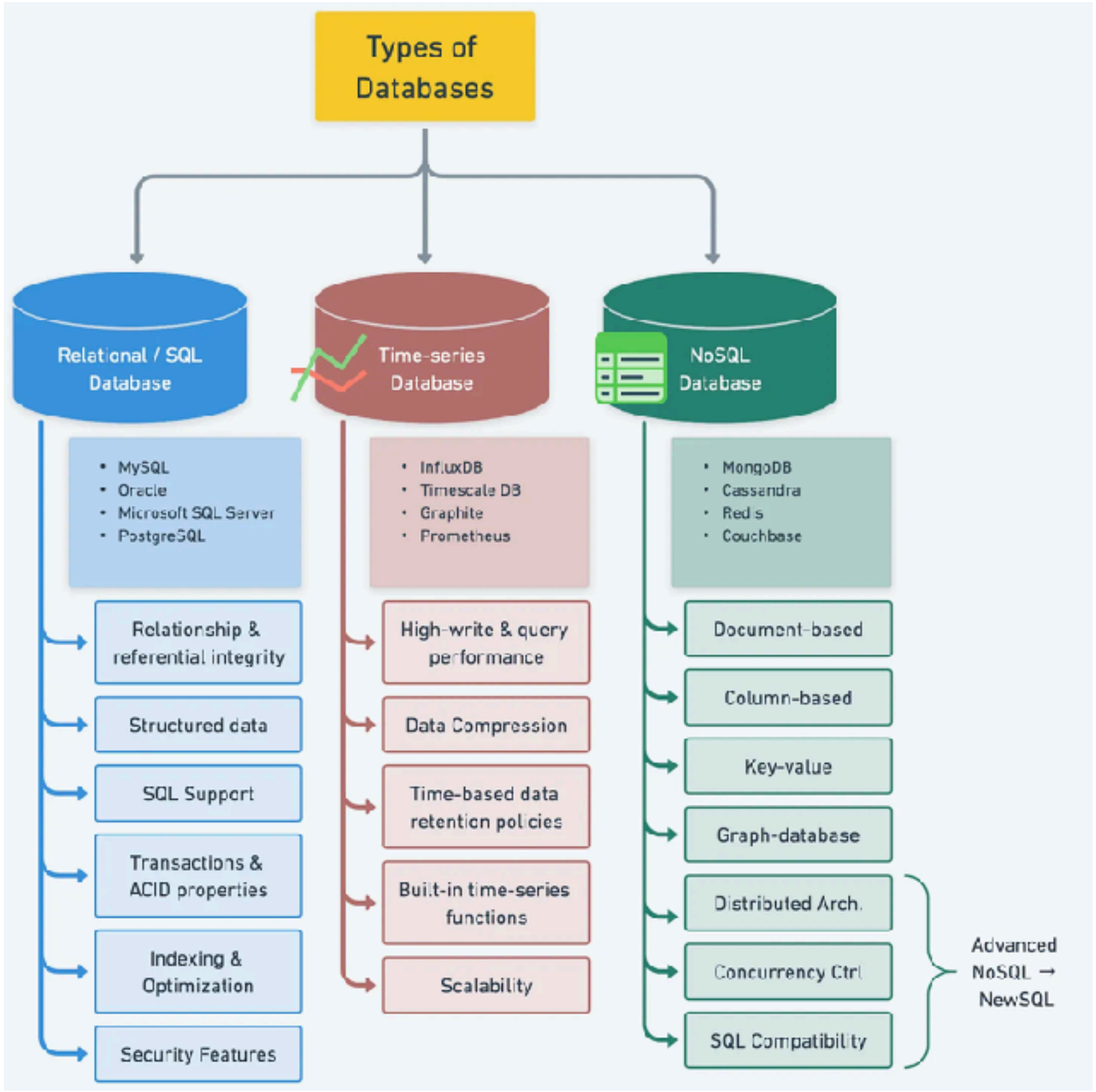


Microsoft®
SQL Server®

NoSQL







https://blog.bytebytego.com/p/understanding-database-types

Relational Database ?

Relational Database ?

Relational **D**ata**B**ase **M**anament **S**ystem

Maintain data in **tables**

Pre-define structure

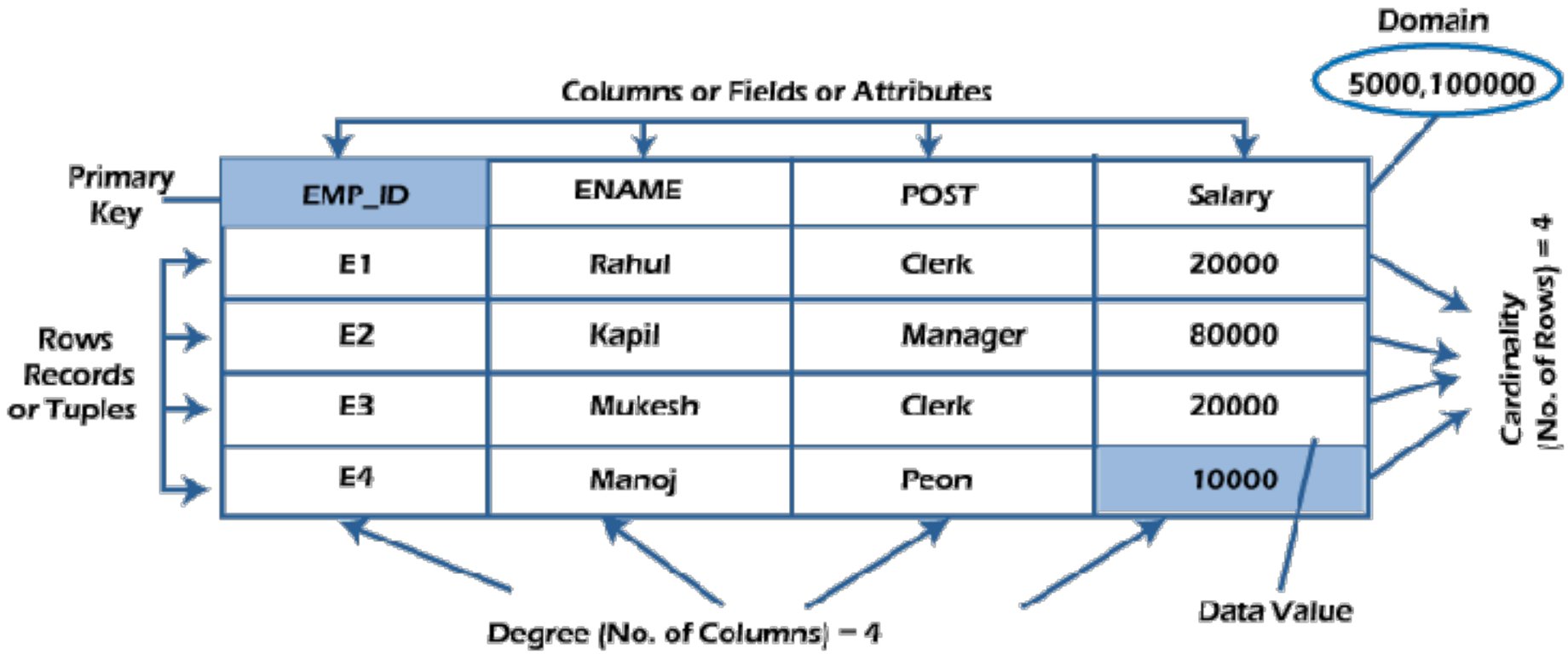
Each table has **Primary key**

Many **columns**

Data is represented in term of **row**

Relational Database ?

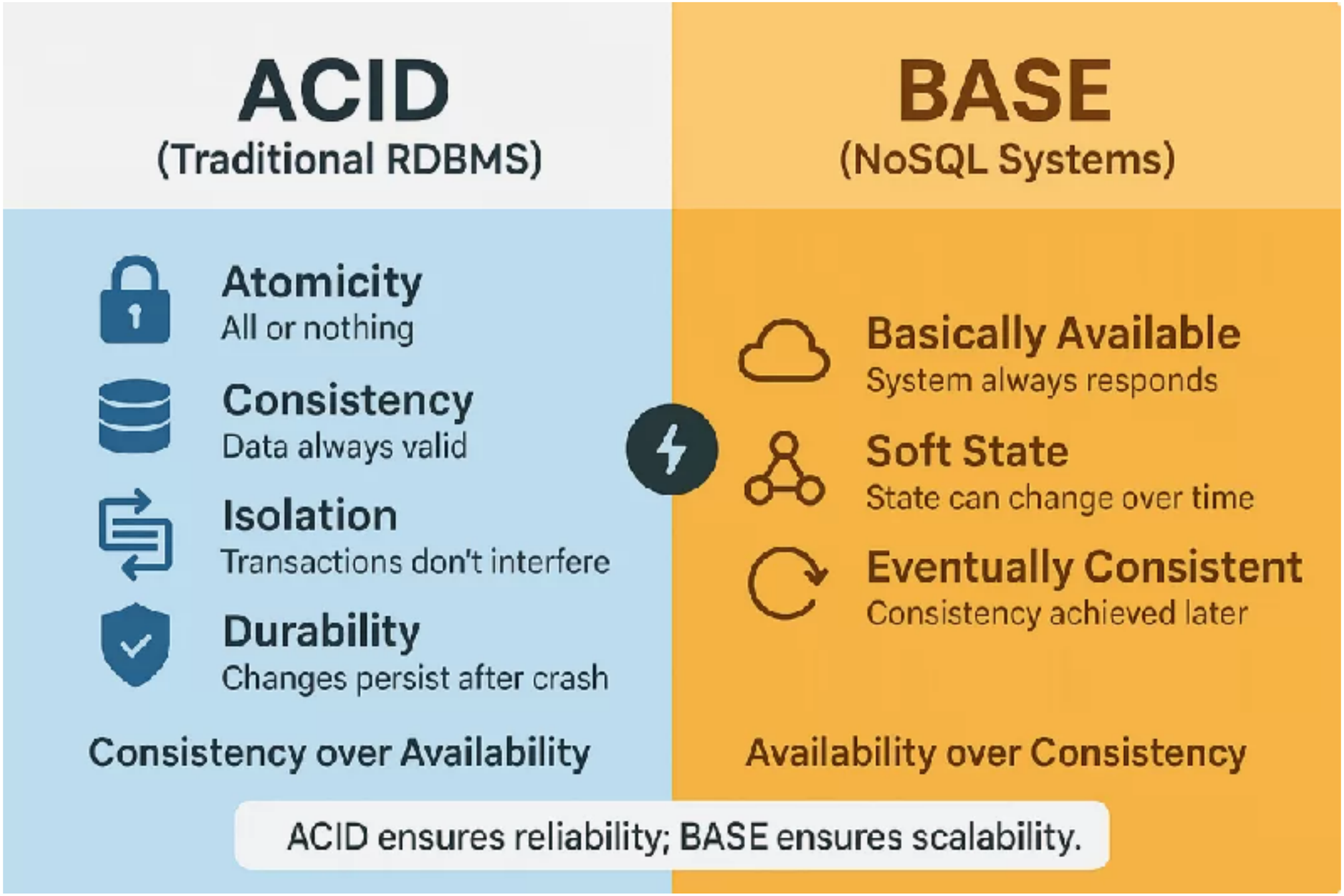
Table = Employee



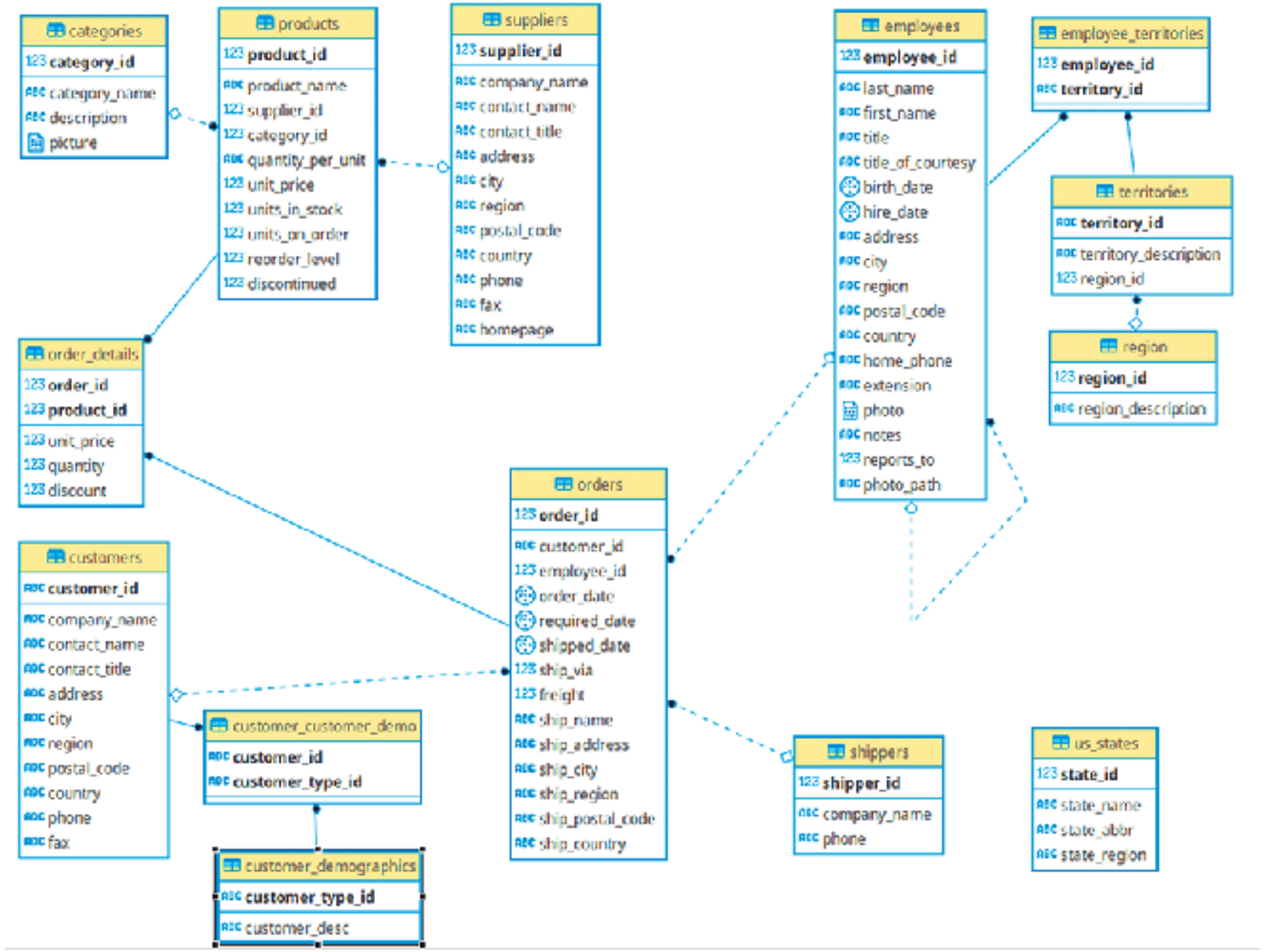
Relational Database

Database	Ease of use	Support complex query	Community support	Licence
MySQL	High	High	Strong	Open source
PostgreSQL	Medium	Very high	Strong	Open source
MSSQL Server	Medium	Very high	Strong	Proprietary
Oracle	Low	Very high	Moderate	Proprietary

ACID vs BASE



Real Design of RDBMS ?



Questions ?

How to write data ?

How to read data ?

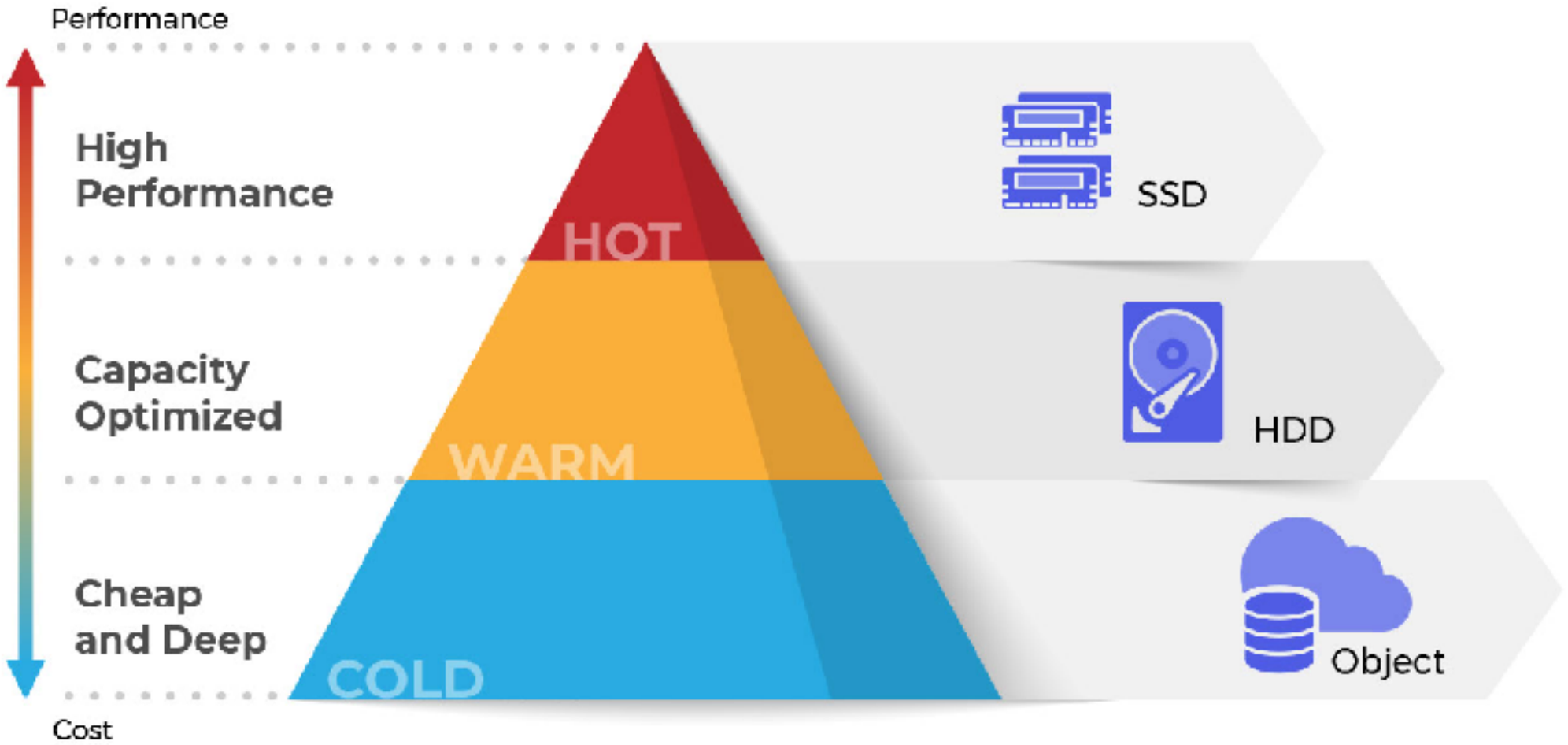
Aggregate data ?

Reporting

Data retention ?

Questions ?

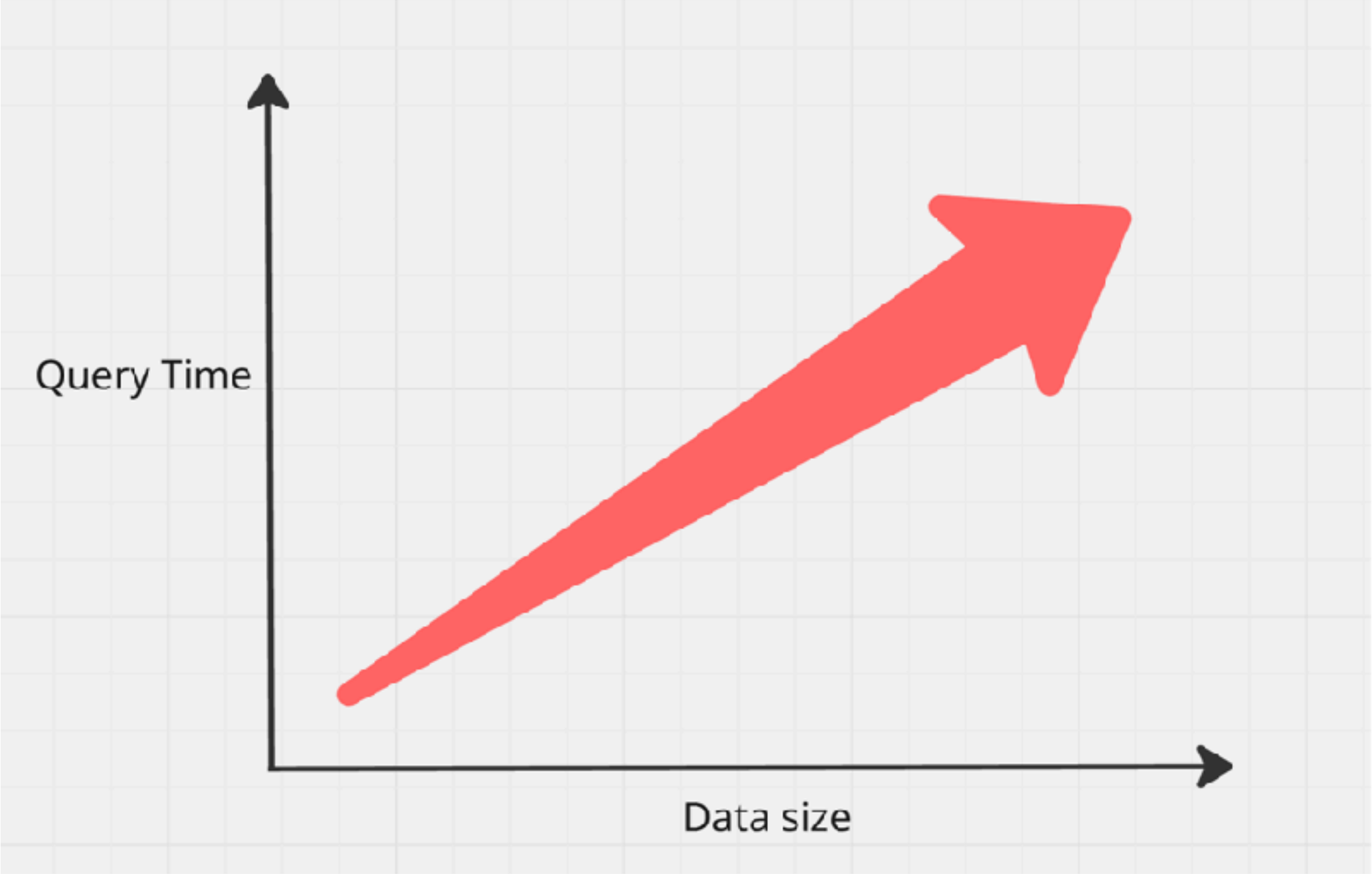
Hot vs Warm vs Cold data ?



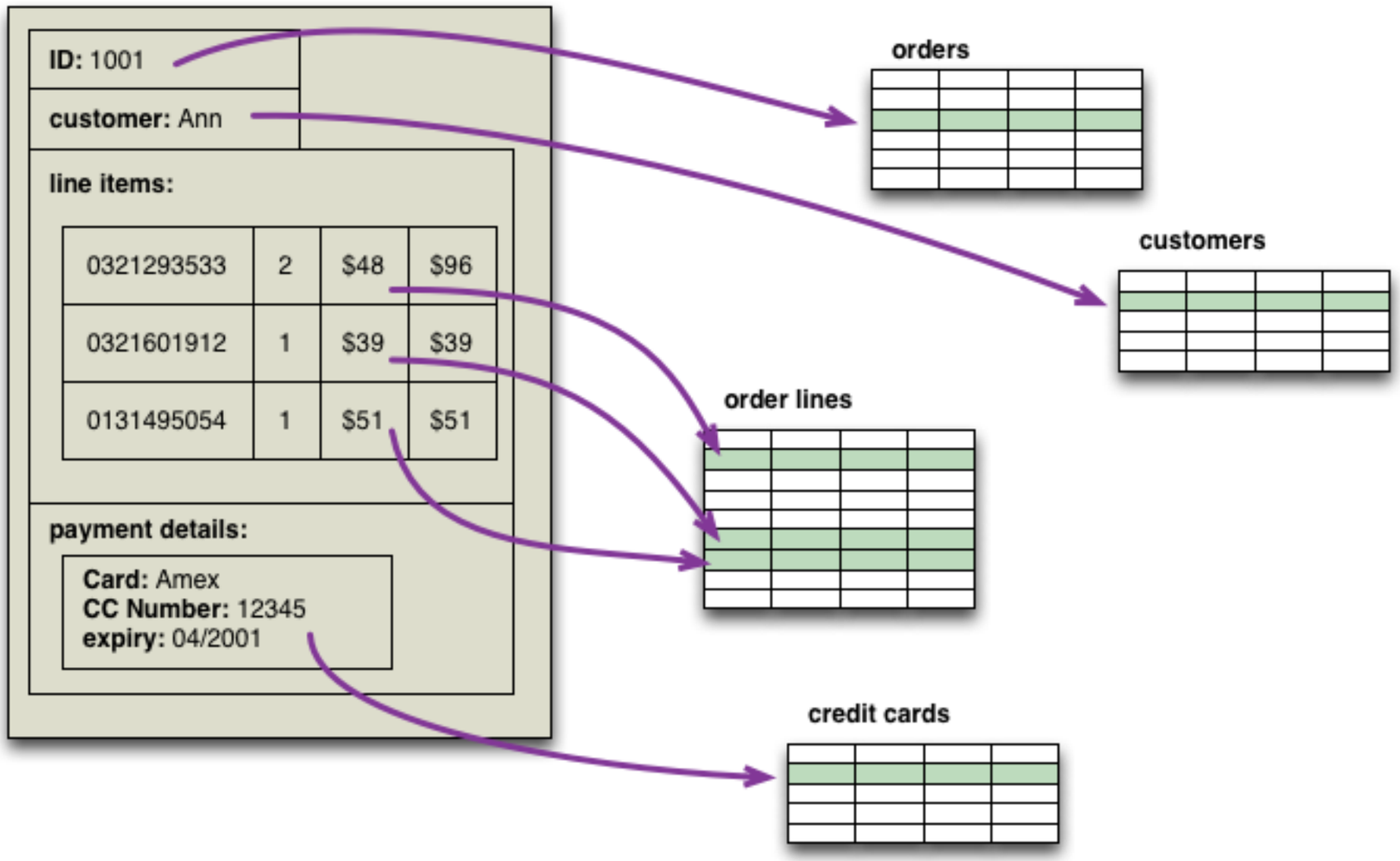
Complex Query ?

```sql
WITH tmp_1 AS
(
SELECT Calc1 =
( (SELECT TOP 1 DataValue
FROM (
SELECT TOP 50 PERCENT DataValue
FROM SOMEDATA
WHERE DataValue IS NOT NULL
ORDER BY DataValue
) AS A
ORDER BY DataValue DESC
) + (SELECT TOP 1 DataValue
FROM (
SELECT TOP 50 PERCENT DataValue
FROM SOMEDATA
WHERE DataValue Is NOT NULL
ORDER BY DataValue DESC
)AS A
ORDER BY DataValue ASC))/2
 ),tmp_2 AS
(SELECT AVG(DataValue) Mean, MAX(DataValue) - MIN(DataValue) AS MaxMinRange
FROM SOMEDATA
),tmp_3 AS
(
SELECT TOP 1 DataValue AS Calc2, COUNT(*) AS Calc2Count
FROM SOMEDATA
GROUP BY DataValue
ORDER BY Calc2Count DESC
)
SELECT Mean, Calc1, Calc2 , MaxMinRange AS [Range]
FROM tmp_1 CROSS JOIN tmp_2 CROSS JOIN tmp_3;
```

Problems ?



NoSQL



https://martinfowler.com/bliki/AggregateOrientedDatabase.html

Polyglot Persistence

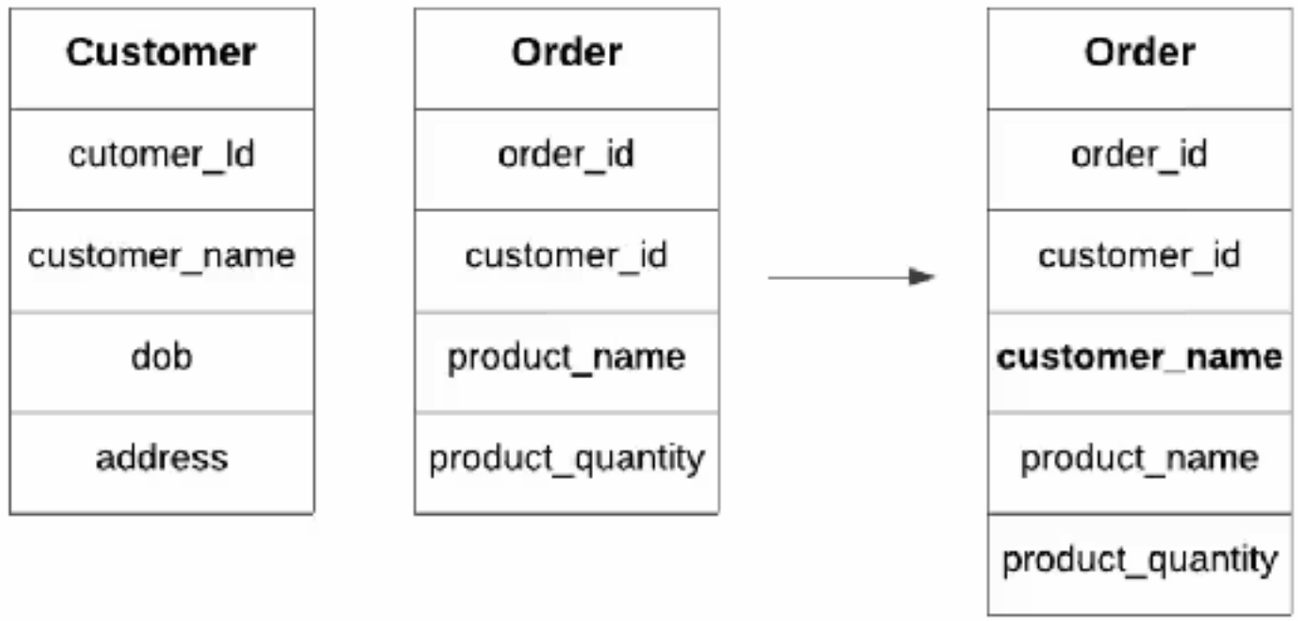
Right tool for the right job

https://martinfowler.com/bliki/PolyglotPersistence.html

Data Modeling

Data Modeling

Normalization
Denormalization
Pre-joins, pre-aggregation
Caching strategies



Normalization ?

Cleansing data
Reorganized data
Remove redundant data
Improve efficient for store data

Organize data in consistent way

Normalization

Divide in to several tables
Link tables with **relationships !!**

Design for write or read ?

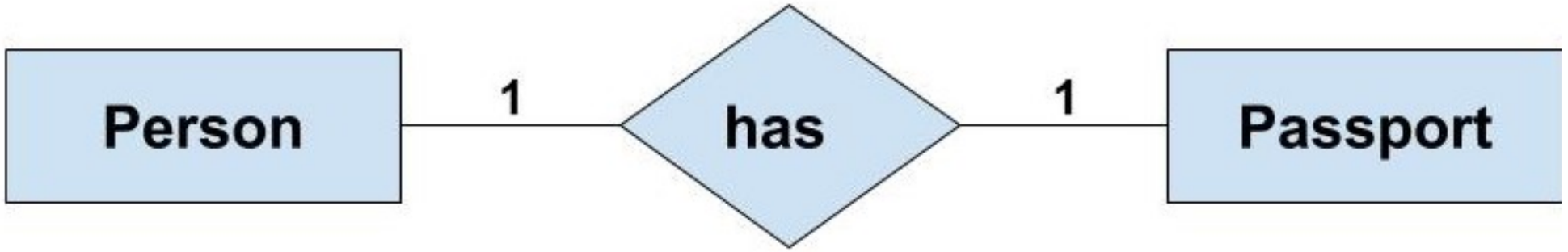
Relationships

Relationship between table

One-to-one

One-to-many

Many-to-many

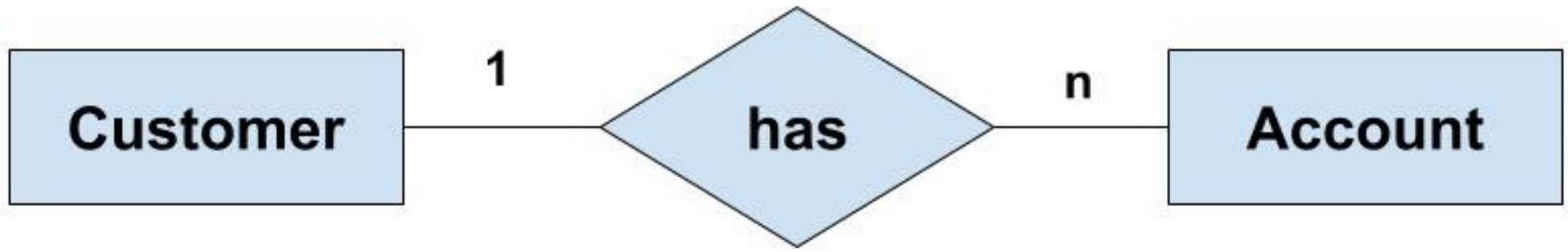


Relationship between table

One-to-one

One-to-many

Many-to-many



Relationship between table

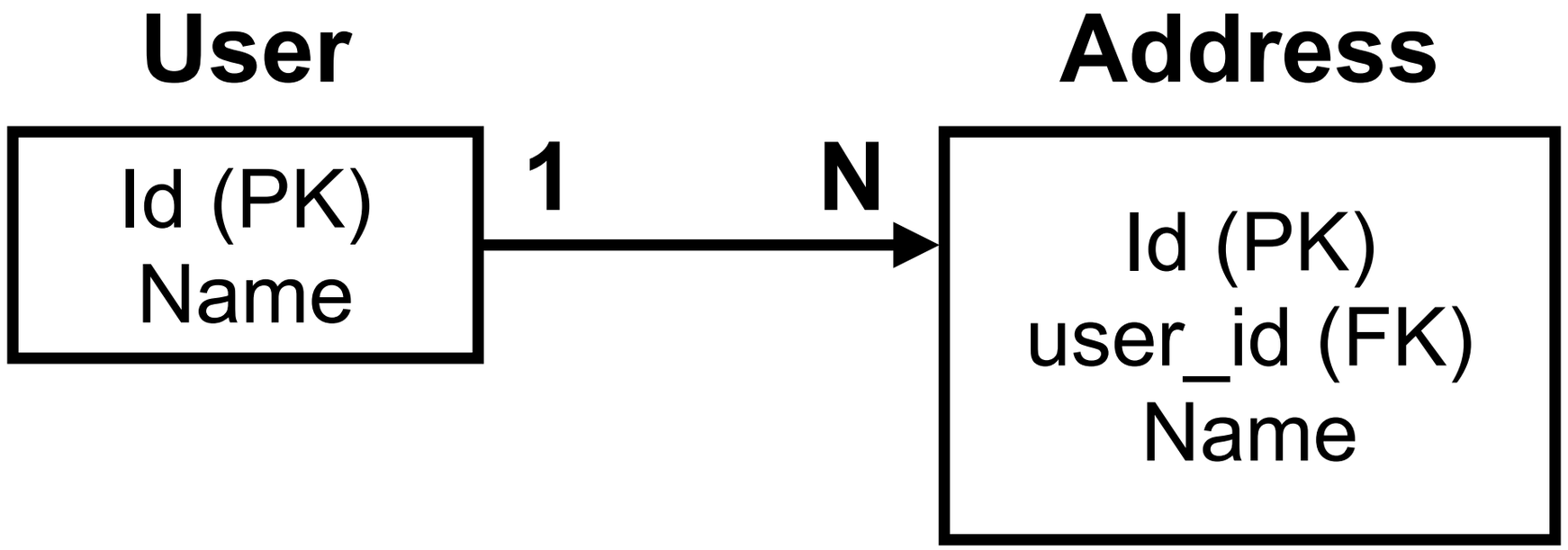
One-to-one

One-to-many

Many-to-many

Relationships

Primary key (PK)
Composite key
Foreign key (FK)



Normalization forms

First Normal form (1NF)

2NF

3NF

4NF

5NF

6NF

First Normal Form (1NF)

No repeat data in table

Every column should only have single value

Every row should be unique

Sample data in 1NF

Name	Address	Gender	T-shirt Size
User 1	Address 1	Male	Large
User 2	Address 2	Female	Small
User 2	Address 2	Female	Medium
User 3	Address 3	Male	Medium

Second Normal Form (2NF)

Apply 1NF
Have one primary key

Problem in 1NF

Name	Address	Gender	T-shirt Size
User 1	Address 1	Male	Large
User 2	Address 2	Female	Small
User 2	Address 2	Female	Medium
User 3	Address 3	Male	Medium

Sample data in 2NF

User ID	Name	Address	Gender
1	User 1	Address 1	Male
2	User 2	Address 2	Female
3	User 3	Address 3	Male

Sample data in 2NF

User ID	T-shirt Size
1	Large
2	Small
2	Medium
3	Medium

Third Normal Form (3NF)

Apply 2NF

It should only be dependent on the primary key

if any changes to the primary key are made, all data that is impacted must be put into a new table

Problem in 2NF

User ID	Name	Address	Gender
1	User 1	Address 1	Male
2	User 2	Address 2	Female
3	User 3	Address 3	Male

Sample data in 3NF

User ID	Name	Address	Gender
1	User 1	Address 1	1
2	User 2	Address 2	2
3	User 3	Address 3	1

Sample data in 3NF

Gender ID	Gender name
1	Male
2	Female

Sample data in 3NF

User ID	T-shirt Size
1	Large
2	Small
2	Medium
3	Medium

3NF is normal form …

Benefits of data normalization

Freeing up space
Improve query response time
Reduce data inconsistency
Easier to use and analyze

Challenges of data normalization

Slower query response rates
Accurate knowledge is required
Add complexity for teams
Denormalization as an alternative !!

Primary Key (PK)

Foreign Key (FK)

Other **Constraints**

Ensure data quality
Prevent inconsistency at the database level

Not Null	Unique	Check	Default

Data Types

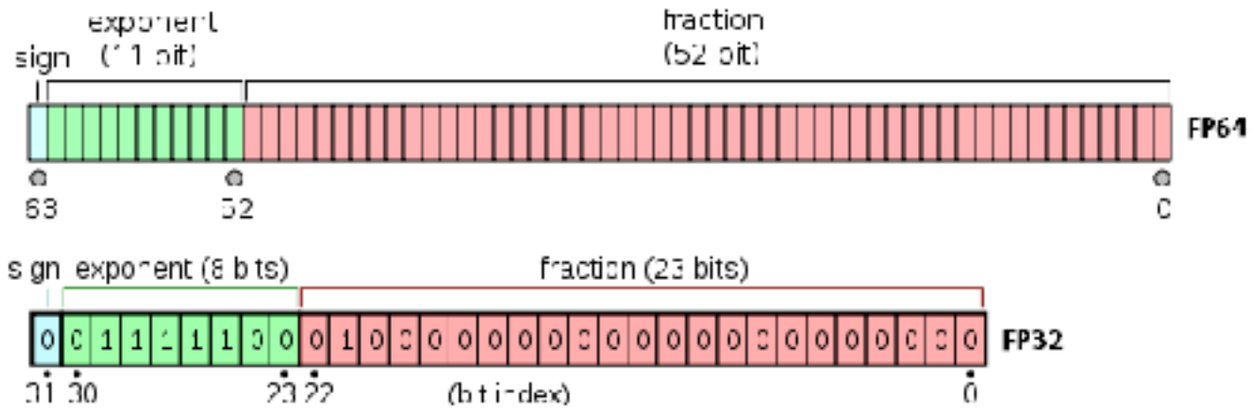
https://www.postgresql.org/docs/current/datatype.html

Data Types ?

Data types	Memory (storage)	Use cases
SMALLINT	2 bytes	Small length data Status code
INTEGER	4 bytes	Standard choice for ID, count
BIGINT/SERIAL	8 bytes	Large data set (Billion records)
UUID	16 bytes	Unique id in distributed system
JSONB	Variable	Binary-parsed JSON

Data Types ?

Data types	Memory (storage)	Use cases
REAL	4 bytes	Single precision floating-point
DOUBLE	8 bytes	Double precision floating-point
NUMERIC	Variable	Exact arithmetic in banking
Money	8 bytes	Fixed-fractional currency amount



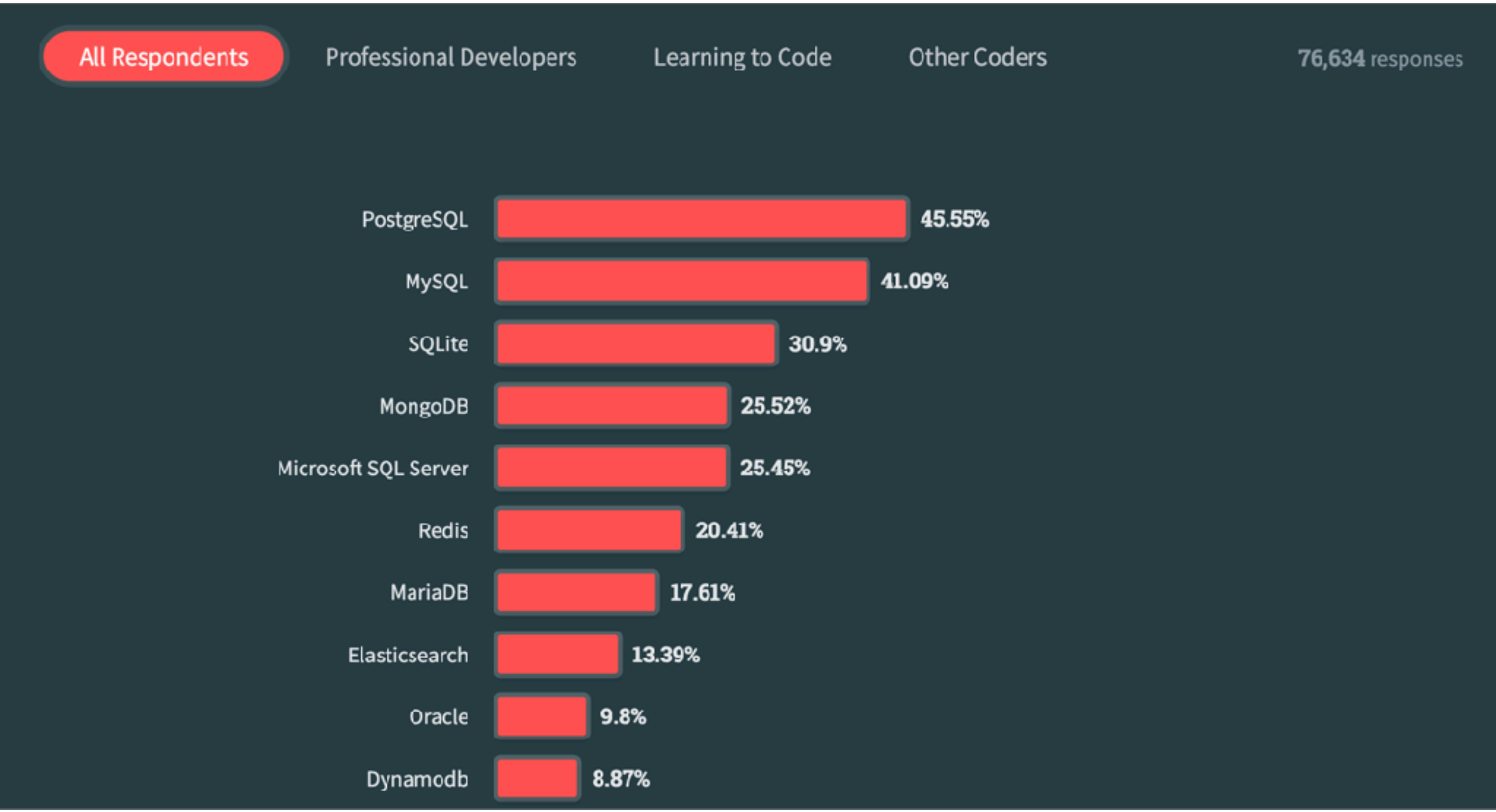
String Data Types ?

Features	Char(n)	Varchar(n)	Text/CLOB
Max length	N characters	Physical	~ 1 GB (Unlimited)
Storage	Blank-padded to n	Content only	Content only + compression
Use cases	Fixed-length data	General use cases	Long-form content, logs

Workshop with PostgreSQL

PostgreSQL

PostgreSQL Database

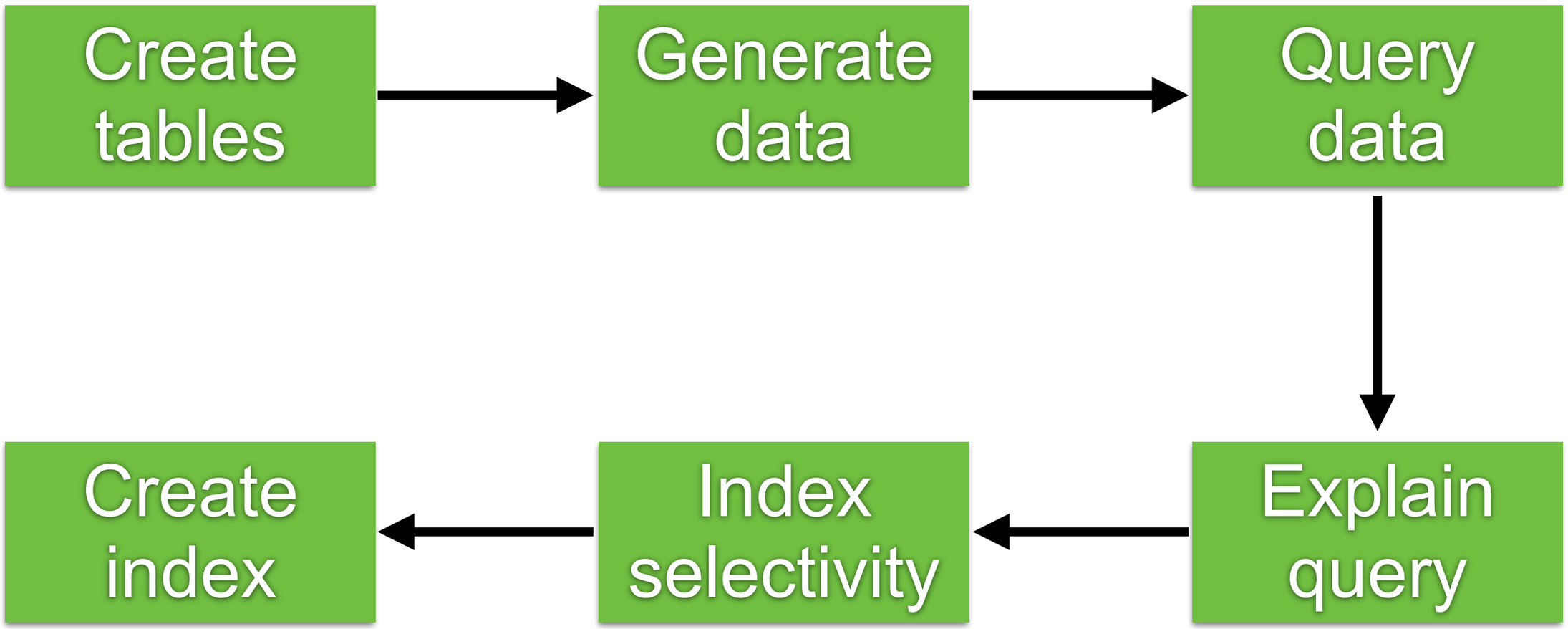


https://survey.stackoverflow.co/2023/#most-popular-technologies-database

Book Store

Book Store

Step in workshop

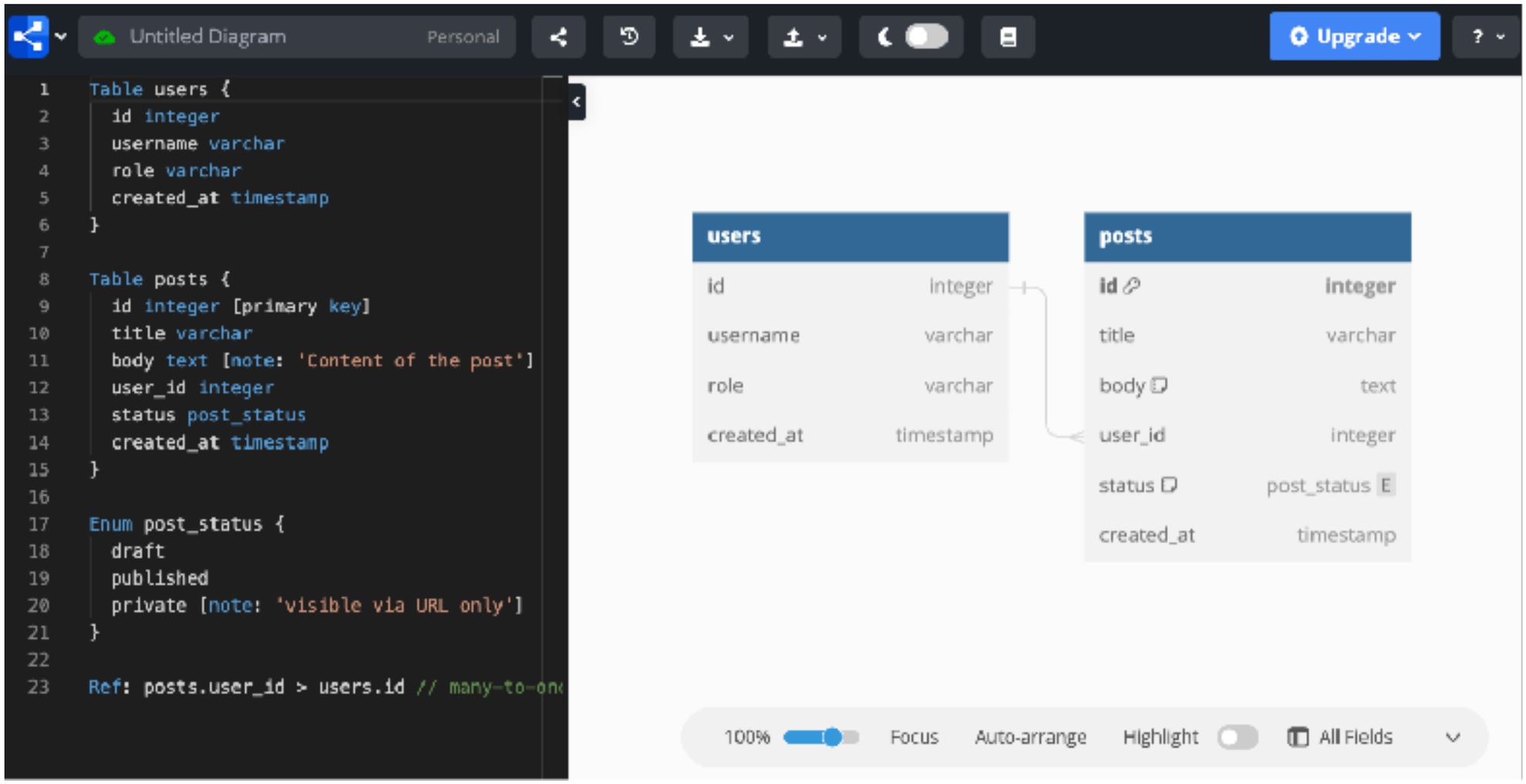


https://github.com/up1/workshop-database/tree/main/workshop/basic-workshop

Design

Design table with DBML

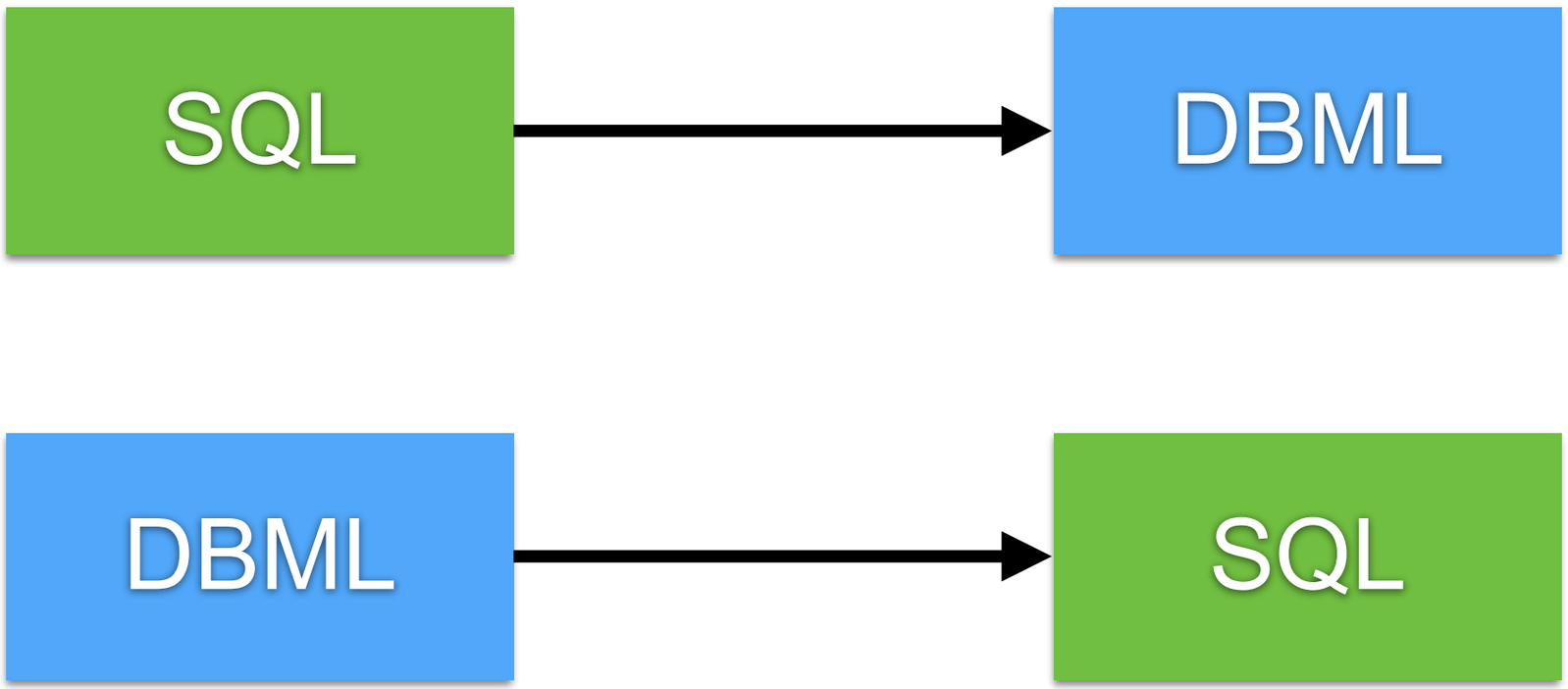
Database Markup Language



https://dbml.dbdiagram.io/home/

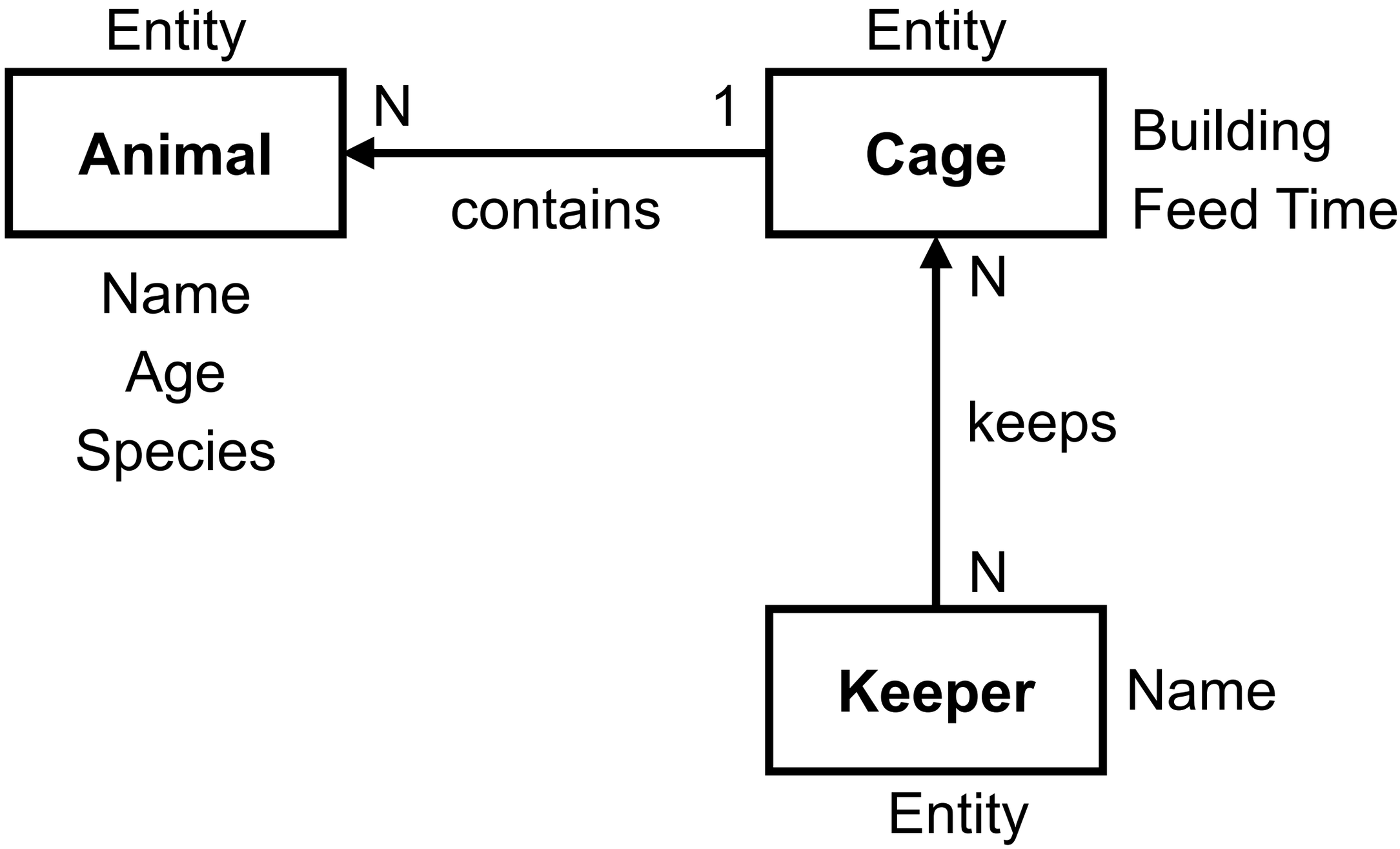
Convert SQL to DBML

$npm install -g @dbml/cli



https://dbml.dbdiagram.io/cli/#convert-a-sql-file-to-dbml

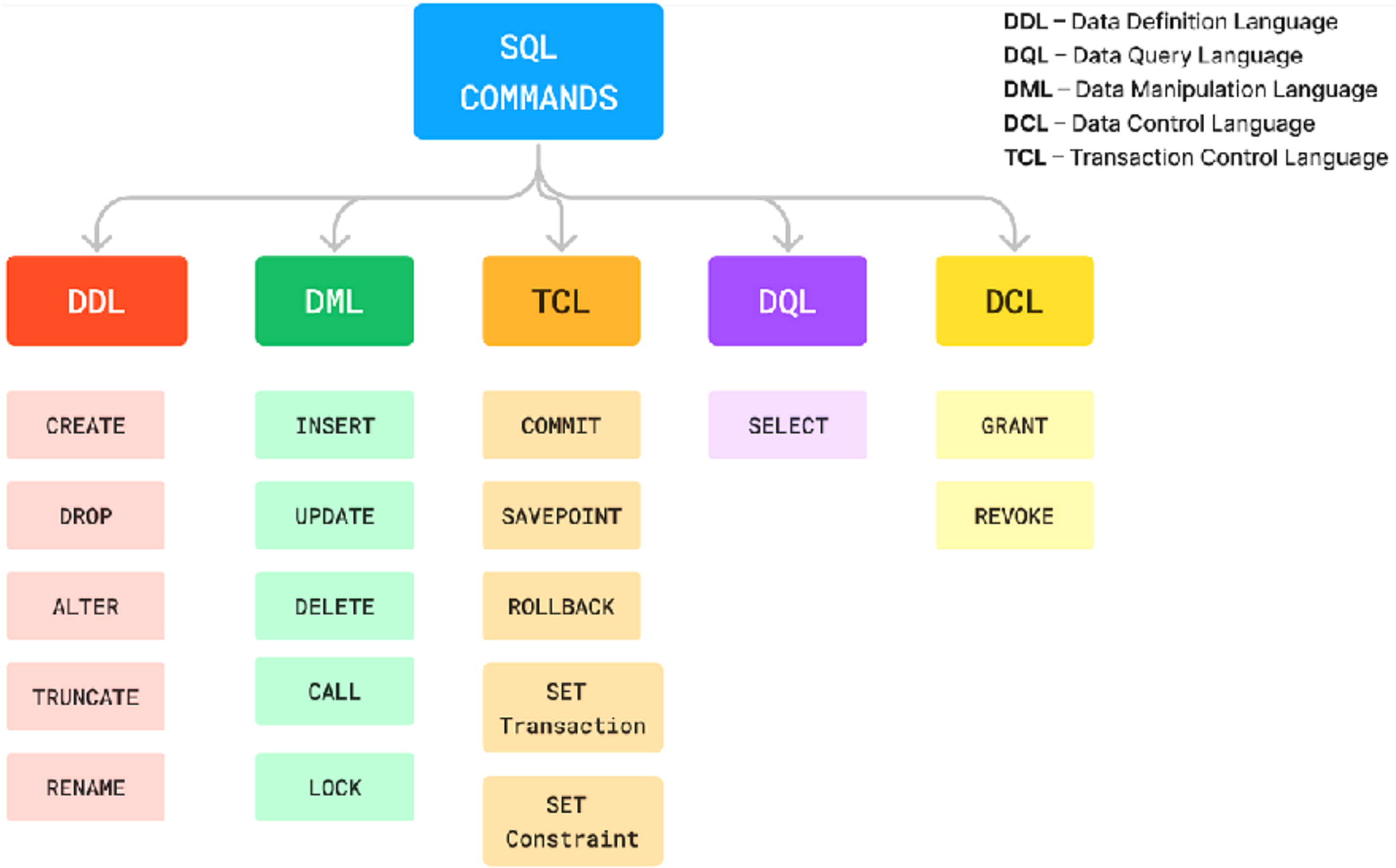
Design with entity relation diagram



How to manage RDBMS ?

Database Language

Data Definition Language (DDL)
Data Manipulation Language (DML)
Data Control Language (DCL)
Transaction Control Language (TCL)



Data Definition Language (DDL)

Create
Alter
Drop
Truncate

Data Manipulation Language (DML)

Select
Insert
Update
Delete

Delete vs Truncate ?

Delete vs Truncate

Delete

Row-level operation

Mark every row matching the `WHERE`-clause as deleted

Slow operation

Truncate

Table operation

Empty the table

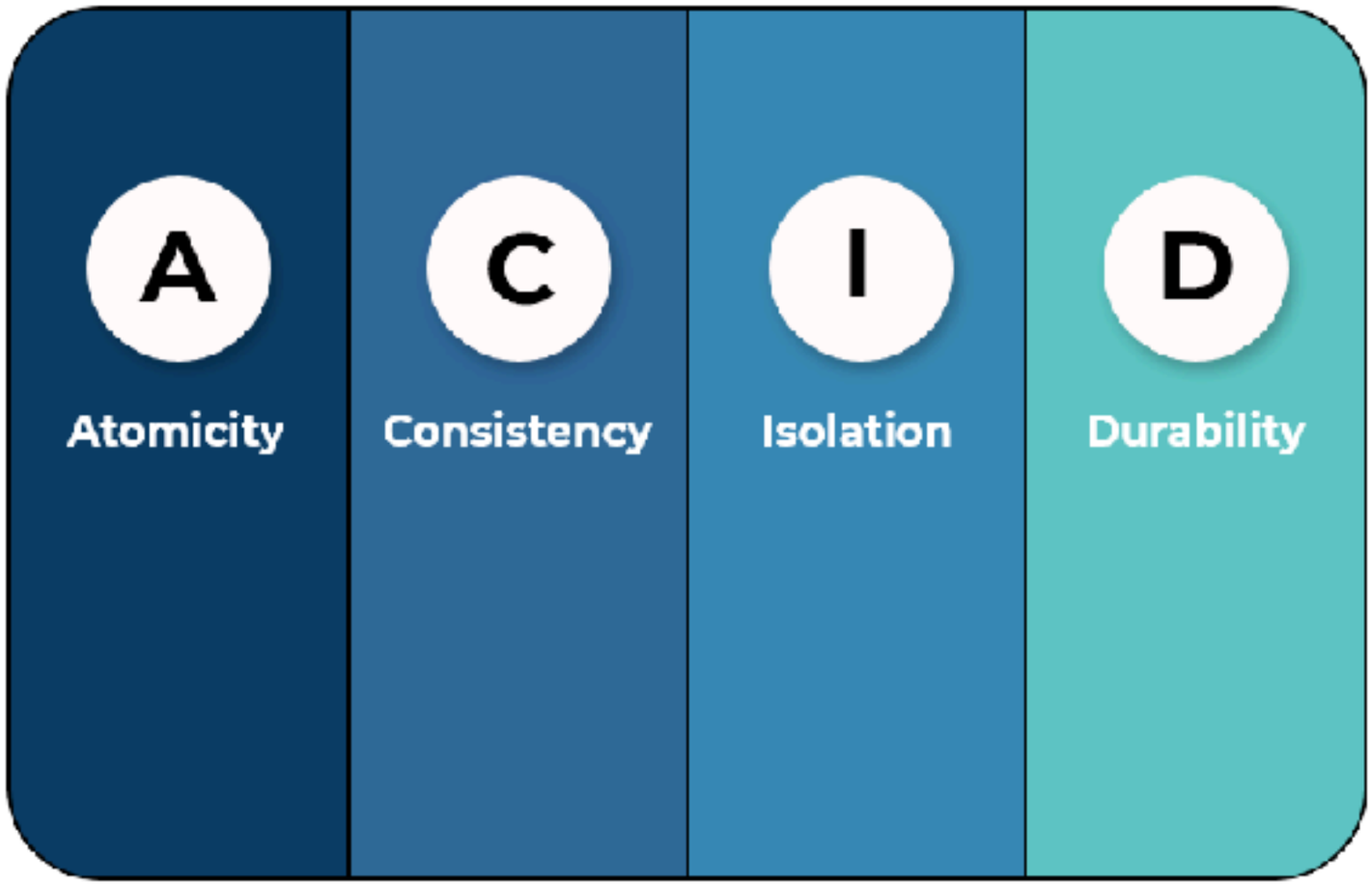
Start a new data file

ACID in RDBMS

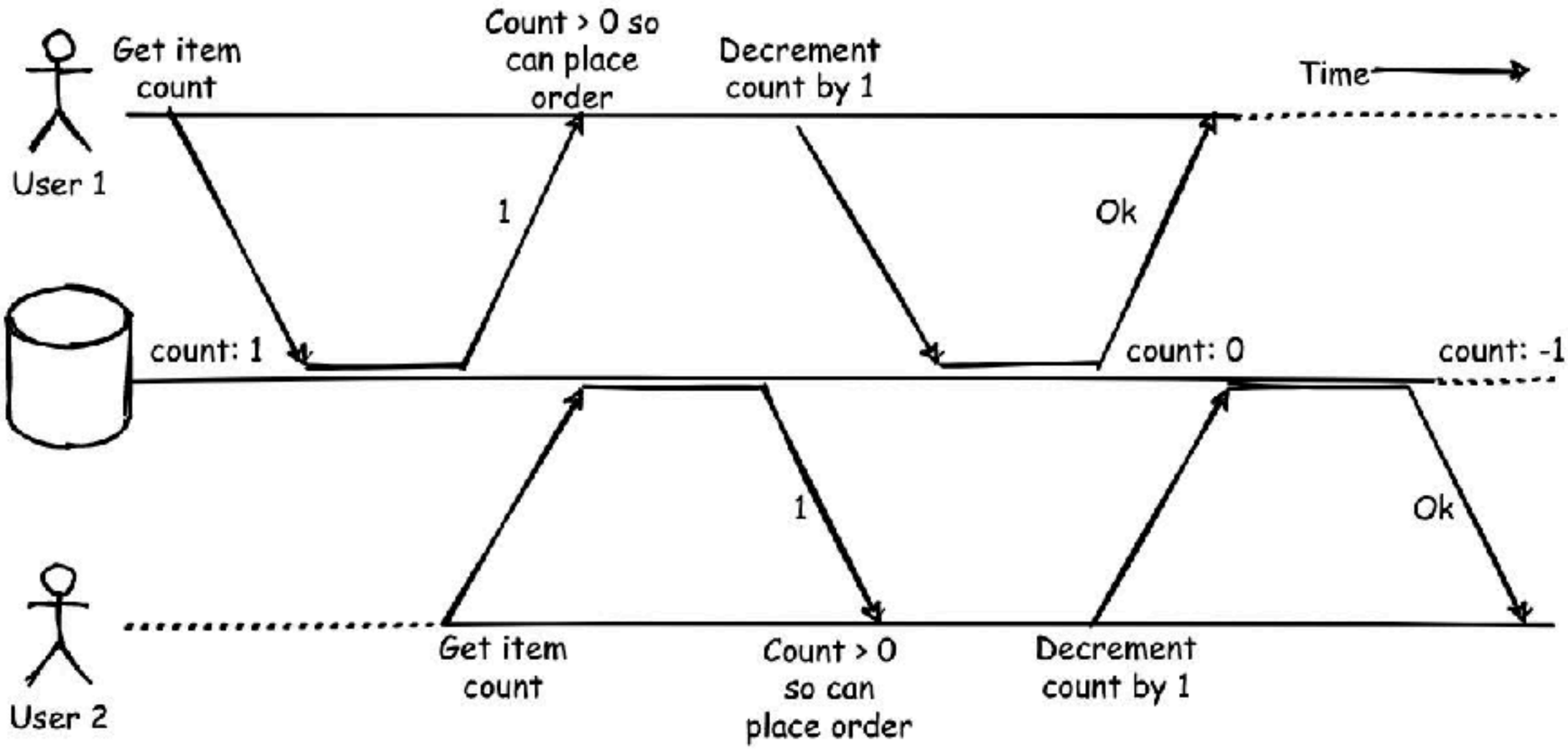
https://en.wikipedia.org/wiki/ACID

ACID in RDBMS

Ensure database in valid state even in the event of unexpected errors

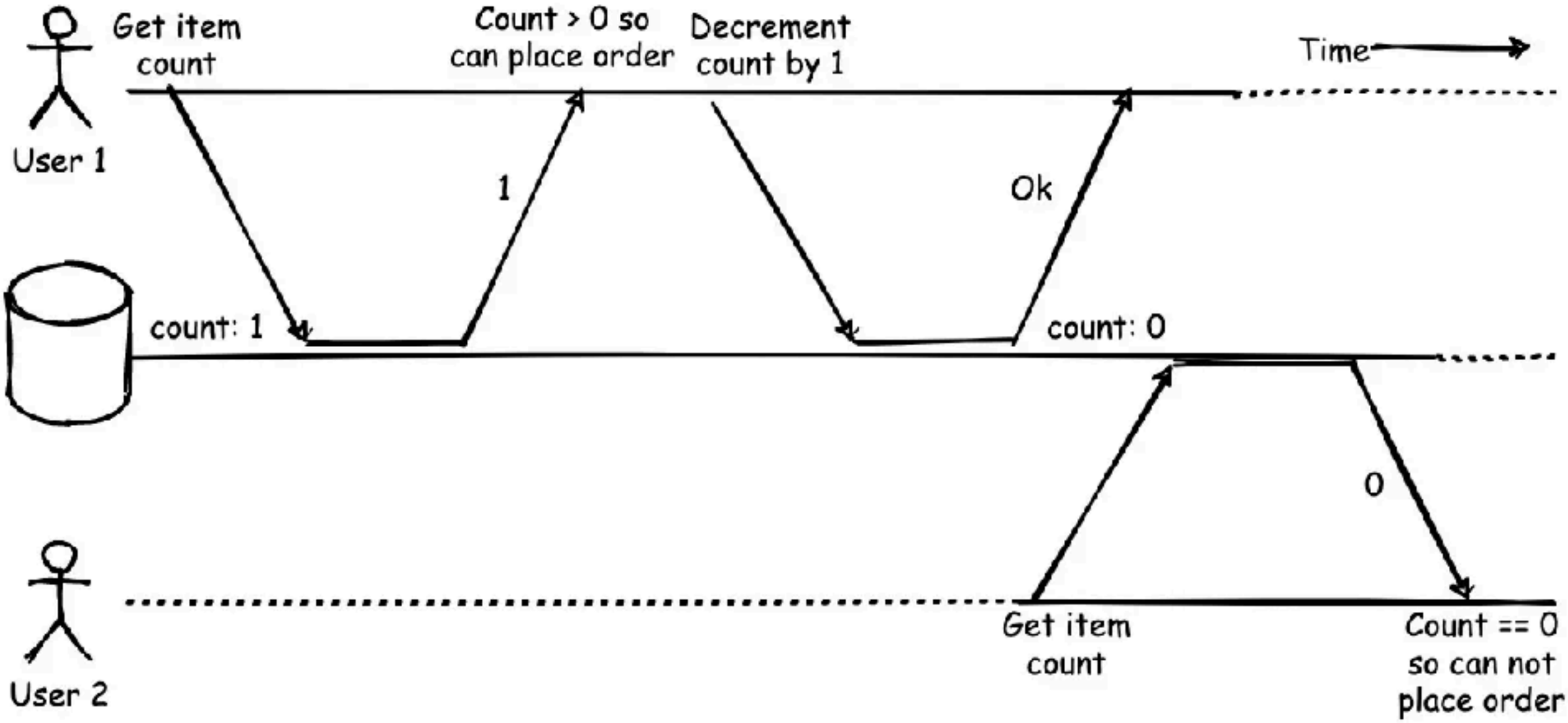


Race condition



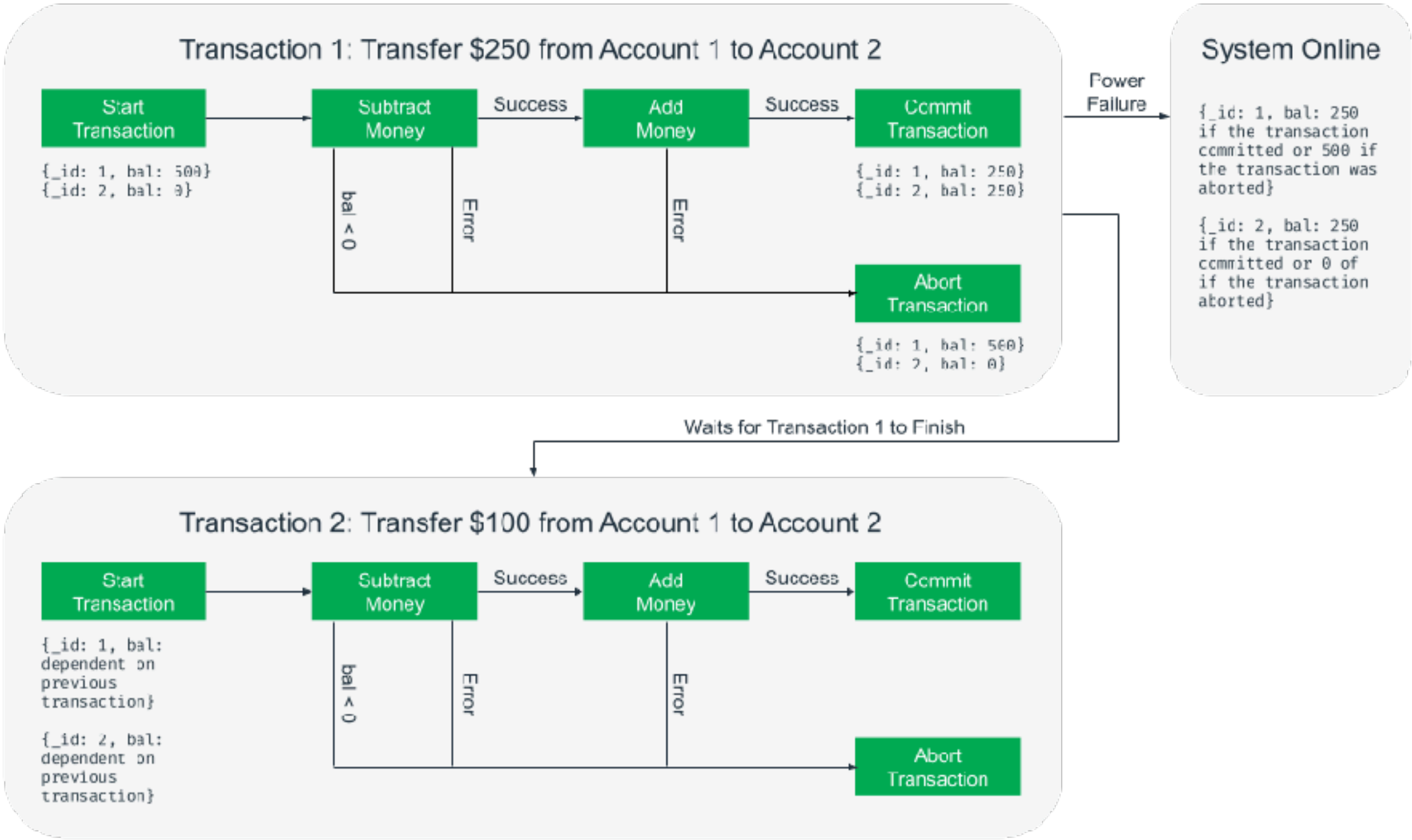
https://en.wikipedia.org/wiki/Race_condition#Example

Sequencial



https://www.mongodb.com/basics/acid-transactions

ACID in RDBMS



https://www.mongodb.com/basics/acid-transactions

SQL
Structure **Q**uery **L**anguage

SQL

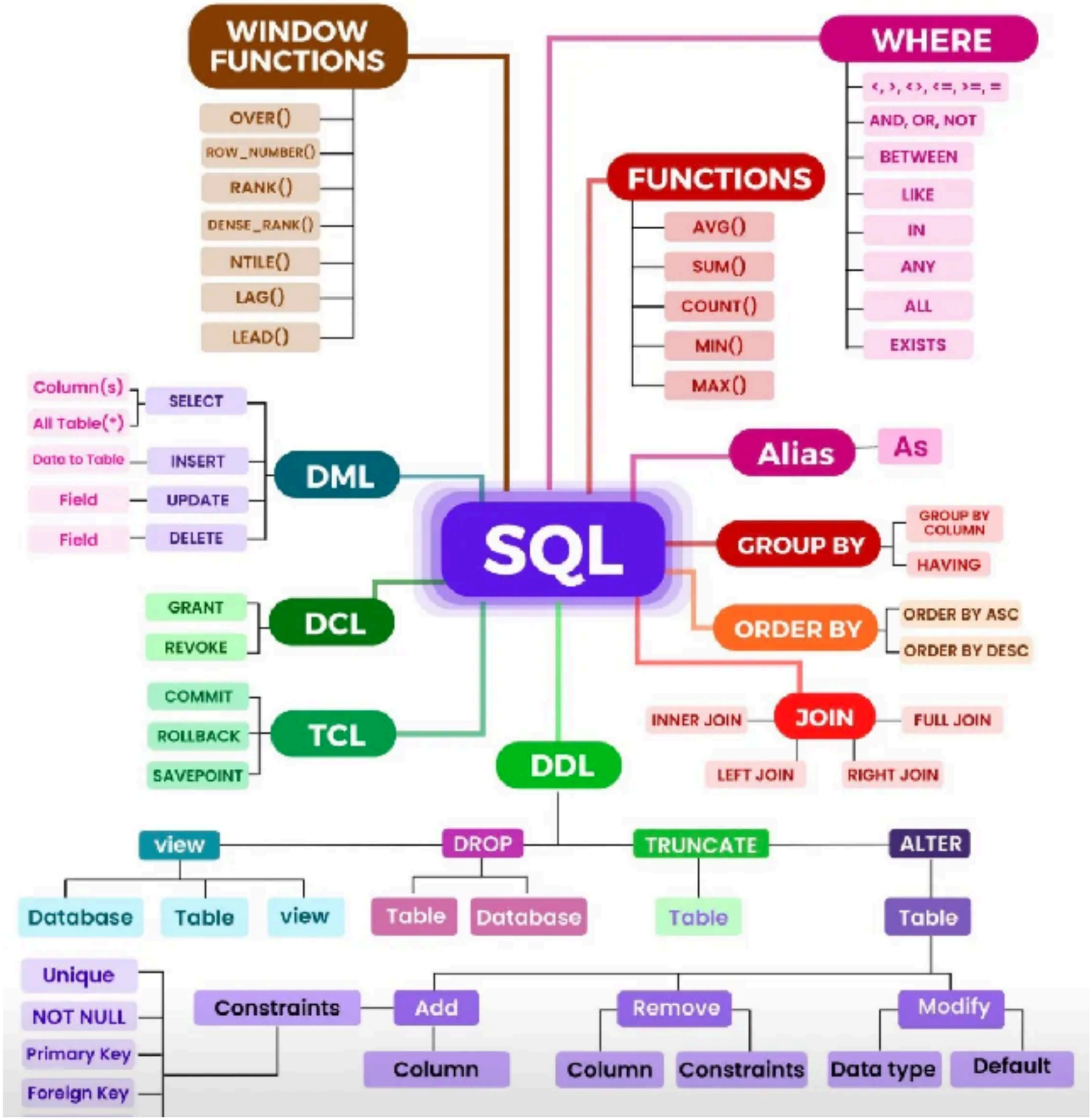
Maintain data in **tables**

Pre-define structure

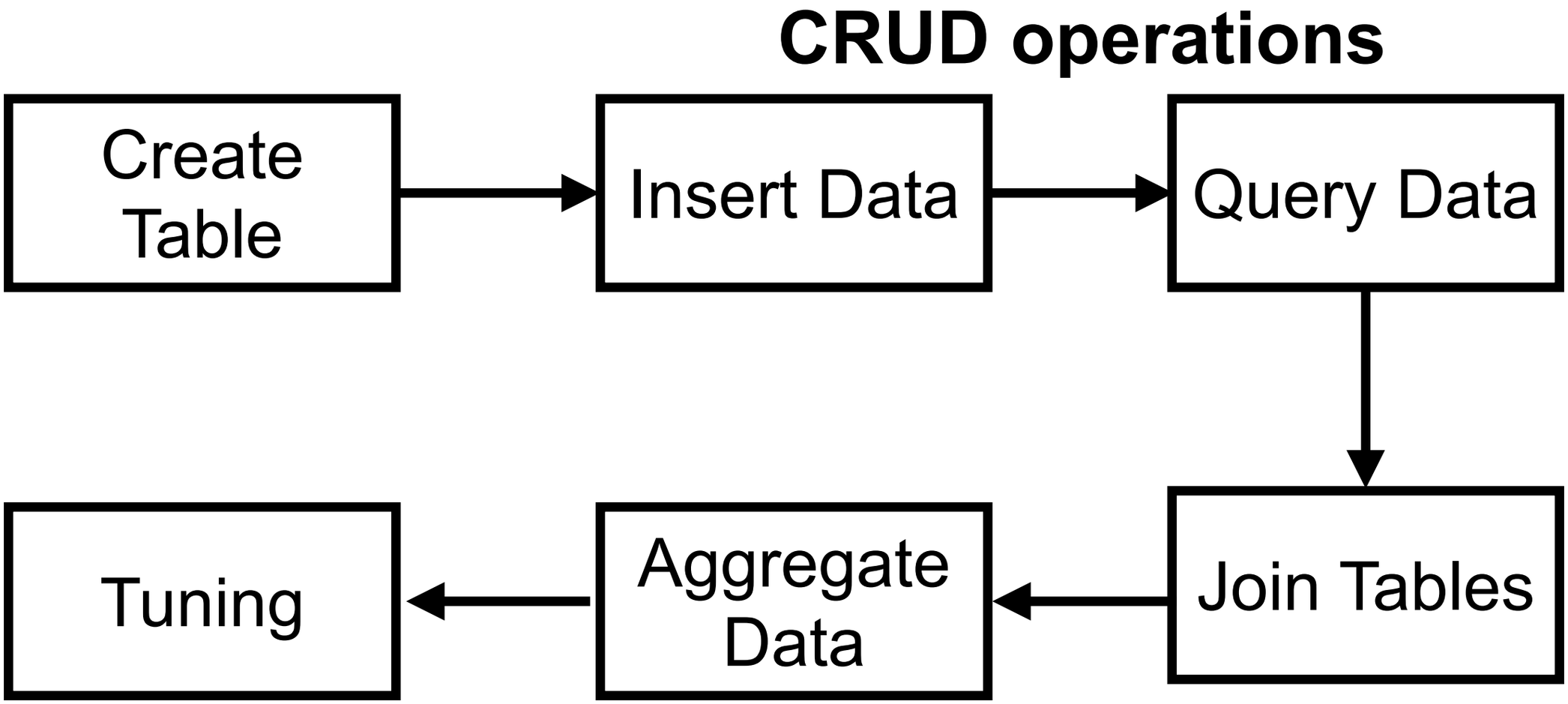
Each table has **Primary key**

Many **columns**

Data is represented in term of **row**



Learning paths

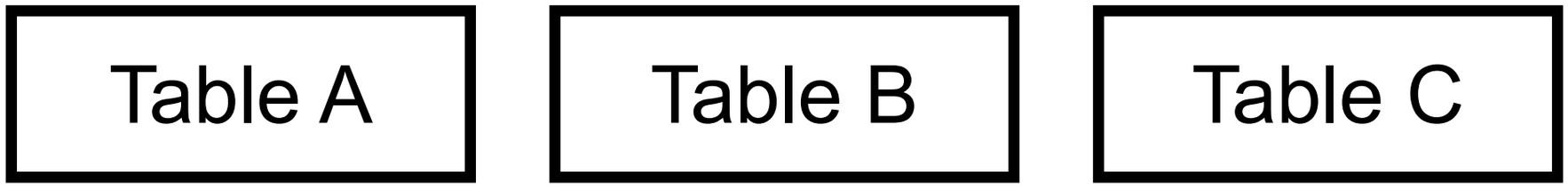


Query Data

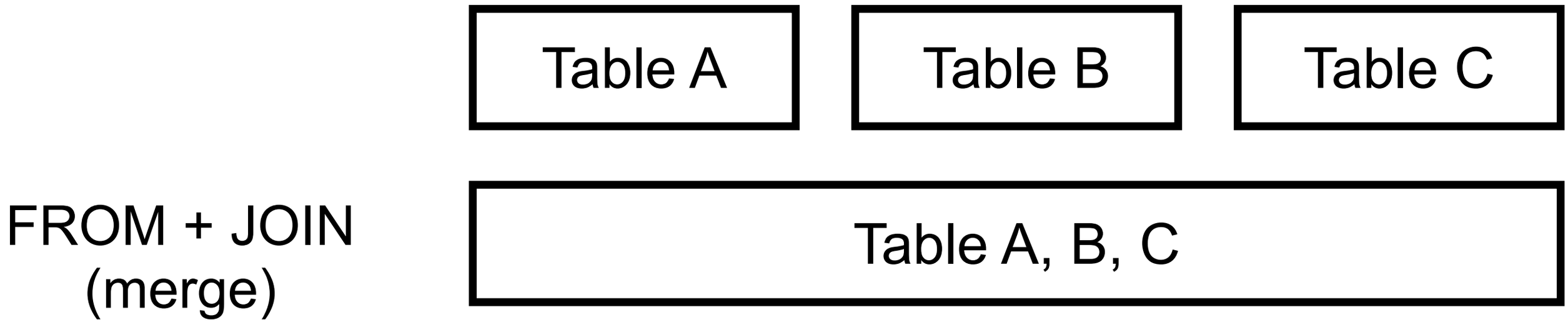
SELECT columns
FROM table
WHERE condition A
AND condition B

GROUP BY columns
ORDER BY columns **ASC|DESC**

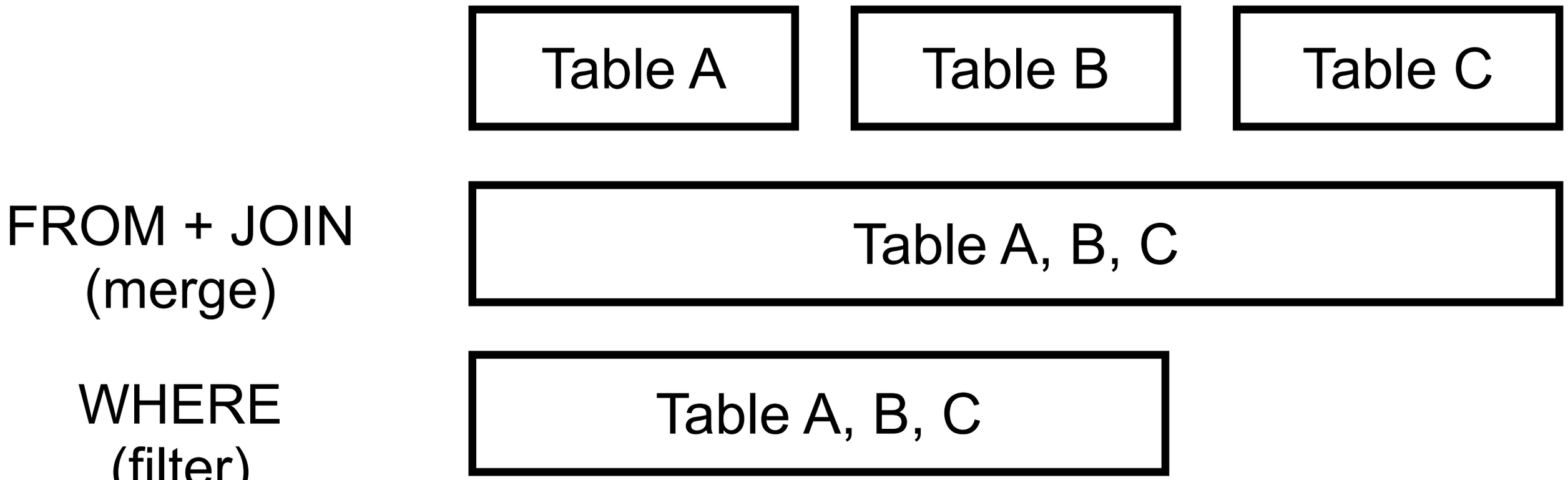
Select Data process



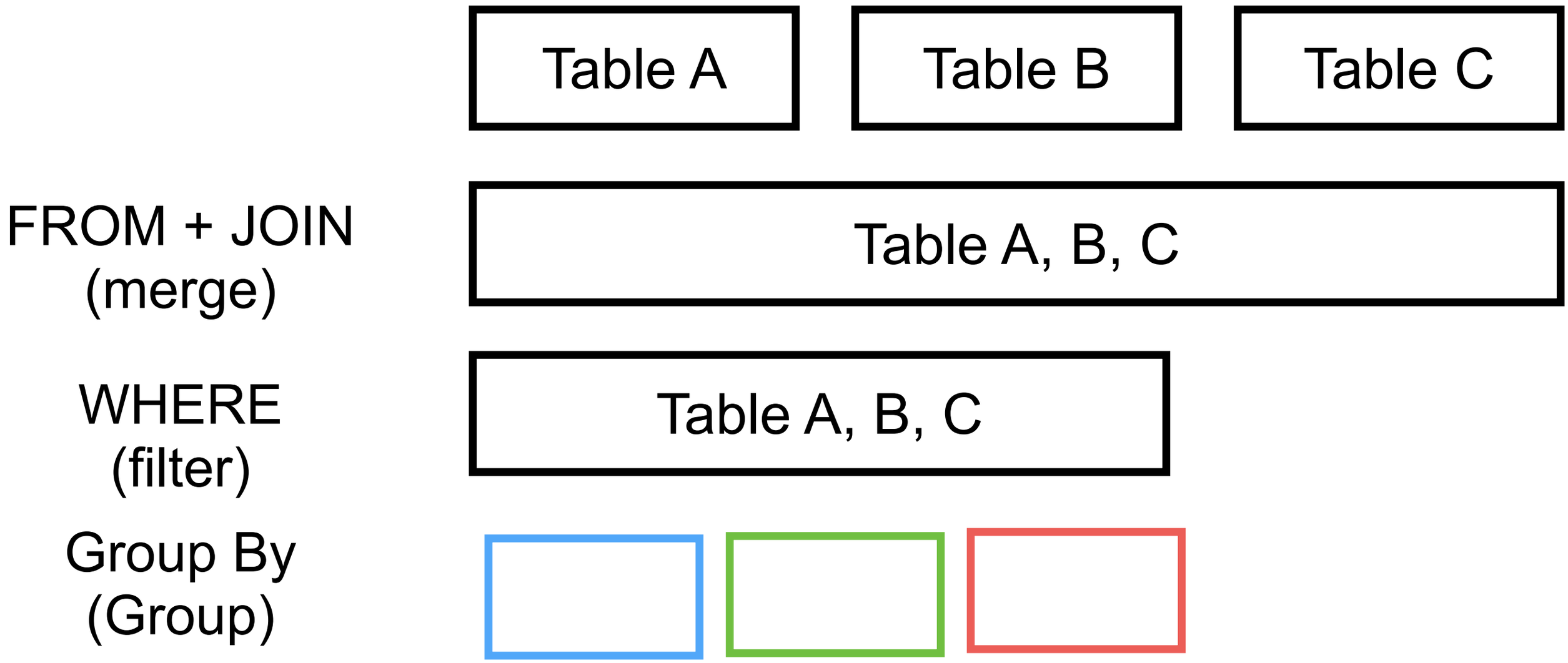
Select Data process



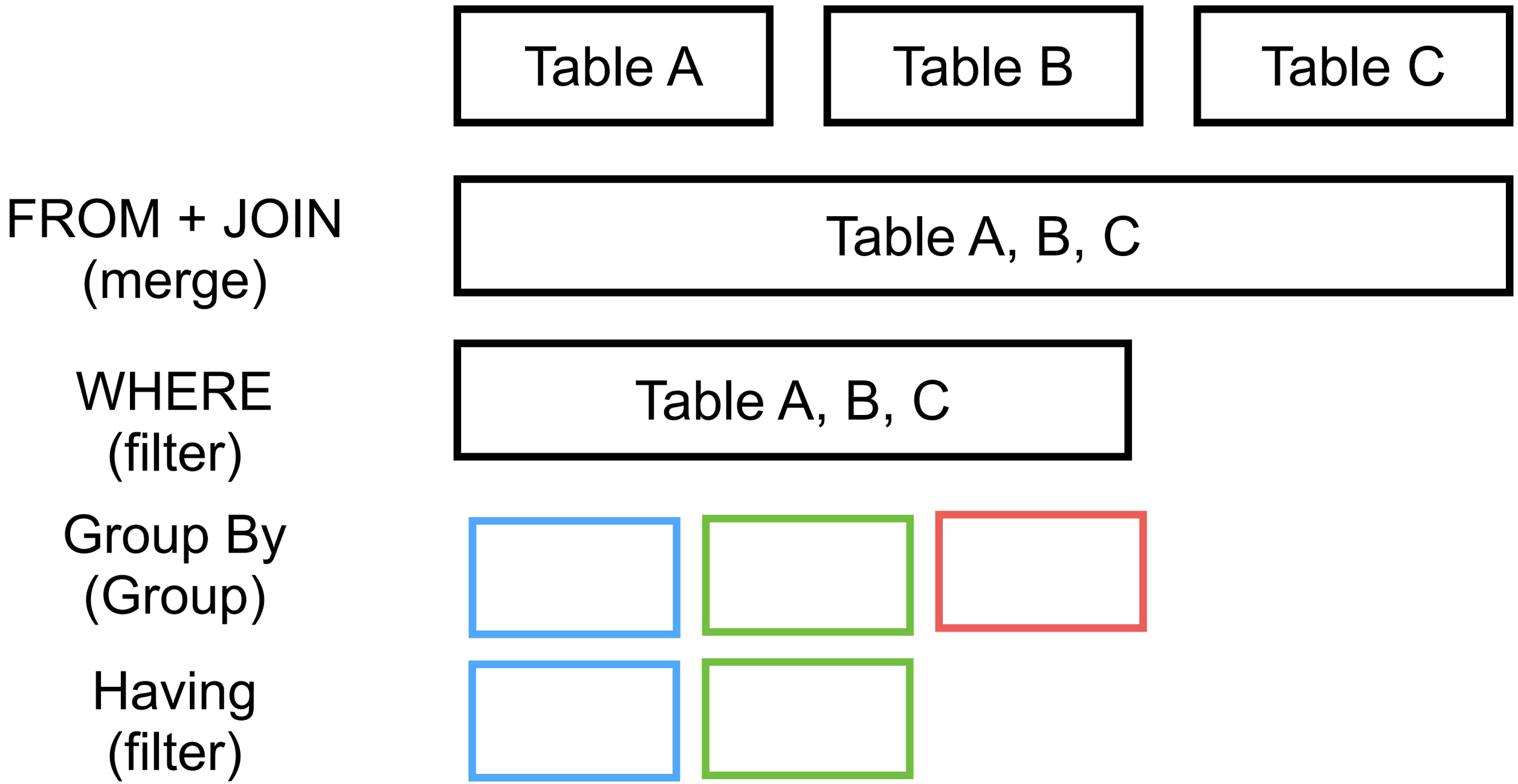
Select Data process



Select Data process



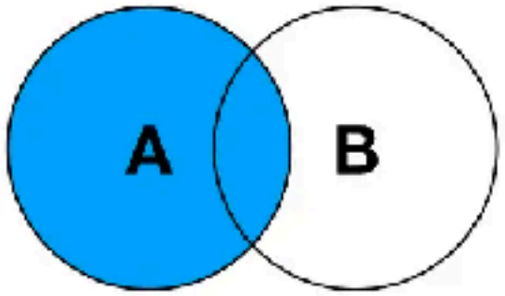
Select Data process



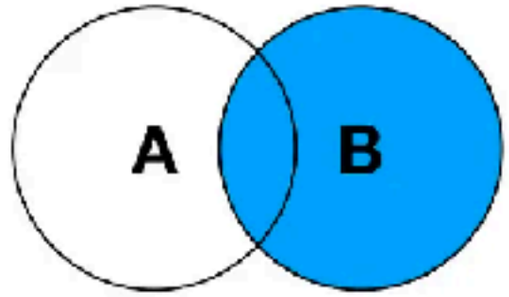
Joining

https://www.postgresql.org/docs/current/tutorial-join.html

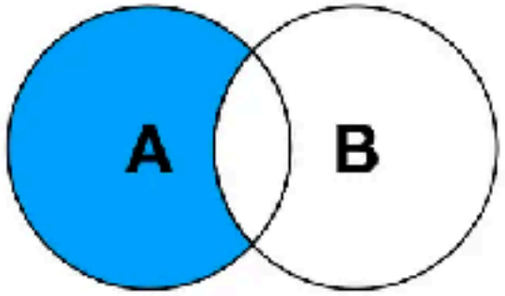
SQL JOINS



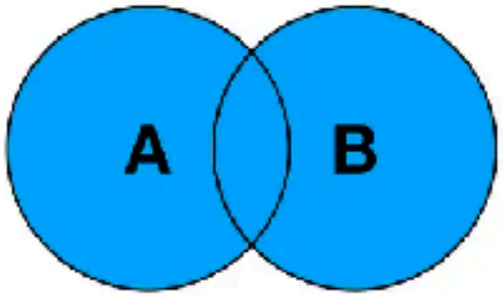
LEFT JOIN



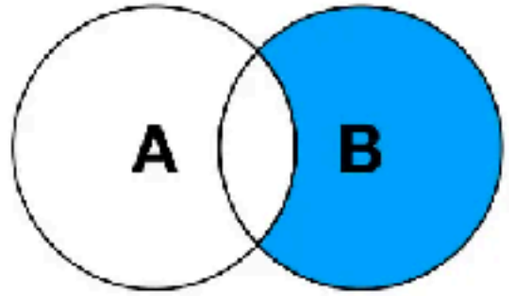
RIGHT JOIN



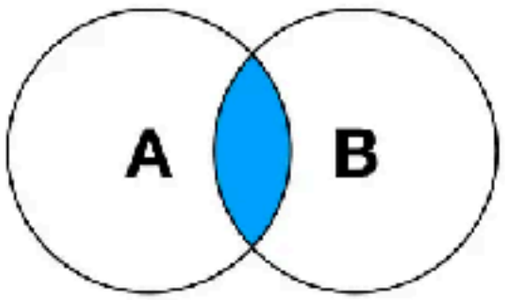
LEFT JOIN EXCLUDING
INNER JOIN



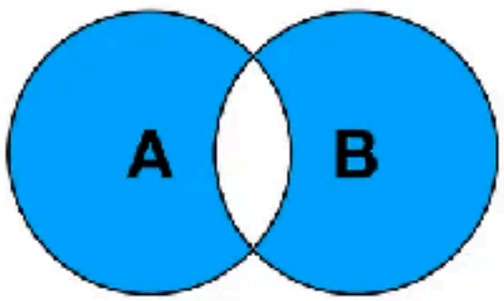
FULL OUTER JOIN



RIGHT JOIN EXCLUDING
INNER JOIN



INNER JOIN



FULL OUTER JOIN EXCLUDING
INNER JOIN

Self-join

Join in PostgreSQL

Left join

Inner join

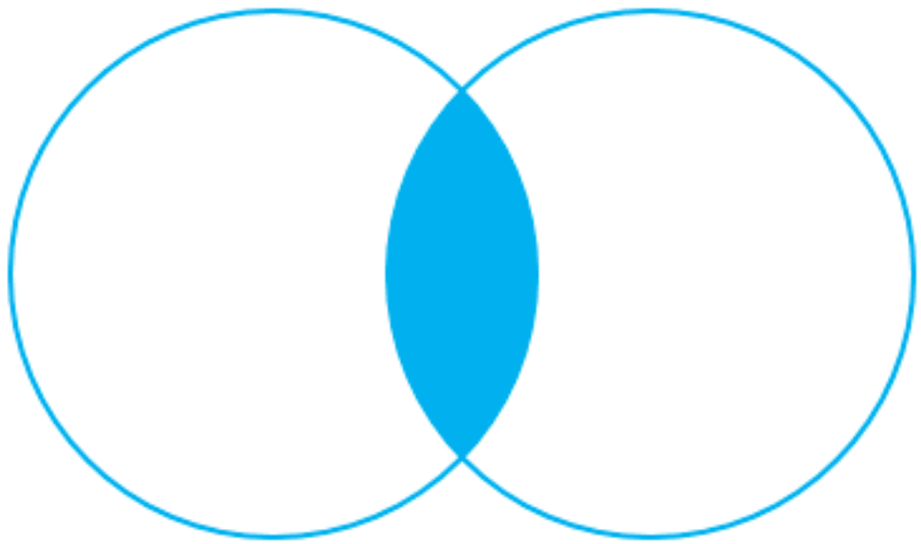
Right join

Full outer join

Workshop

Basket A	Basket B
Orange	Apple
Banana	Watermelon
Cucumber	Pear

Inner join



INNER JOIN

Inner join

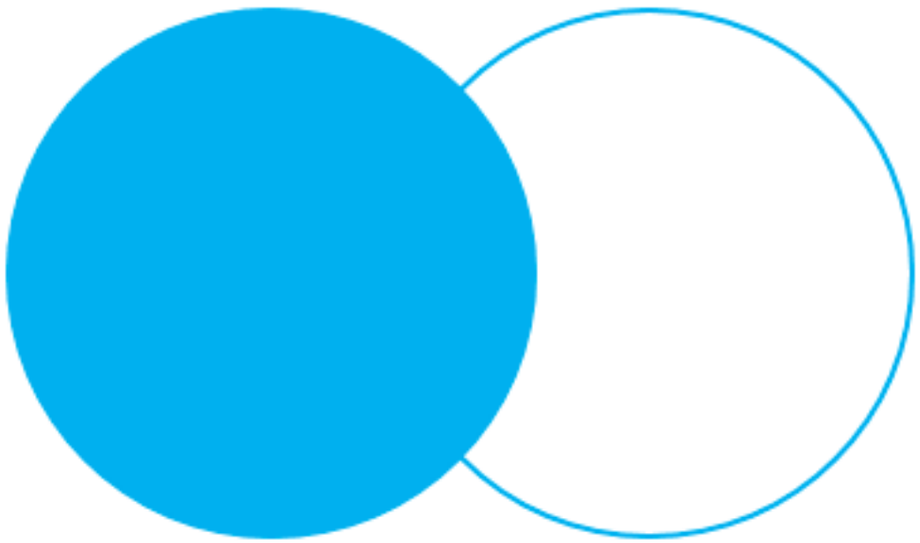
Basket A

Apple
Orange
Banana
Cucumber

Basket B

Orange
Apple
Watermelon
Pear

Left join (1)



LEFT OUTER JOIN

Left join (1)

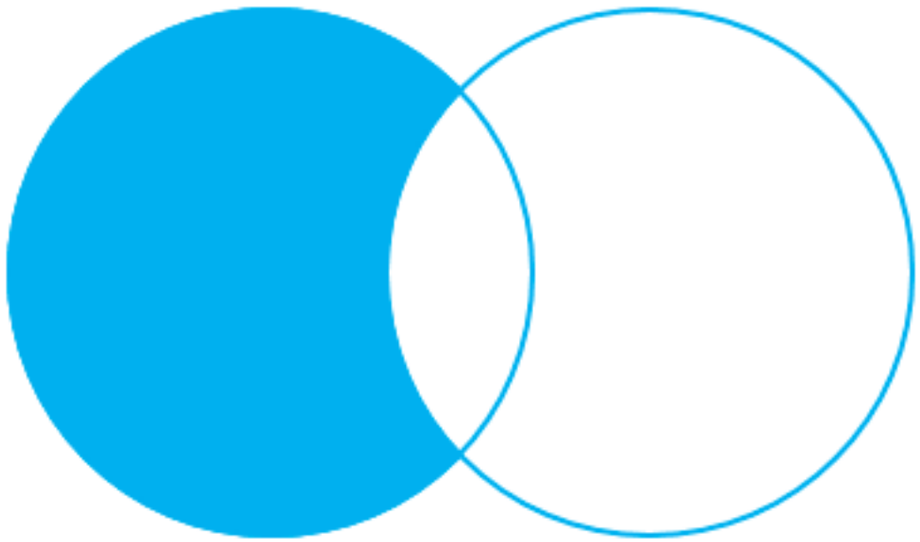
Basket A

Apple
Orange
Banana
Cucumber

Basket B

Apple
Watermelon
Pear

Left join (2)

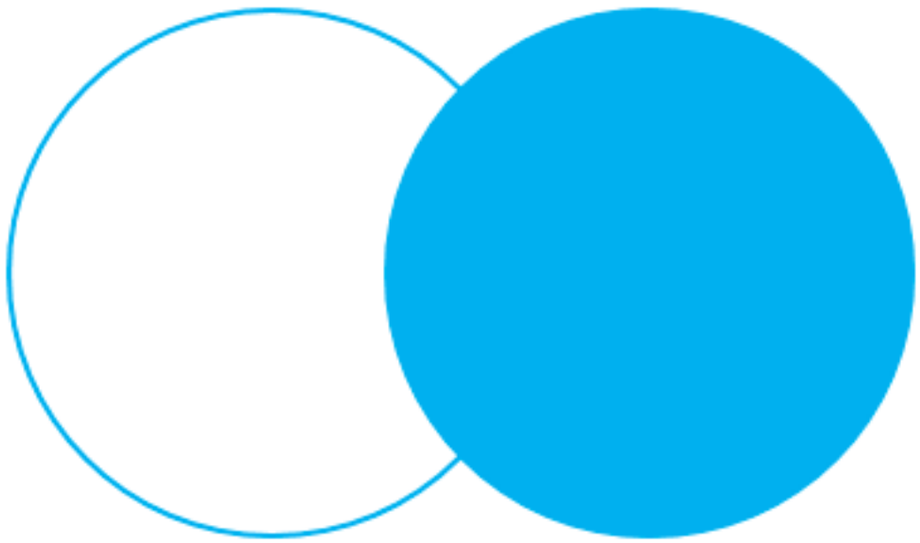


LEFT OUTER JOIN – only rows from the left table

Left join (2)

Basket A	Basket B
Orange	Apple
Banana	Watermelon
Cucumber	Pear

Right join (1)



RIGHT OUTER JOIN

Right join (1)

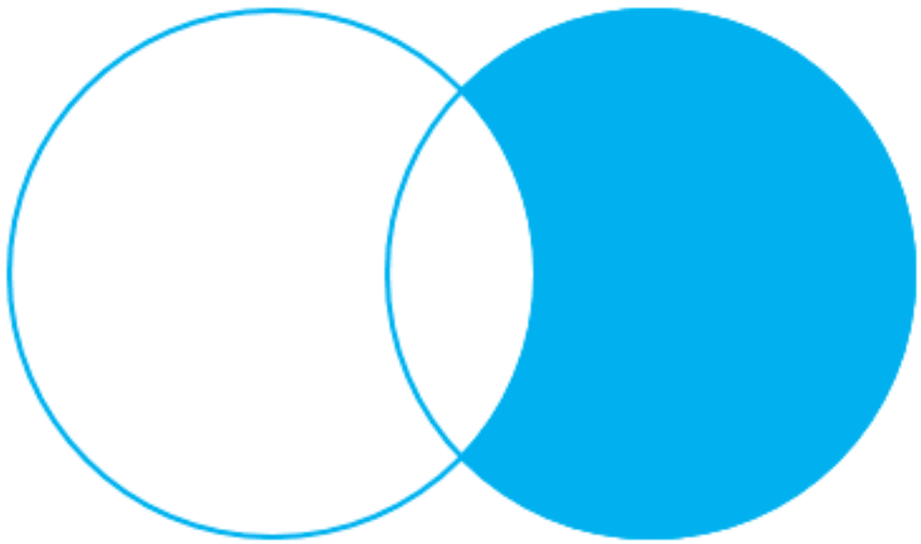
Basket A

Apple
Orange
Banana
Cucumber

Basket B

Orange
Apple
Watermelon
Pear

Right join (2)

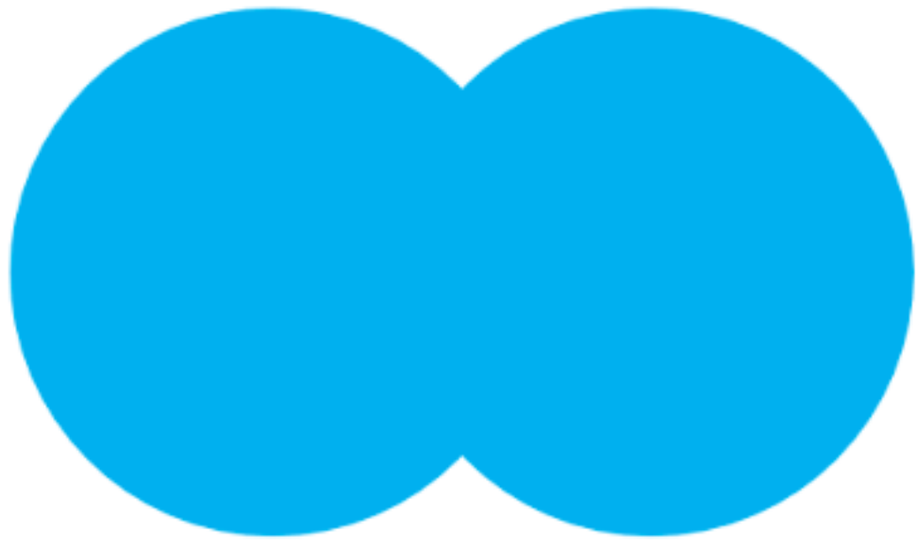


RIGHT OUTER JOIN – only rows from the right table

Right join (2)

Basket A	Basket B
Apple	Orange
Orange	Apple
Banana	**Watermelon**
Cucumber	**Pear**

Full outer join (1)



FULL OUTER JOIN

Full outer join (1)

Basket A

Apple
Orange
Banana
Cucumber

Basket B

Orange
Apple
Watermelon
Pear

2

Full outer join (1)

Basket A

Apple
Orange
Banana
Cucumber

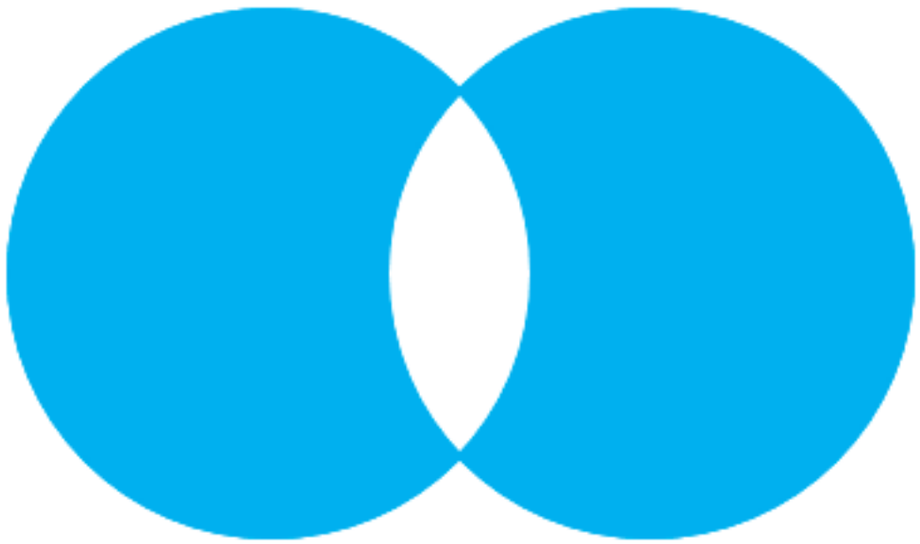
2

Basket B

Apple
Watermelon
Pear

Full outer join (1)

Full outer join (2)



FULL OUTER JOIN – only
rows unique to both tables

Full outer join (2)

Basket A

Apple
Orange
Banana
Cucumber

Basket B

Apple
Watermelon
Pear

2

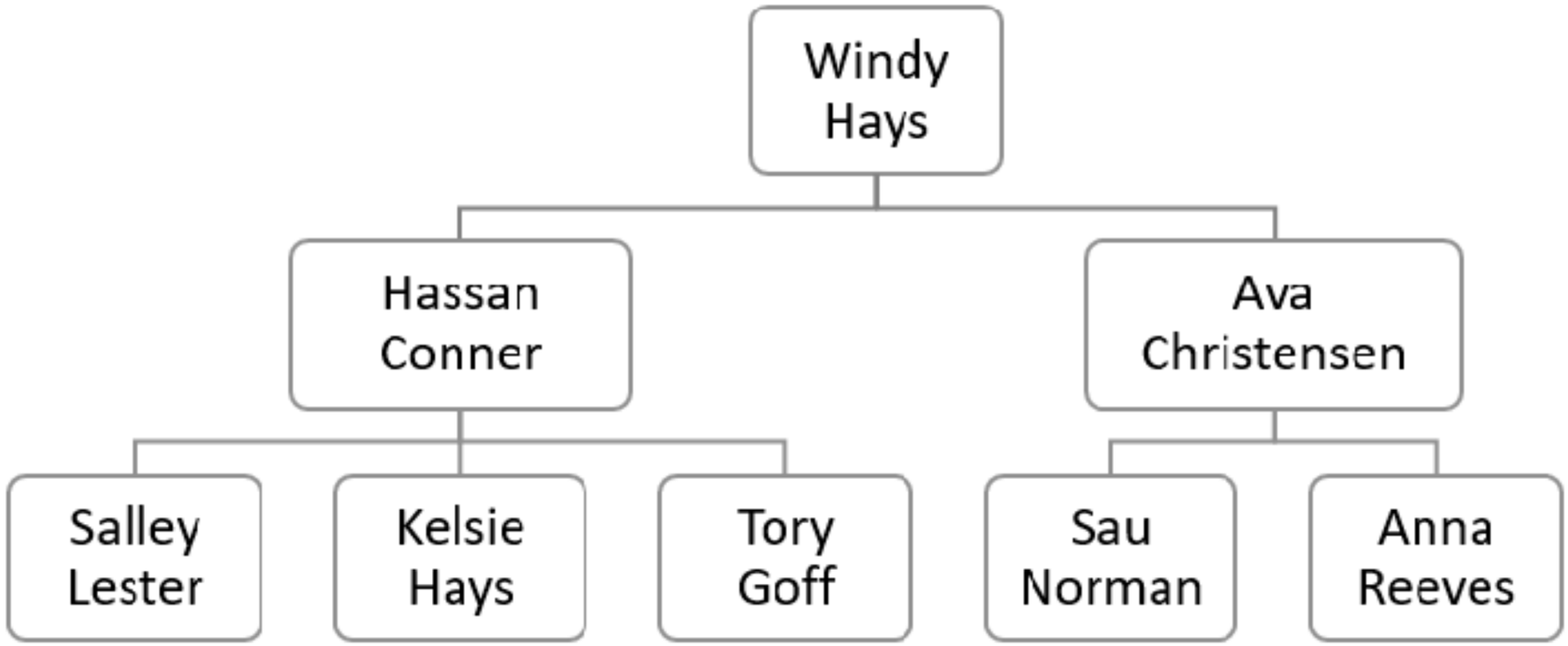
Full outer join (2)

Workshop with Joins

https://github.com/up1/workshop-database/blob/main/workshop/joins.md

Self-join !!!

Regular join that joins a table to itself
Query hierarchical data



Workshop with Self-Join

https://github.com/up1/workshop-database/blob/main/workshop/self-join.md

Problems ?

Problems

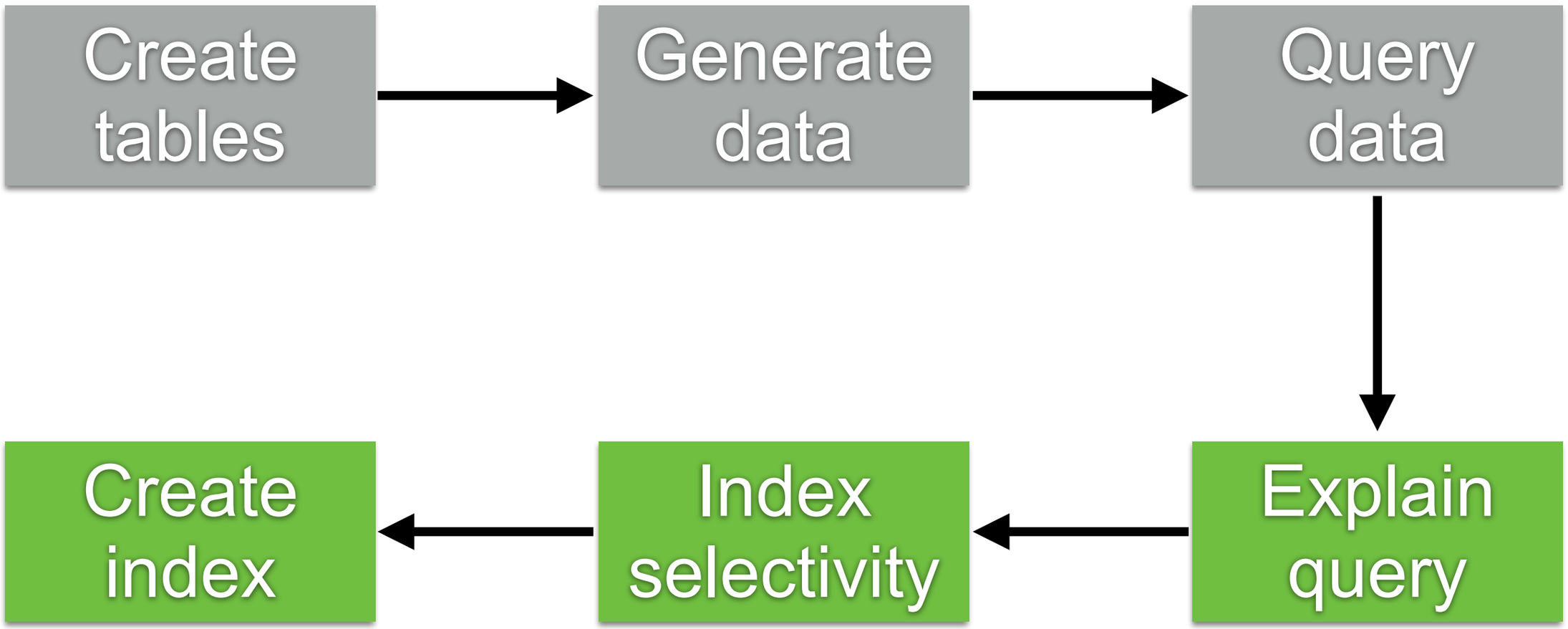
Slow query
Accurate knowledge is required
Add complexity
Need denormalization ?
Rewrite query ?

Database vs application caching ?

Improvement for query data

When ? Why ? How ? Who ?

Step in workshop



Tips for SQL Optimization

Bad	Good
SELECT *	SELECT column1, column2
SELECT ALL	SELECT with Limit, Offset, Top
Many joins !!	Use appropriate joins
SELECT distinct !! For large datasets	Use group by, union , exists, row_number
%column%	Use full text search feature
count(*) > 0 or IN	exists (select 1 from table WHERE …)
Don’t use subquery	Use derived table or pre-joined tables

Always
Explain Analyze Query !!

Query plan in PostgreSQL

EXPLAIN ANALYZE <SQL>

Query | Query History

```
1  EXPLAIN ANALYZE select * from book;
```

Data Output | Messages | Graph Visualiser | Explain | Notifications

	QUERY PLAN text
1	Seq Scan on book (cost=0.00..2334.00 rows=100000 width=89) (actual time=0.007..8.479 rows=100000 loops...
2	Planning Time: 0.209 ms
3	Execution Time: 12.950 ms

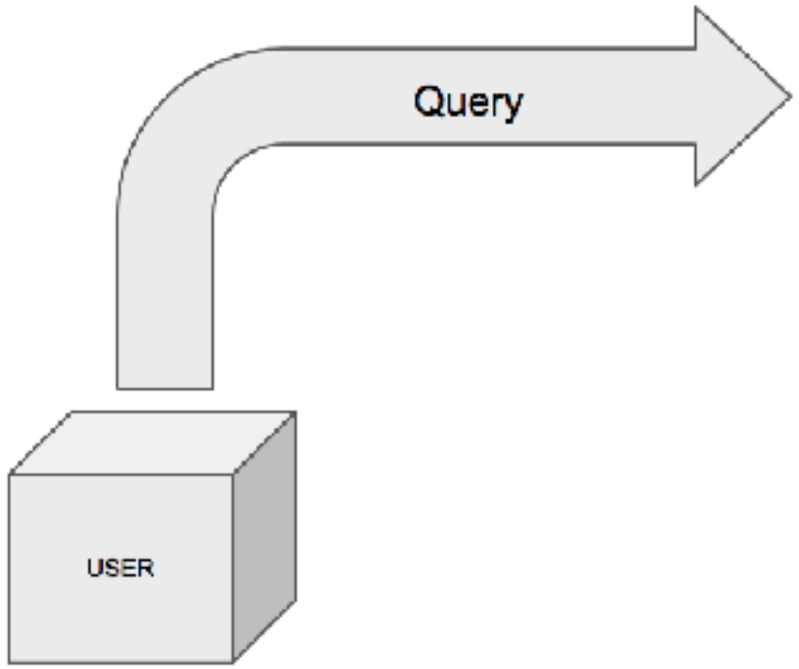
https://github.com/up1/workshop-database/tree/main/workshop#query-plan

Indexing Strategies

https://www.postgresql.org/docs/current/indexes-types.html

Index

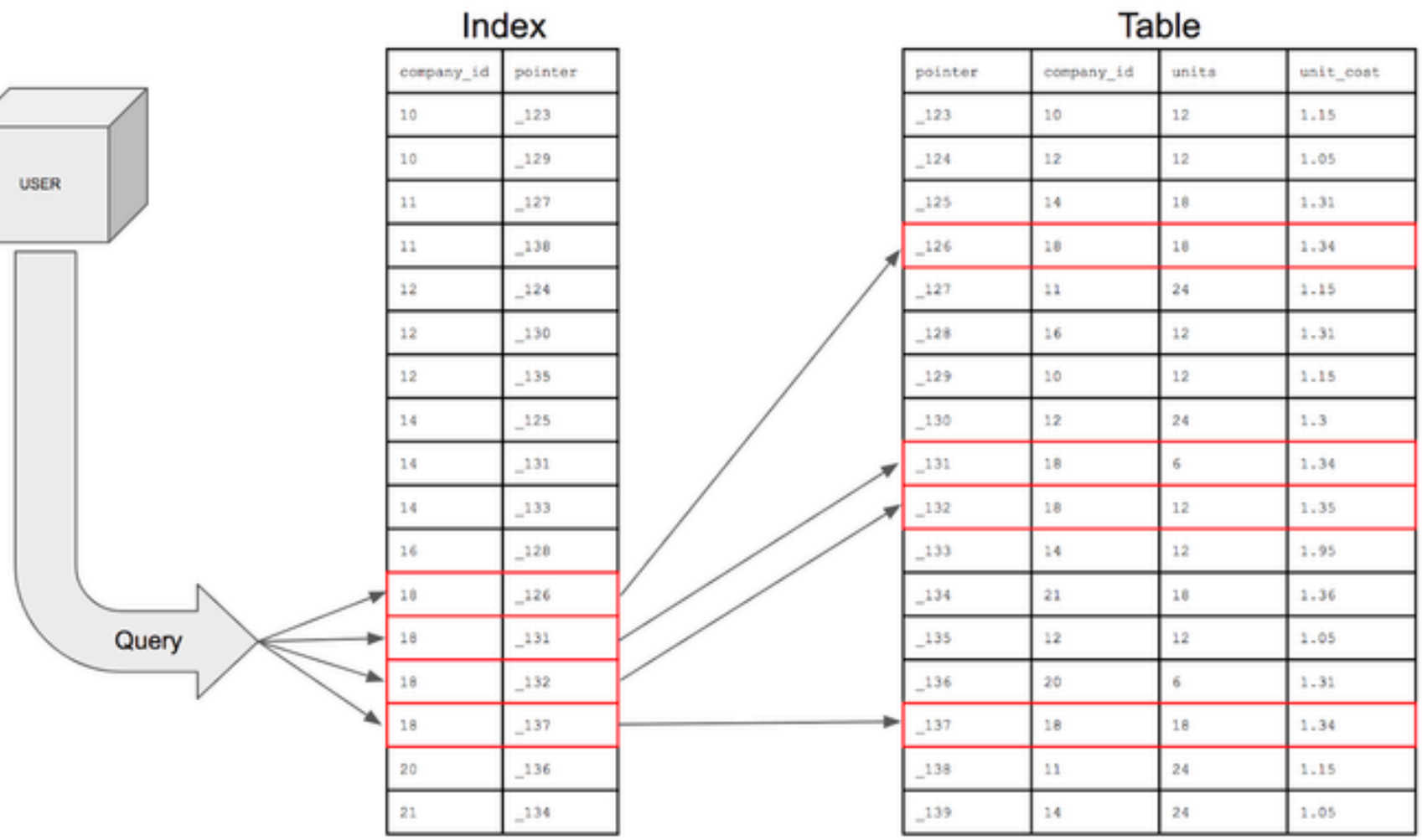
Without Index (full table scan)



Table

company_id	units	unit_cost
10	12	1.15
12	12	1.05
14	18	1.31
18	18	1.34
11	24	1.15
16	12	1.31
10	12	1.15
12	24	1.3
18	6	1.34
18	12	1.35
14	12	1.95
21	18	1.36
12	12	1.05
20	6	1.31
18	18	1.34
11	24	1.15
14	24	1.05

With Index



With Index

Table

	Column A	Column B
RowId 1	Some value 1	Some another value 1
RowId 2	Some value 2	Some another value 2
RowId 3	Some value 3	Some another value 3
RowId 4	Some value 1	Some another value 4
RowId 5	Some value 5	Some another value 5
RowId 6	Some value 6	Some another value 6
RowId 7	Some value 1	Some another value 7
RowId 8	Some value 8	Some another value 8

Index

Some value 1	"RowId 1" "RowId 4" "RowId 7"
Some value 2	RowId 2
Some value 3	RowId 3
Some value 5	RowId 5
Some value 6	RowId 6
Some value 8	RowId 8

Index by column "Column A"

Main factors for good index ?

How much space does the index use ?
How fast does it read ?
How much overhead does it add to writes ?
How well does it handle concurrency ?

Quick lookup, fast insert, reliable delete

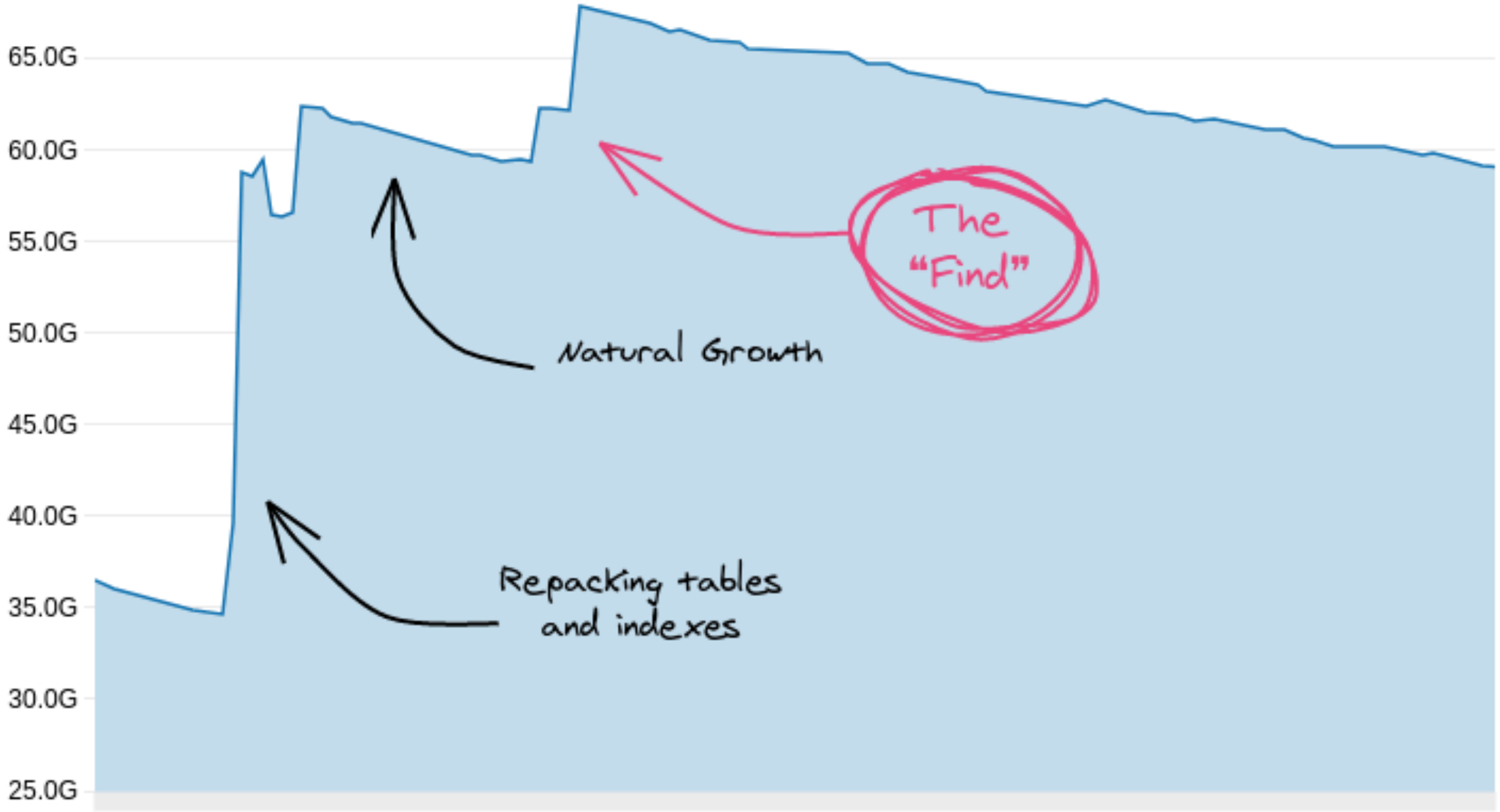
Index selectivity

Factor to select a column to create an index

The number of **distinct values** in the indexed column divided by the number of records in the table is called a selectivity of an index

Prefer indexing columns with selectivity greater than > 0.85 (use Index scan)

Unused Index ?



https://hakibenita.com/postgresql-unused-index-size

Types Index in PostgreSQL

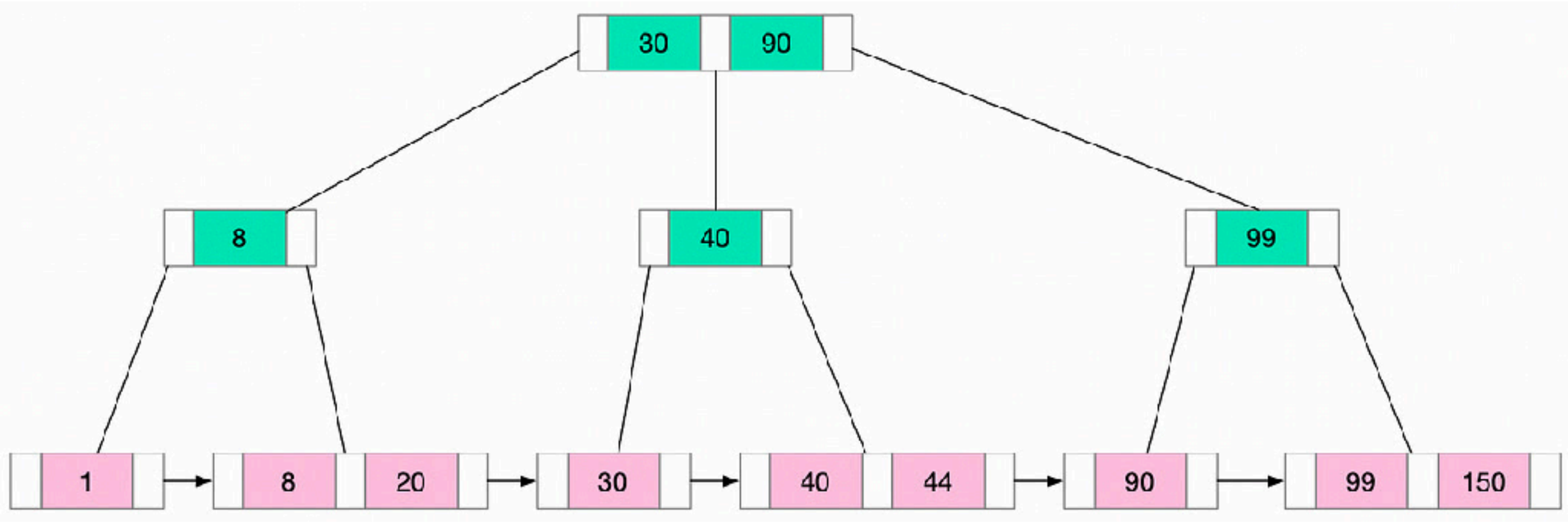
B-Tree
Hash
Block range indexes (BRIN)
Generalized inverted index (GIN)
Generalized inverted search tree index (GiST)
Space partitioned GiST (SP-GiST)

https://www.postgresql.org/docs/current/indexes-types.html

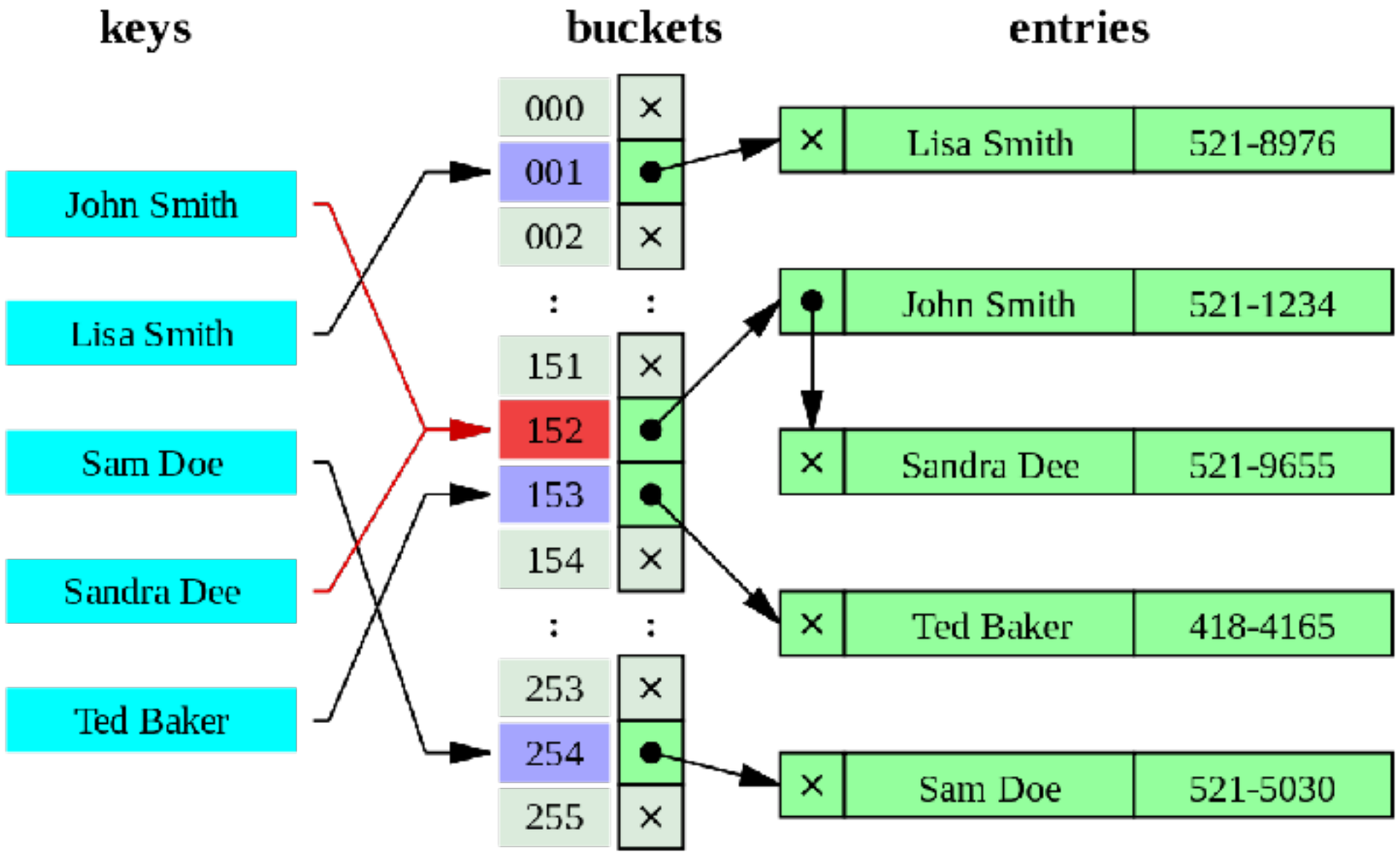
PostgreSQL Index Types

Index type	Data	Target	Use case
B-Tree	High-cardinality scalar columns	Fast equality, range, between operation	PK, FK, timestamp/range scan, sorting
GIN	Composite values (arrays, JSONB, tsvector)	Efficient contains, full-text, array membership	Full text search, JSONB containment, array lookup
GiST	Complex or non-scalar types	Flexible tree, Range and text	PostGIS spatial queries, trigram similarity, range types
BRIN	Very large data, ordered table	Lightweight index for append-only and range scan	Time-series data Log data
Bloom	Many low-cardinality columns	Compact multi-column equality filters	Filter data, check data existing

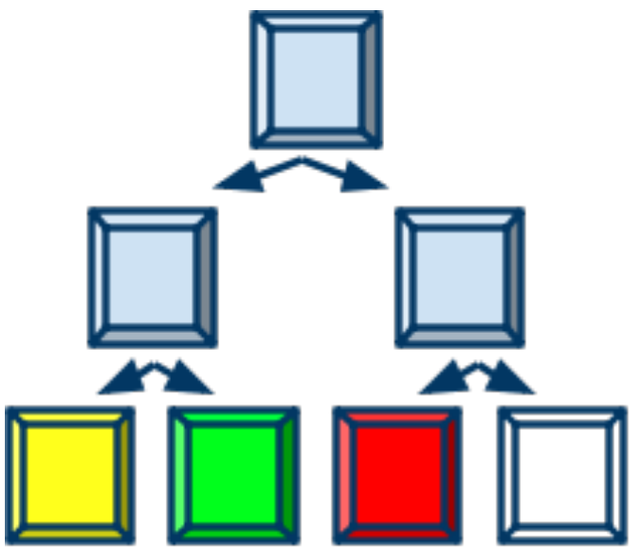
B-Tree +

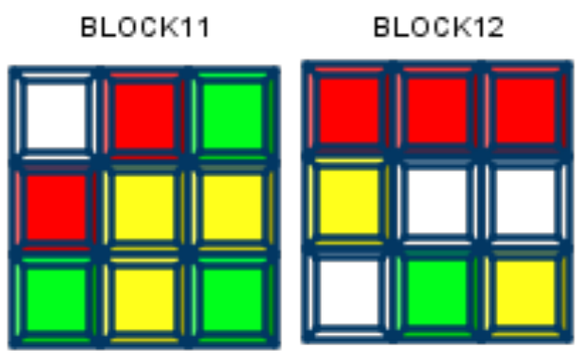


Hash



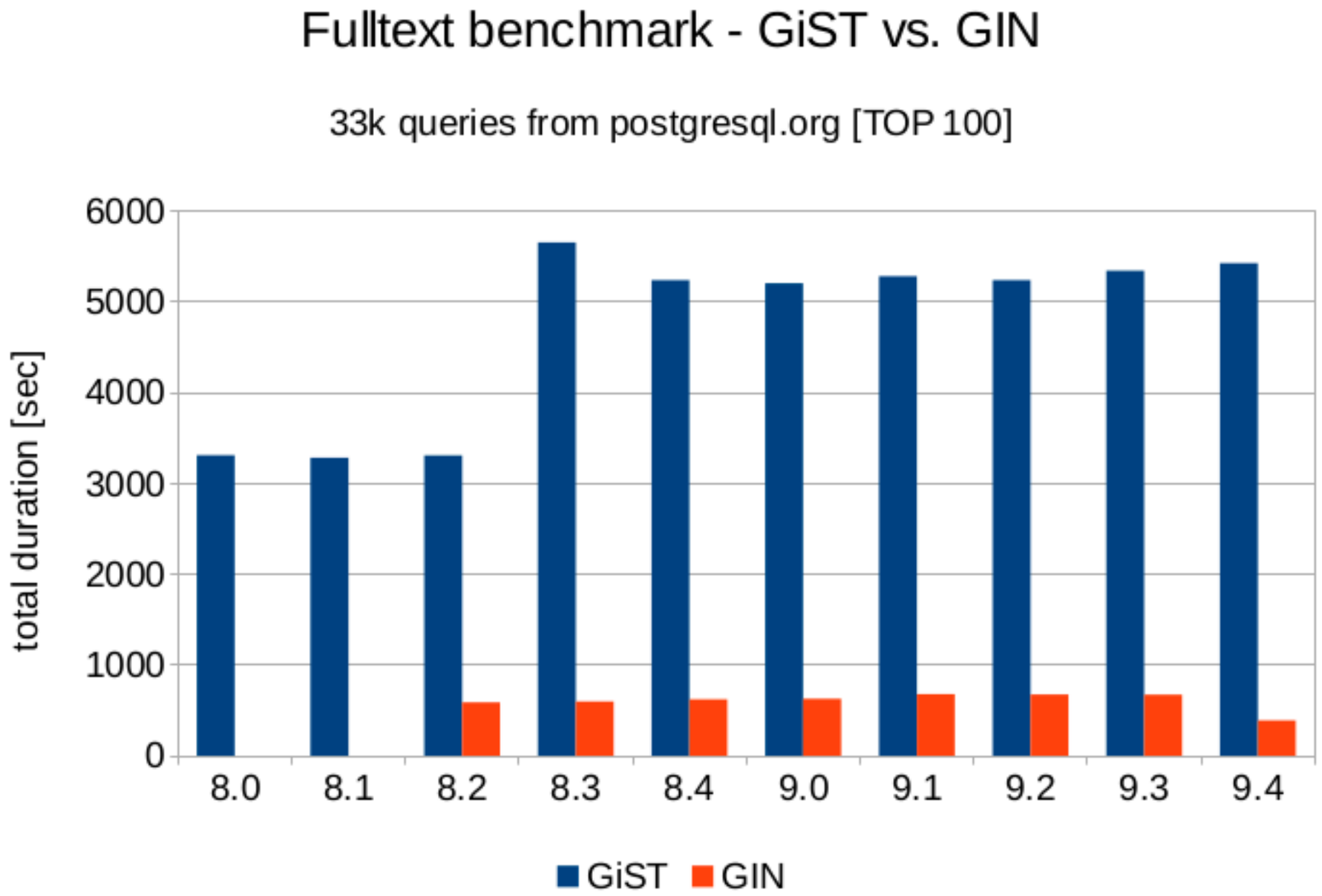
Bitmap





KEY	Start Row ID	End Row ID	BATMAP
YELLOW	11.0	12.9	10000
GREEN	11.0	12.9	01000
RED	11.0	12.9	00100
null	11.0	12.9	00010

GIN vs GiST



Tuning and Use Index ?

Size vs Speed

Extension setup

Combine indexes

Measure impact with **EXPLAIN** (before and after)

Don’t over-index
Unused index

Size of index ?
Usages ?

Workshop with Indexing

https://github.com/up1/workshop-database/tree/main/workshop/basic-workshop

Tuning Performance

Scalability metrics

Throughput (transactions per second)
Latency (response time)
Availability (uptime)
Durability (data safety)

Tuning performance

Normalization vs Denormalization

Partition

Sharding

Database architecture

Database parameteres

Partition

https://hakibenita.com/postgresql-unused-index-size

Partition

Table partitioning refers to **splitting** what is logically one large table
into **smaller physical pieces**

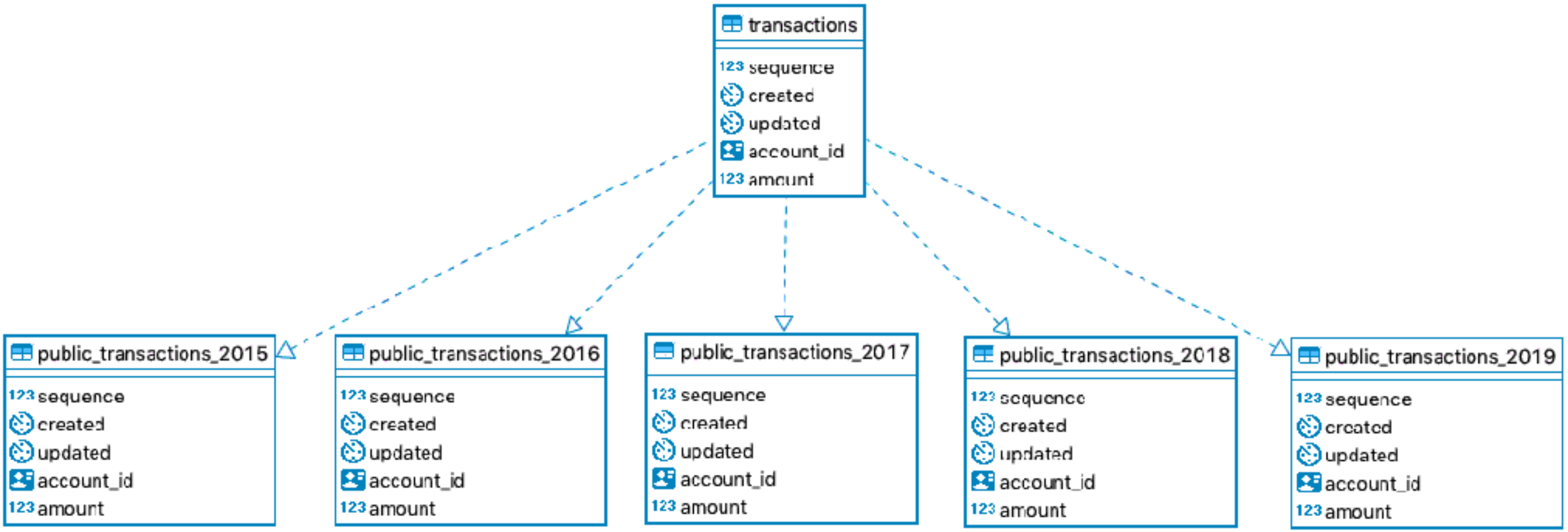
Reduce index size

Partition

Horizontal partitioning
Vertical partitioning

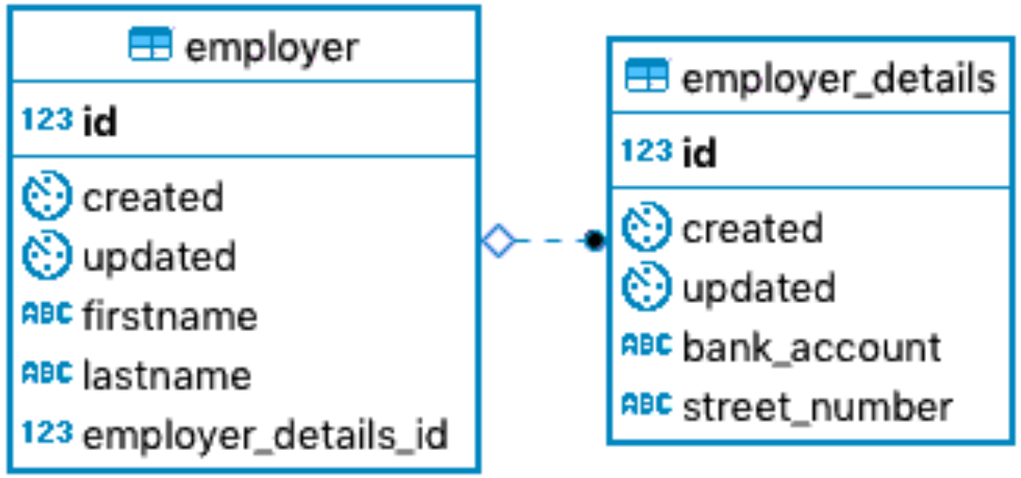
Horizontal partitioning

Putting different rows into different table



Vertical partitioning

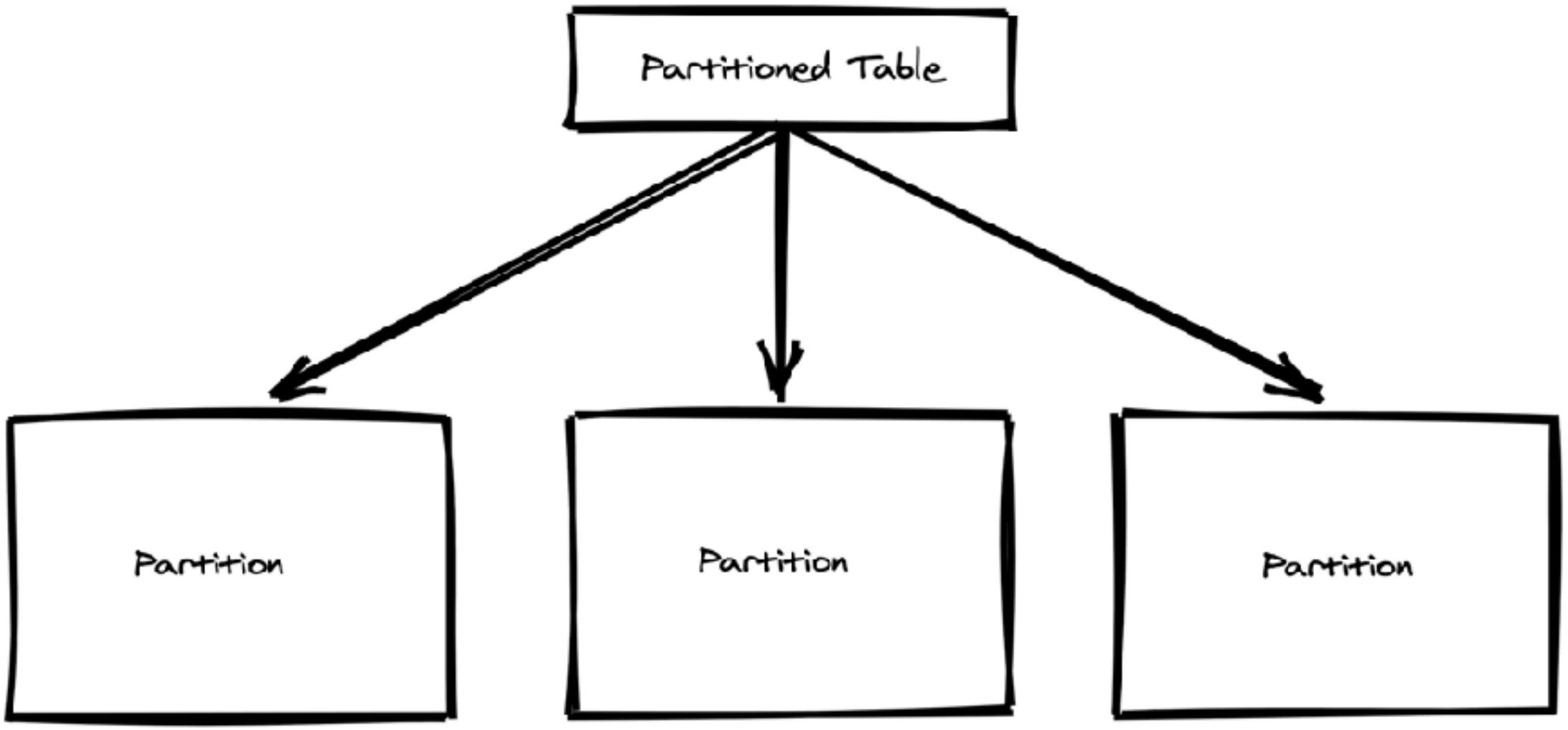
Horizontal partitioning



Types Horizontal partitioning

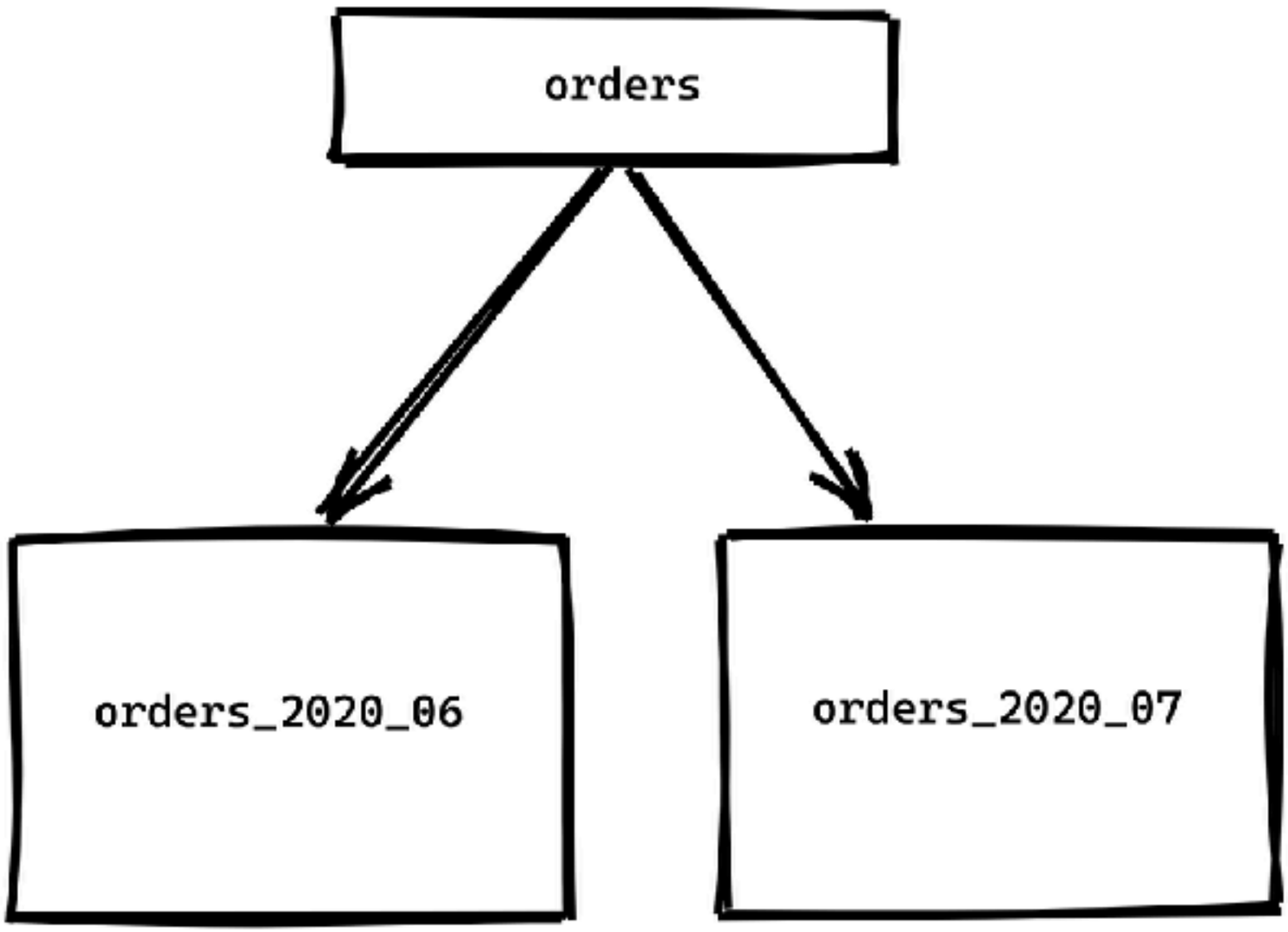
Range
List

Partition tables



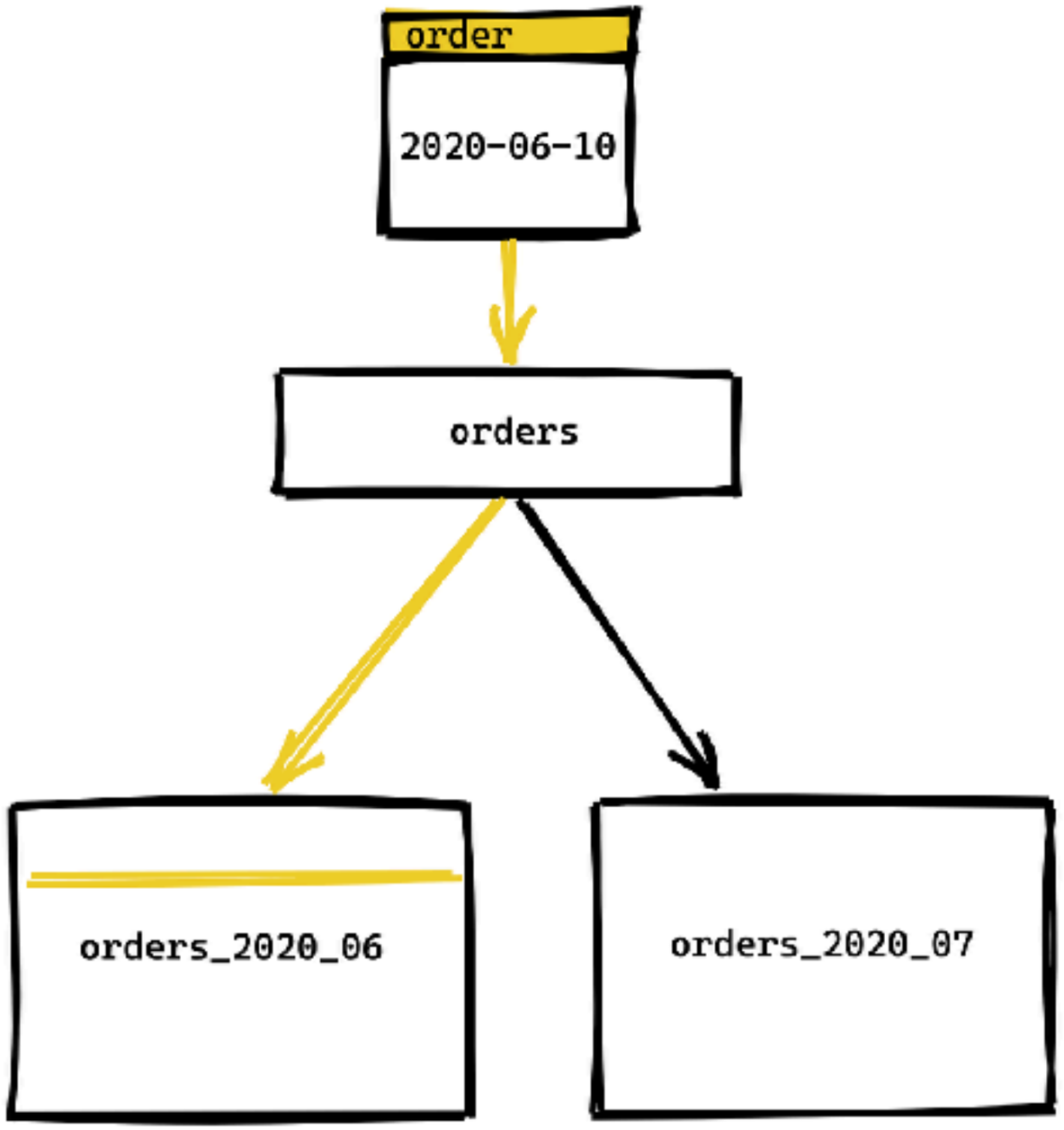
https://chriserwin.com/table-partitioning/

Orders table

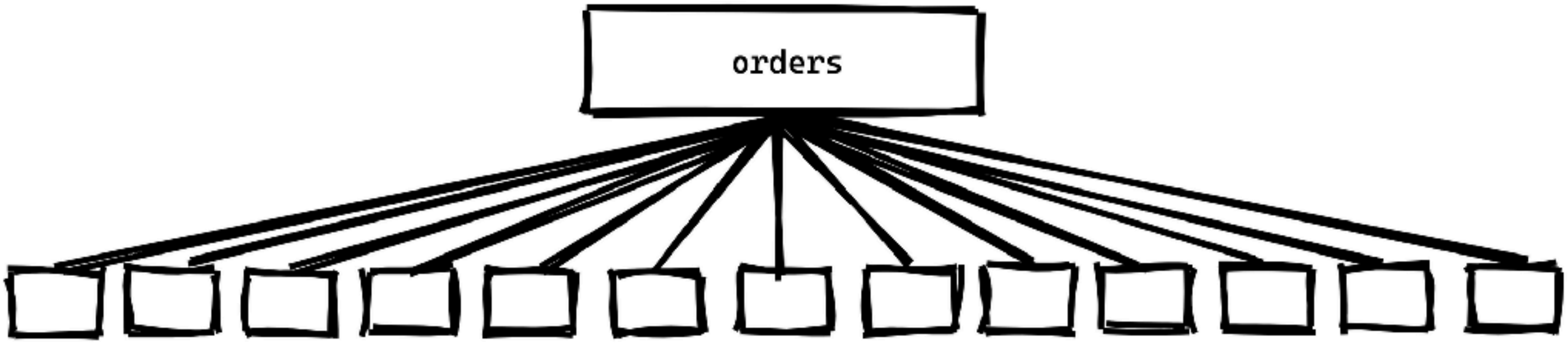


https://chriserwin.com/table-partitioning/

Insert data into orders



https://chriserwin.com/table-partitioning/



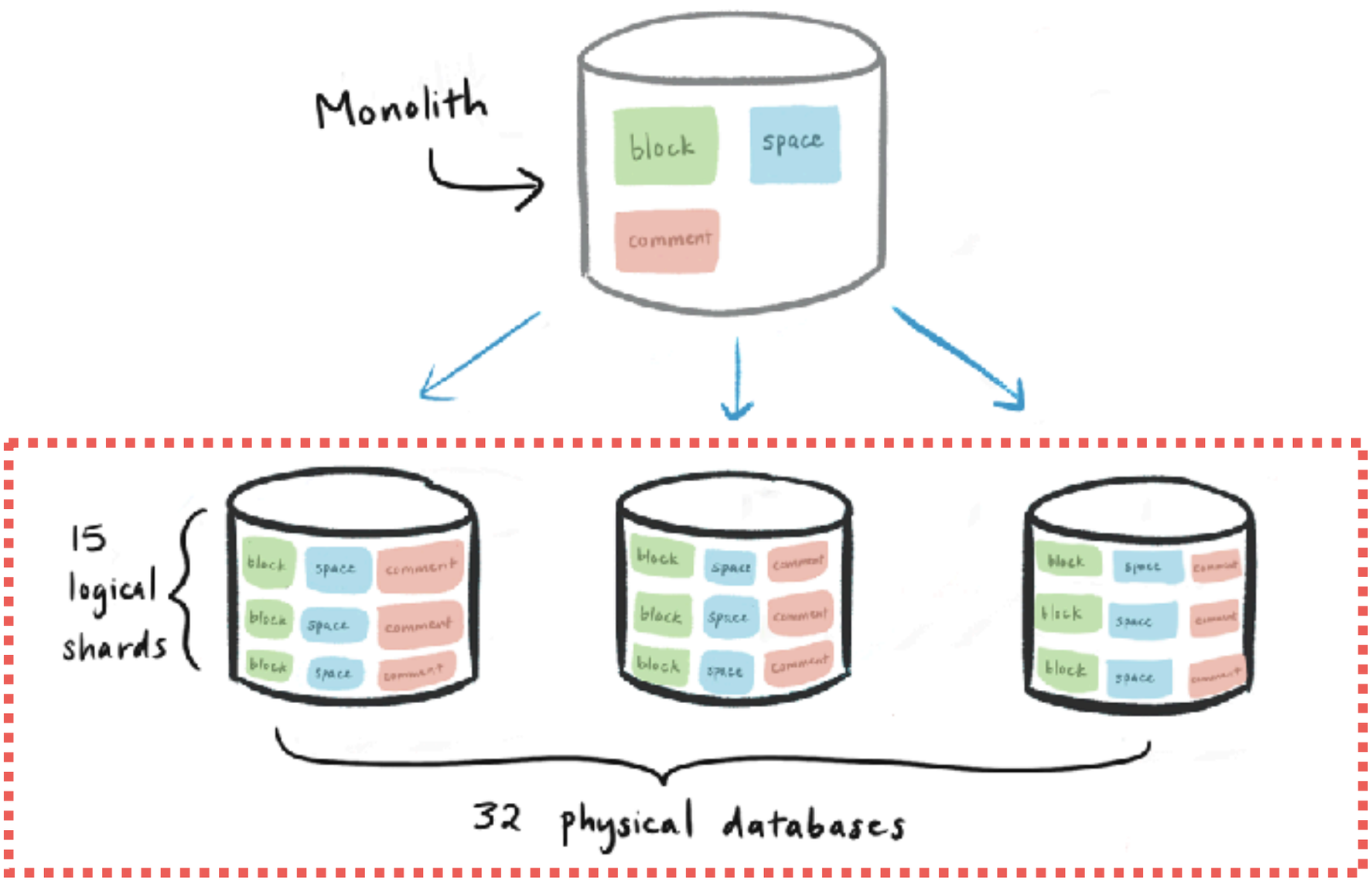
https://chriserwin.com/table-partitioning/

Workshop with Partition

https://github.com/up1/workshop-database/blob/main/workshop/basic-workshop/partition.md

Sharding

Sharding (Physical server)



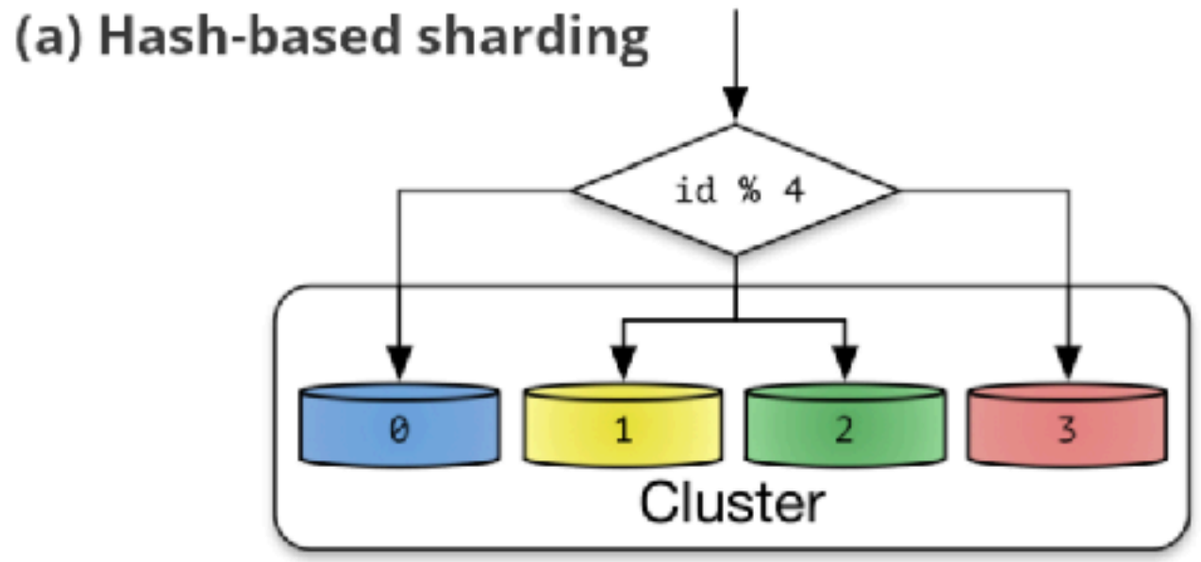
https://www.notion.so/blog/sharding-postgres-at-notion

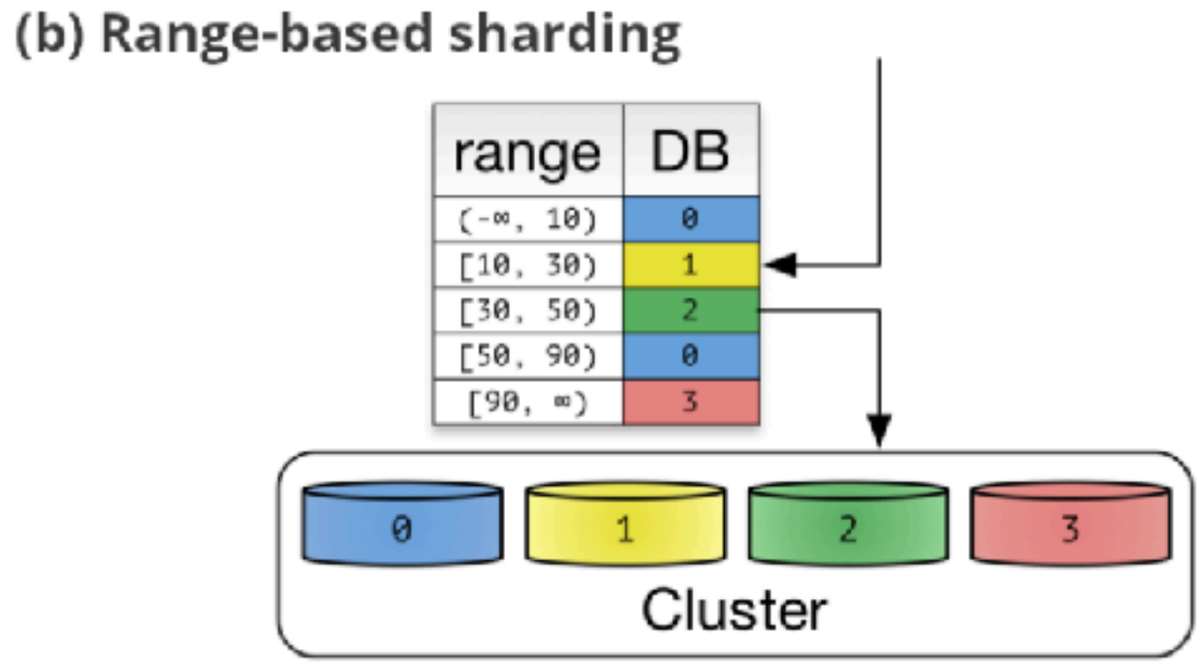
Sharding techniques

Hash
Range
Location
Key/Directory

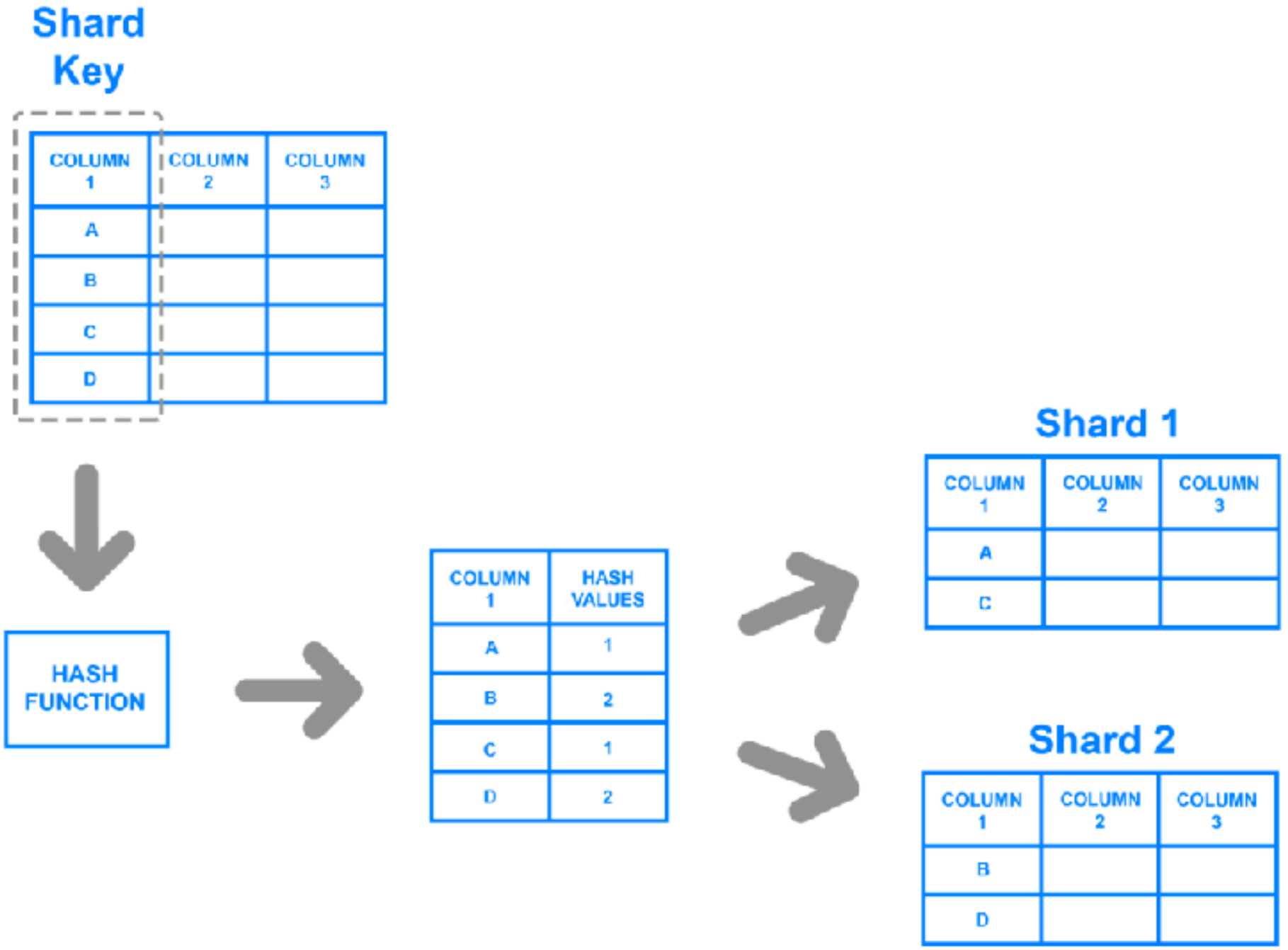
https://www.somkiat.cc/introduction-database-sharding/

Sharding





Shard Key



Range

PRODUCT	PRICE
WIDGET	$118
GIZMO	$88
TRINKET	$37
THINGAMAJIG	$18
DOODAD	$60
TCHOTCHKE	$999

($0-$49.99)

PRODUCT	PRICE
TRINKET	$37
THINGAMAJIG	$18

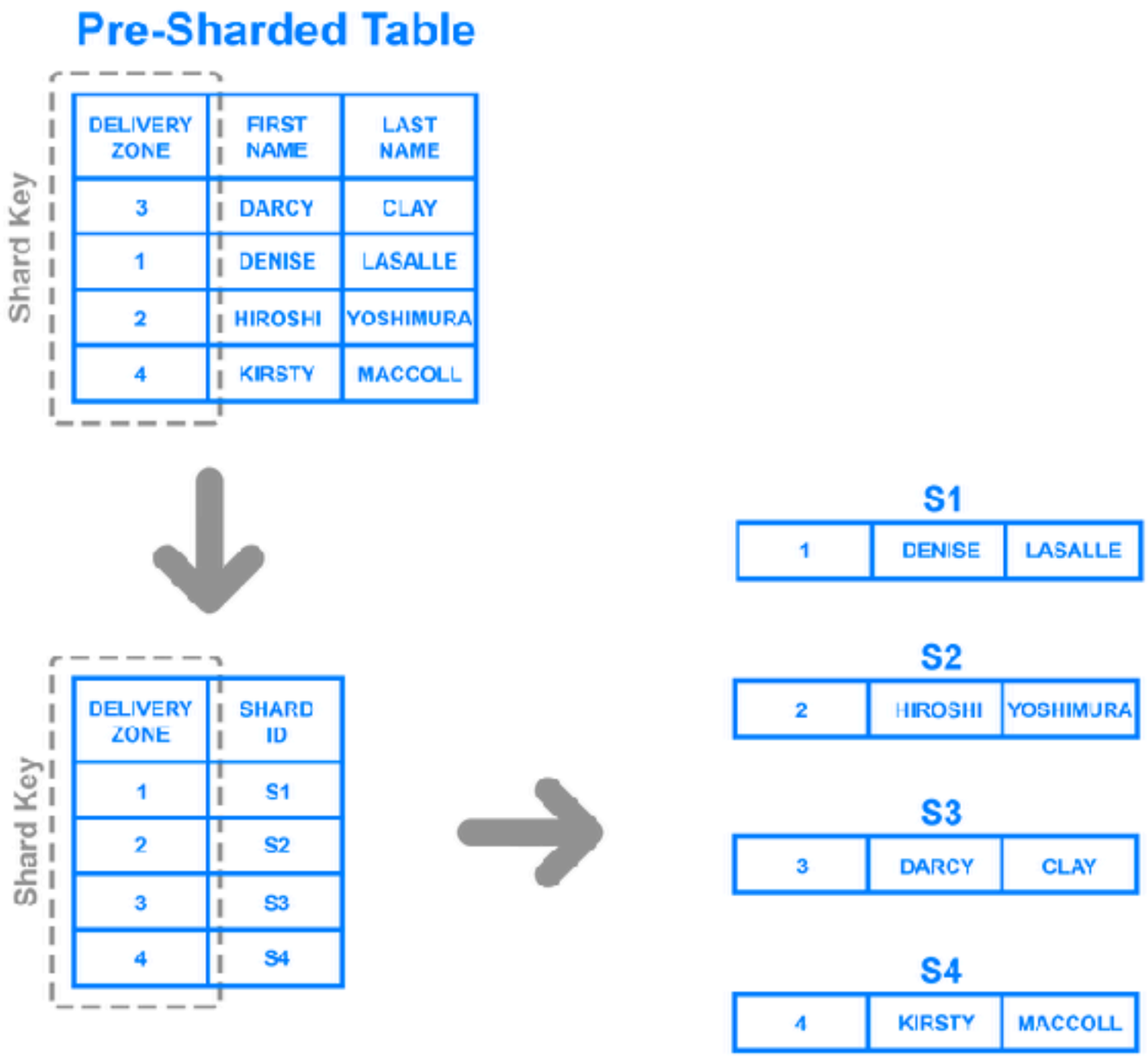
($50-$99.99)

PRODUCT	PRICE
GIZMO	$88
DOODAD	$60

($100+)

PRODUCT	PRICE
WIDGET	$118
TCHOTCHKE	$999

Directory



Sharding

Shard 1

Stores

id	name
1	my book store
5	my other store

Products

id	name	store_id
1	foo	1
2	bar	1
3	baz	1

Purchases

id	product_id	store_id	price
1	2	1	1000
2	1	1	1200
3	3	1	1199

Shard 2

Stores

id	name
2	my sock store
6	old things

Products

id	name	store_id
33	new socks	2
34	old socks	6
35	old tie	6

Purchases

id	product_id	store_id	price
102	35	6	600
43	33	2	800

https://www.citusdata.com/blog/2016/08/10/sharding-for-a-multi-tenant-app-with-postgres/

Challenge of Large datasets

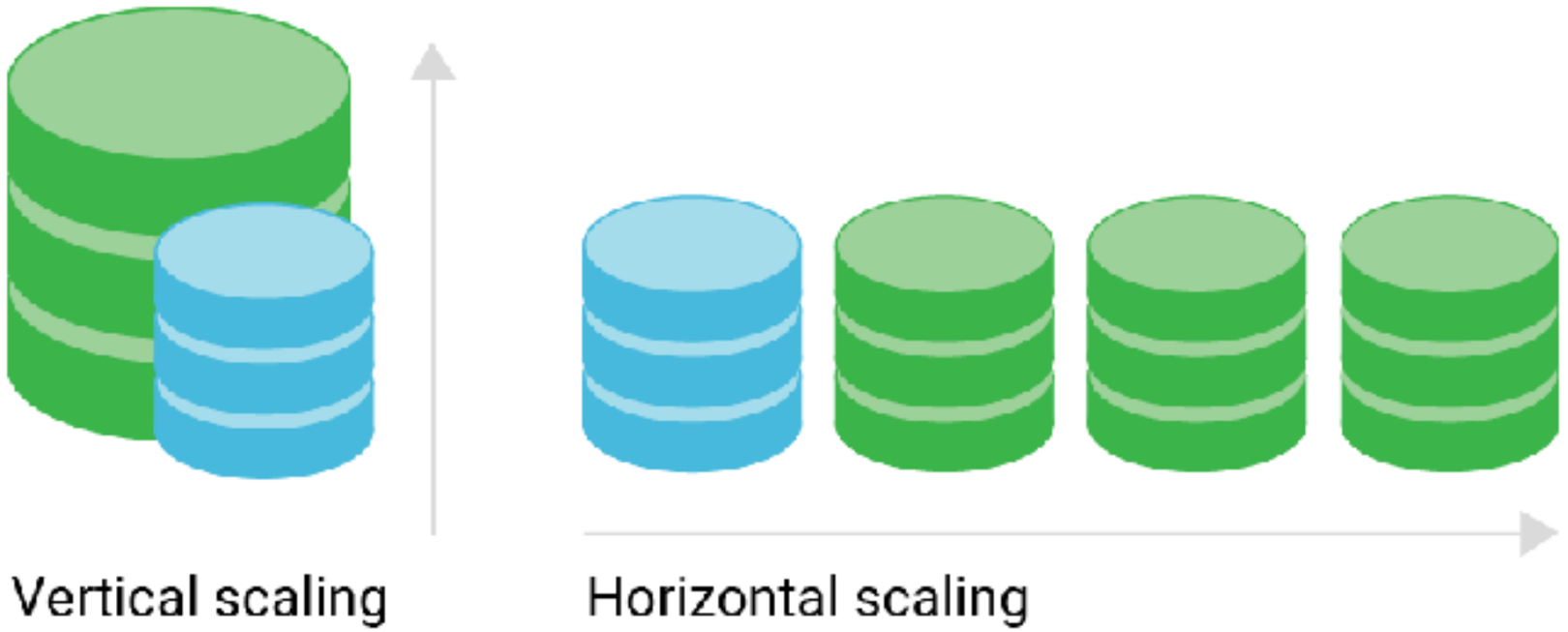
Challenge of Large datasets

Disk I/O, CPU, Memory pressure
Index size and maintenance
Locking and concurrency

Why database slow down
with more data ?

Scaling Database ?

Vertical vs Horizontal

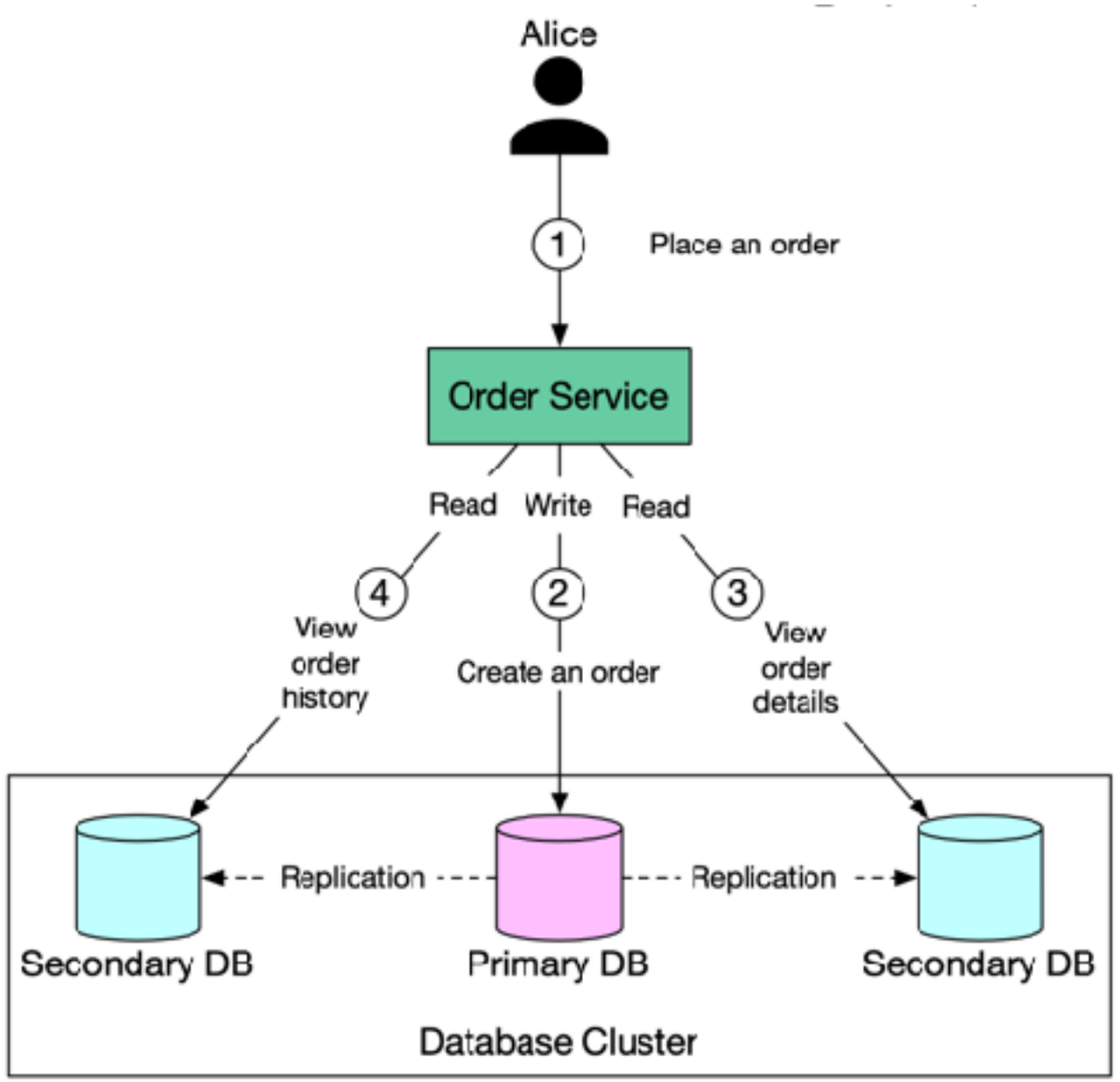


Scalability metrics

Throughput (transactions per second)
Latency (response time)
Availability (uptime)
Durability (data safety)

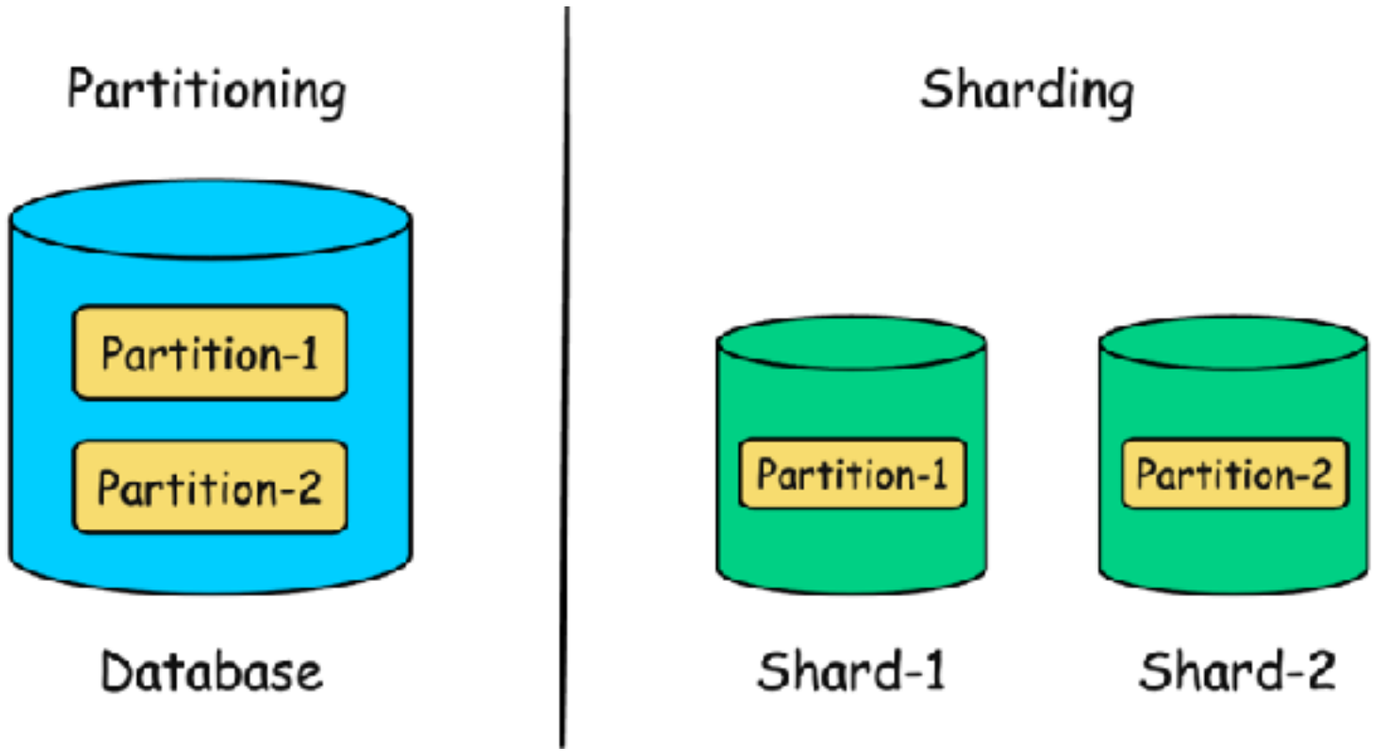
Read replicas

Read-scaling solution



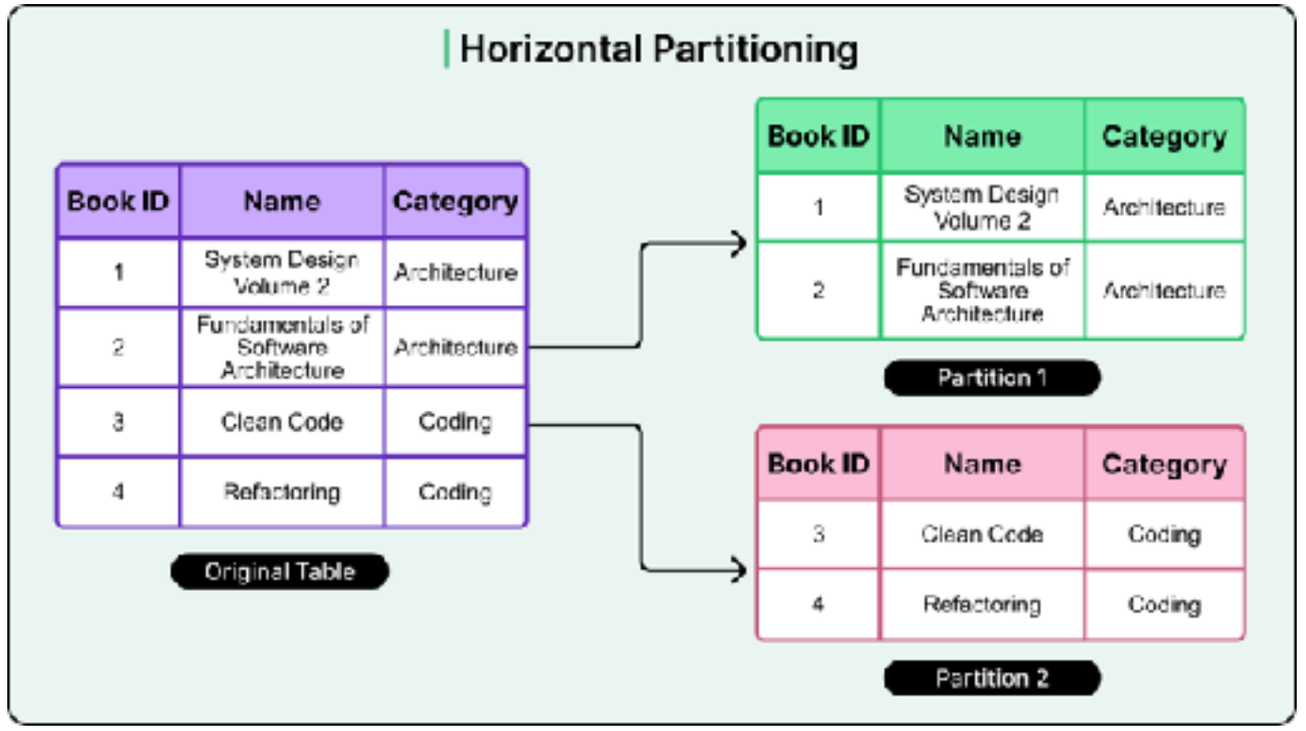
Data distribution strategies

Partitioning (within a single instance)
Sharding (across multiple instances)



Database sharding

Design for large dataset by split into smaller
Divided across multiple instance
Based-on horizontal partitioning
Use when to overcome single instance limitations



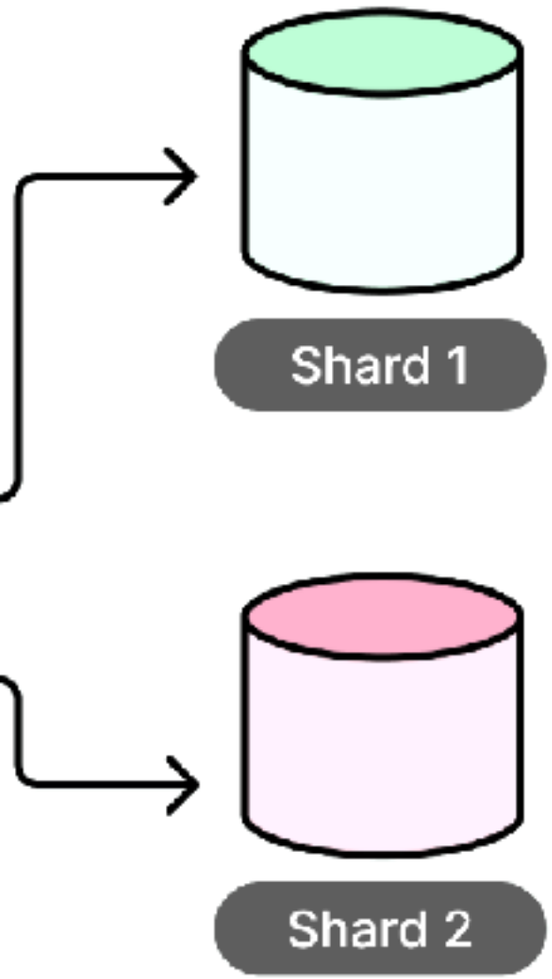
Sharding techniques

Range
Hash or key-based
Directory-based
Request routing

https://blog.bytebytego.com/p/a-guide-to-database-sharding-key

Range-based sharding

Product	Price
A	$15
B	$49
C	$78
D	$131



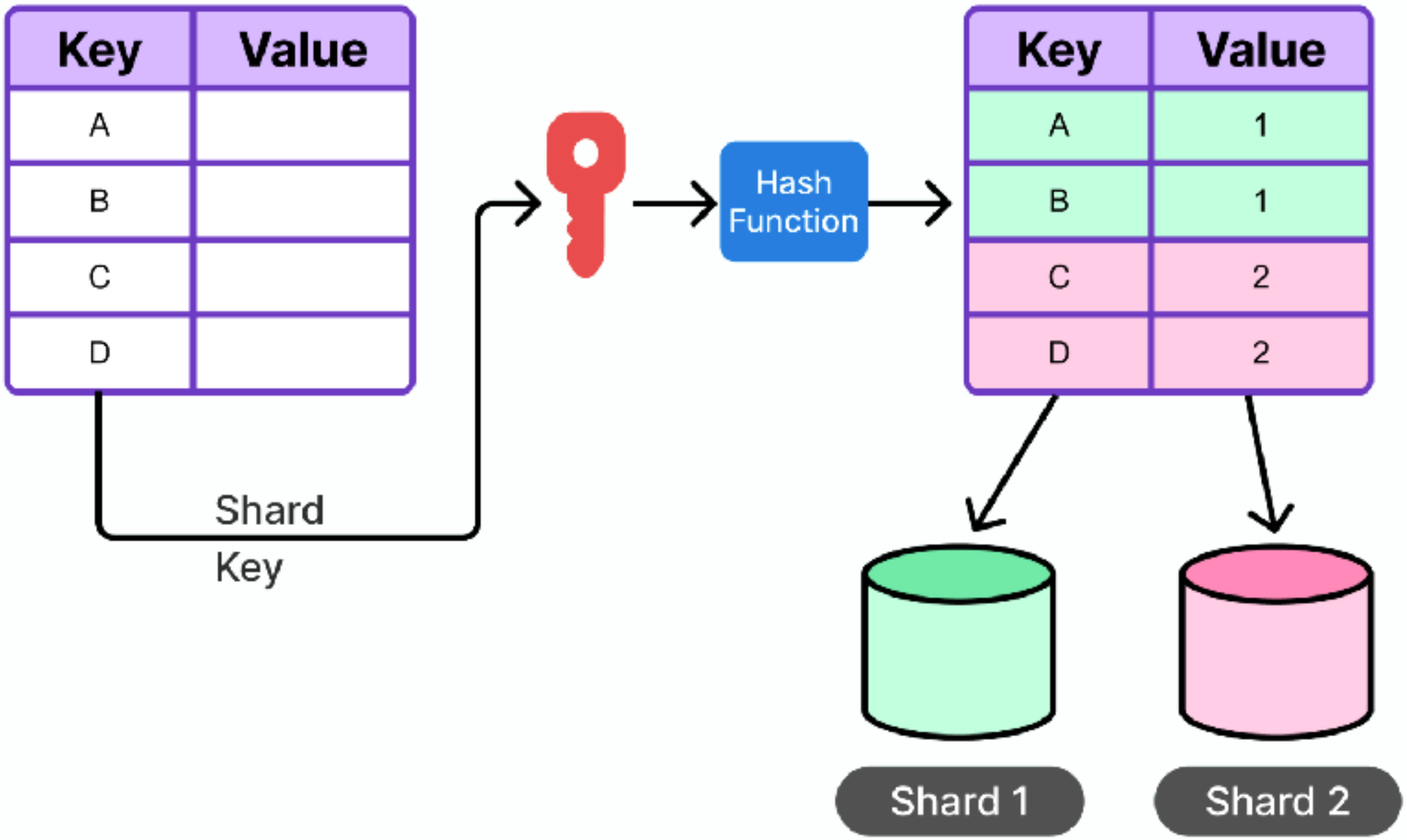
Product	Name
A	$15
B	$49

$0 - $75

Product	Name
C	$78
D	$131

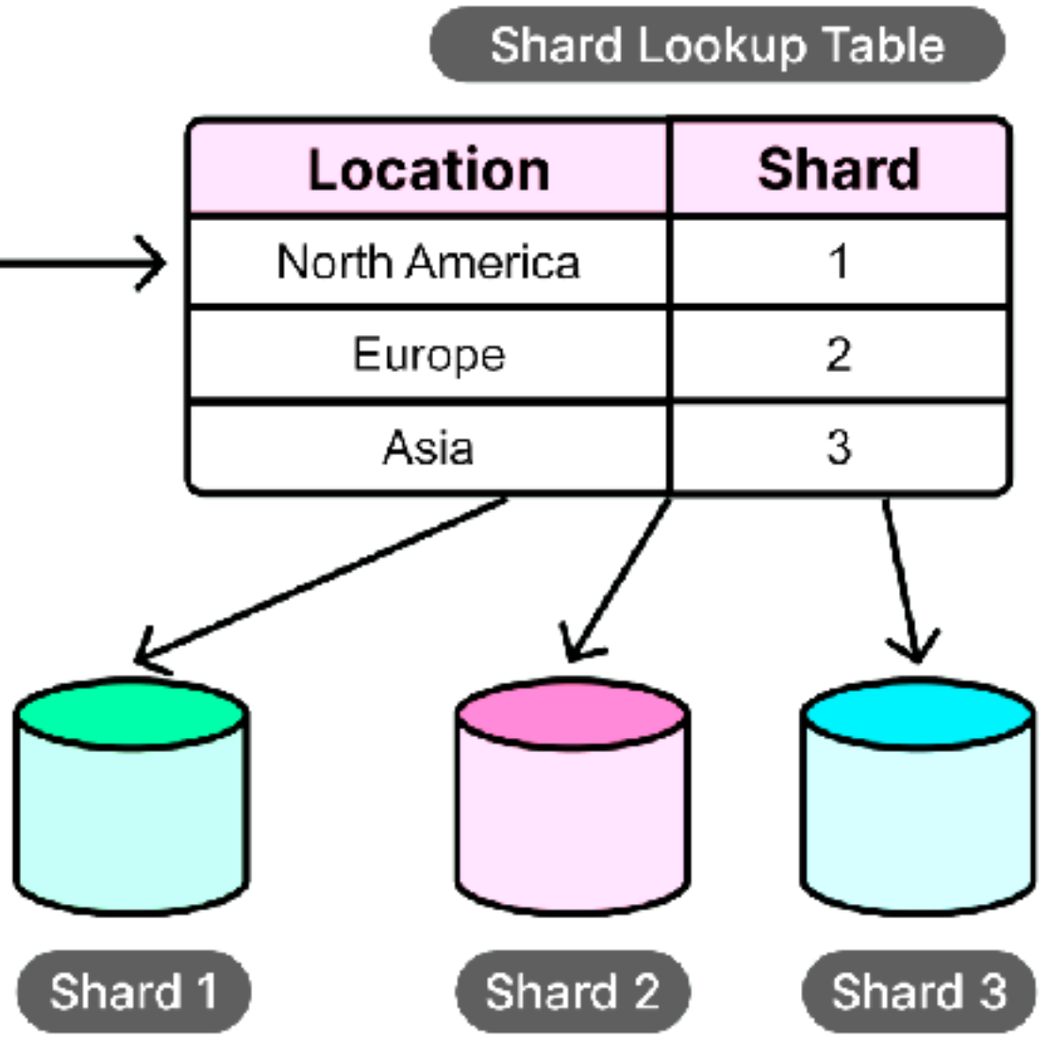
$76-150

Key-based sharding



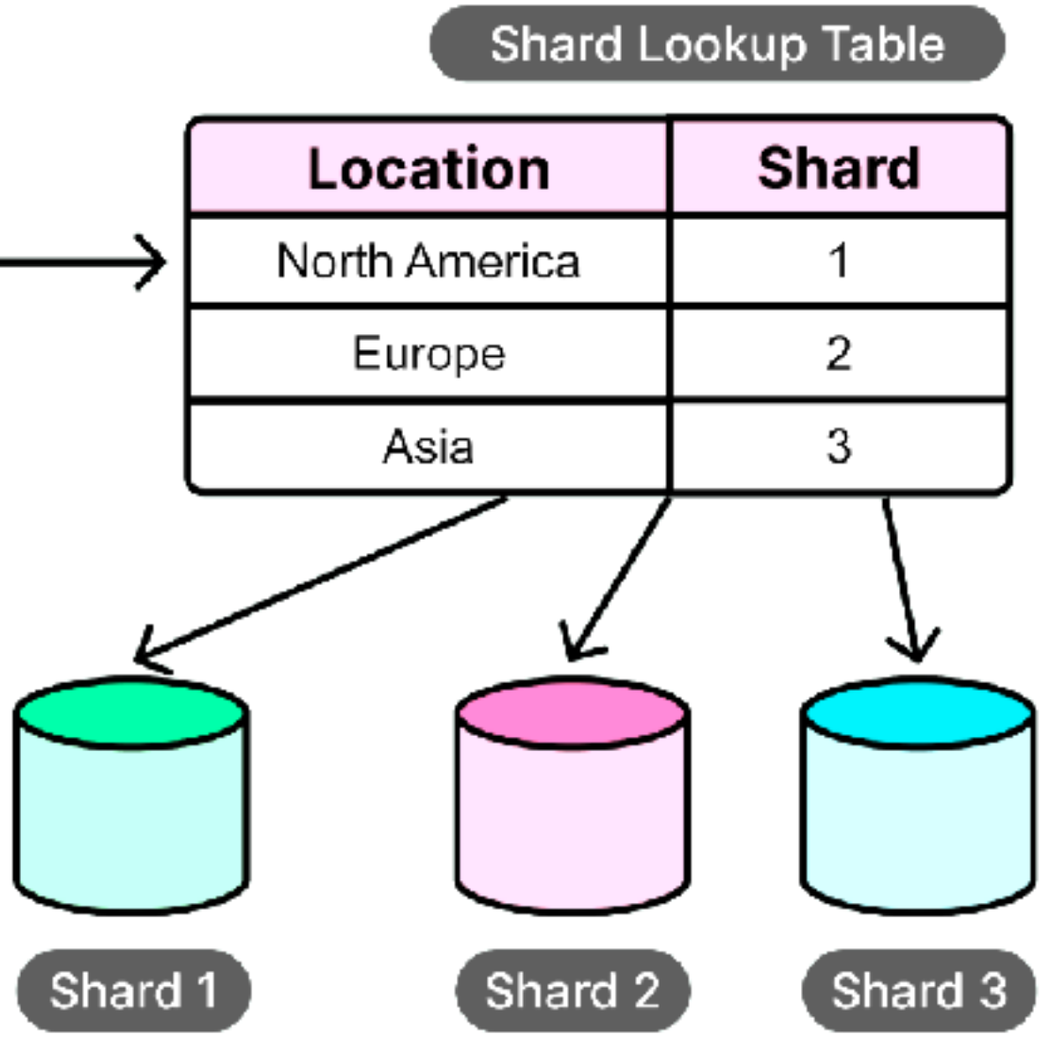
Directory-based sharding

Customer	Location
Adam	North America
Smith	Europe
Joe	Asia
James	Asia

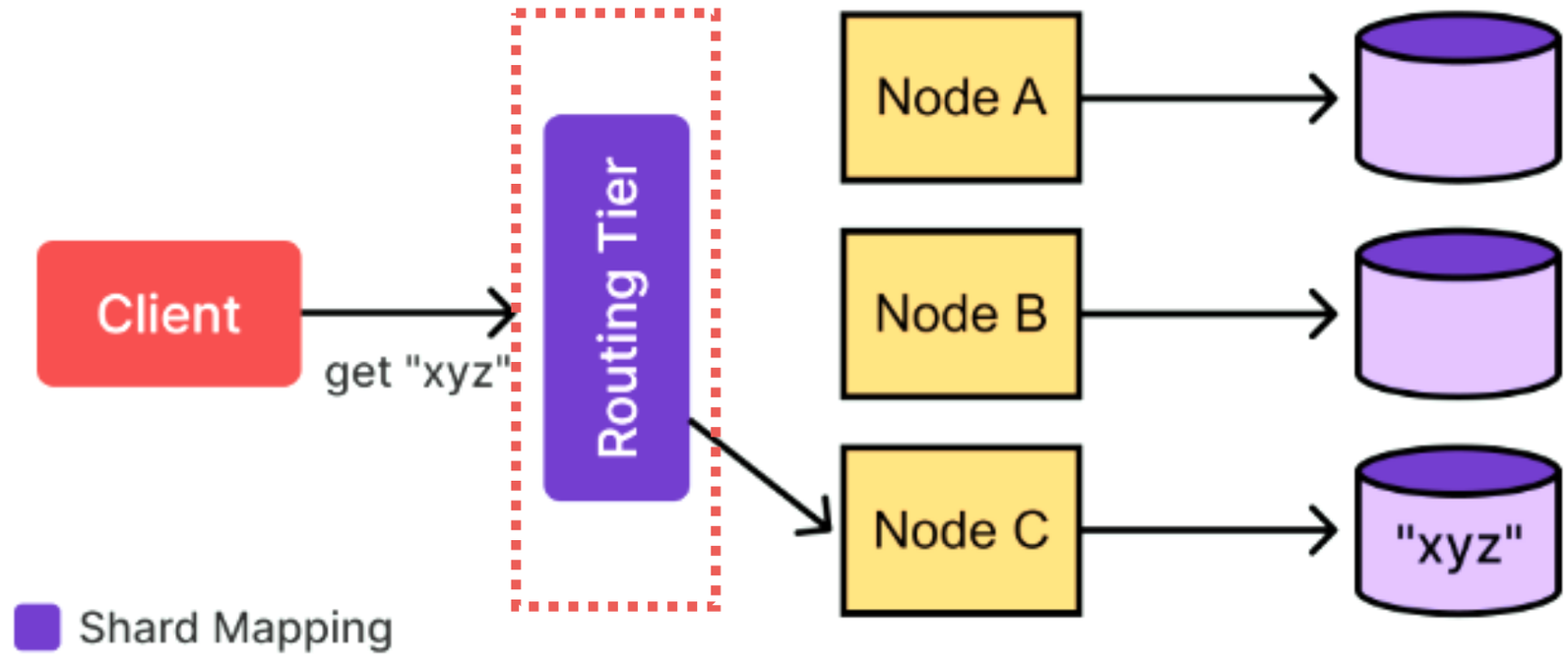


Directory-based sharding

Customer	Location
Adam	North America
Smith	Europe
Joe	Asia
James	Asia



Request routing



Workshop with Sharding

https://github.com/up1/workshop-database/blob/main/workshop/basic-workshop/partition.md

Denormalization

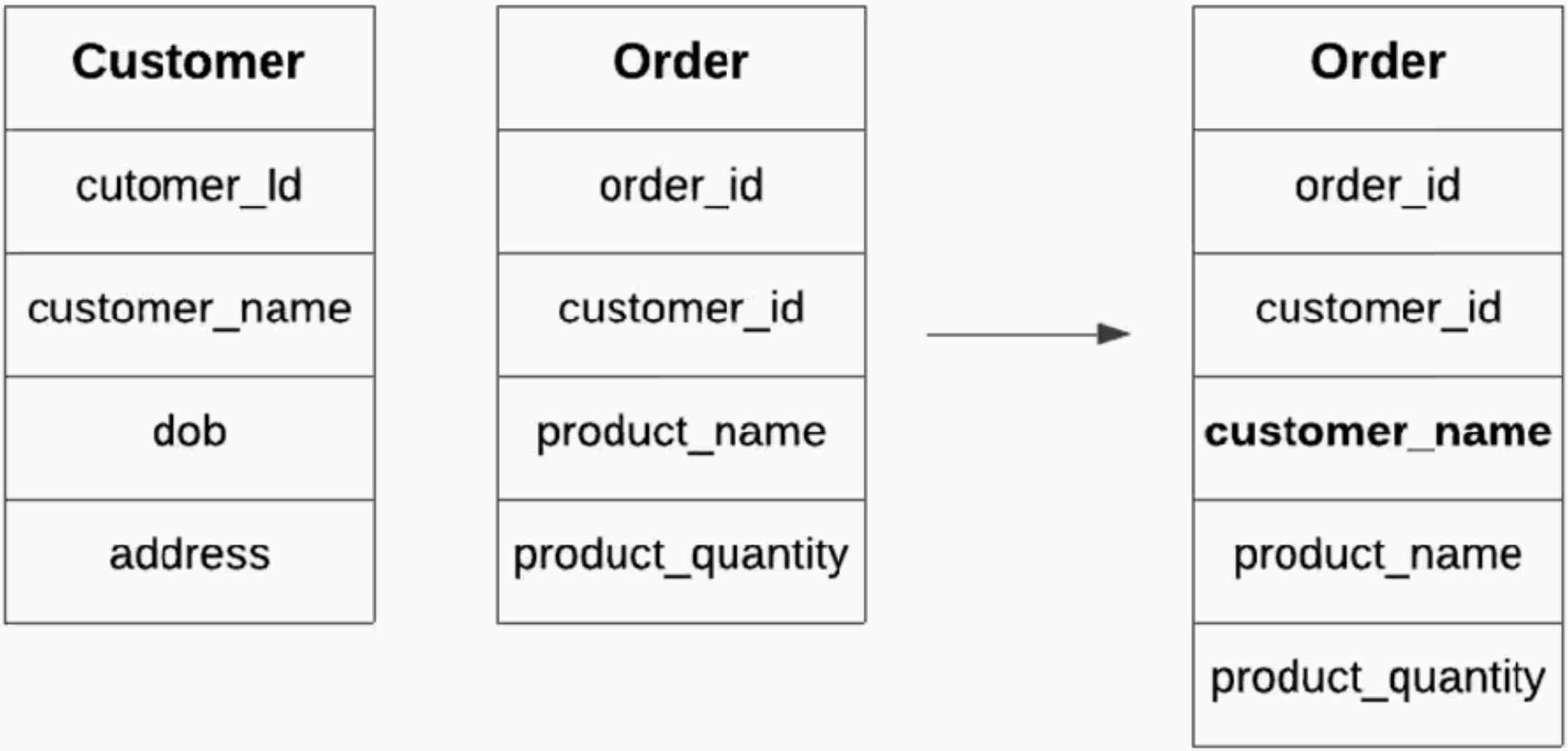
Denormalization techniques

Redundant column/Pre-joining tables
Table splitting (horizontal/vertical)
Add derived columns
Mirrored tables
View vs Materialized views

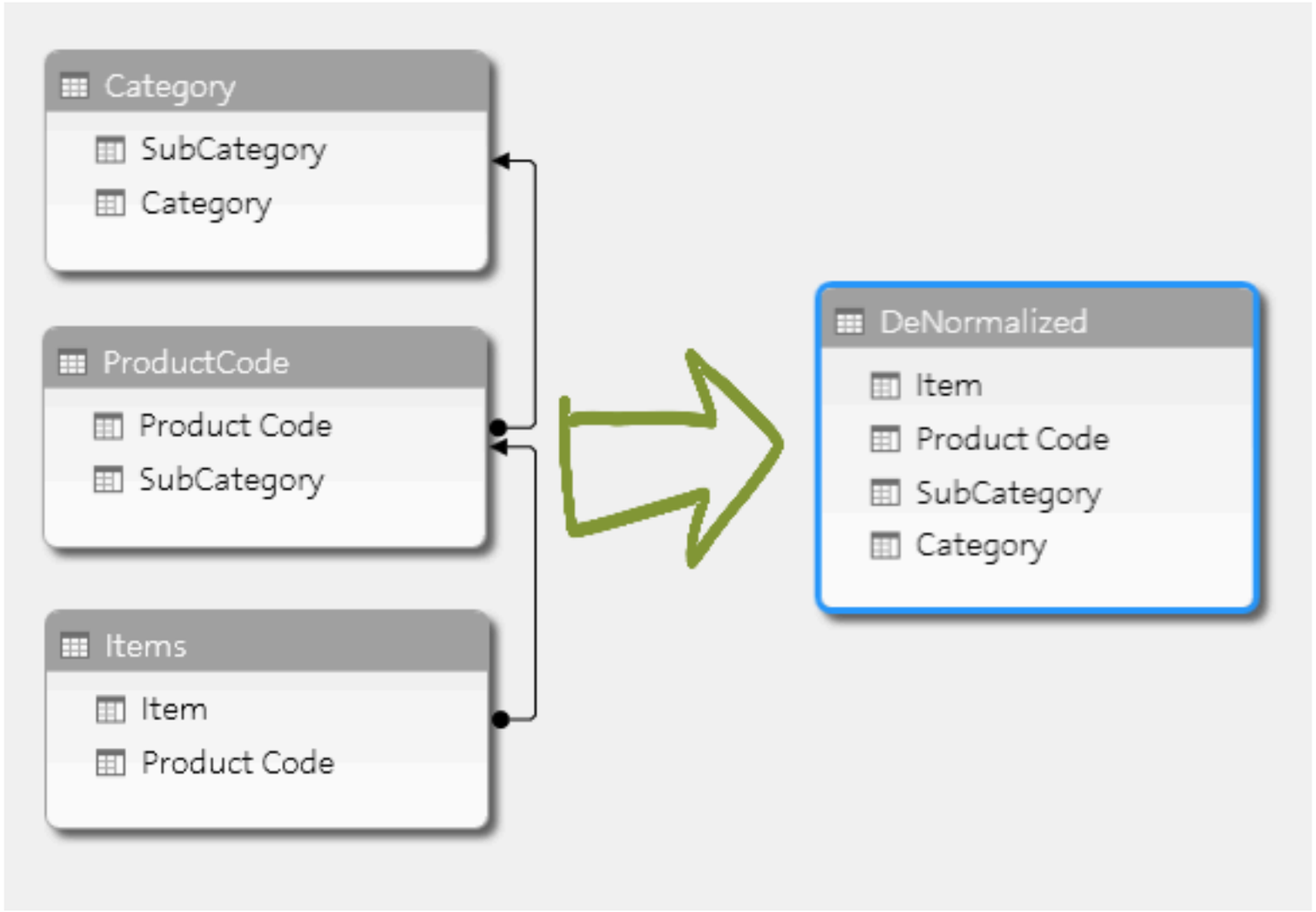
View vs Materialized view ?

Features	View	Materialized view
Storage	Virtual	Physical
Query speed	Slow for complex query	Fast
Data freshness	Realtime	Snapshot, manual refresh
Index	Can not be indexed	Can be indexed
Maintenance	None	Require REFRESH command

Redundant column/Pre-joining tables

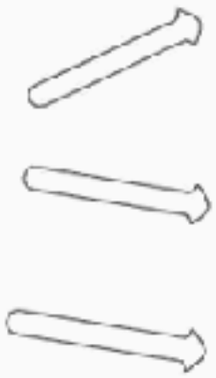


Redundant column/Pre-joining tables



Horizontal splitting

Student_ID	Department_ID	Student_Name
1	1	Alex
2	2	Marie
3	1	Sam
4	3	Sara



Computer Science

Student_ID	Department_ID	Student_Name
1	1	Alex
3	1	Sam

Chemistry

Student_ID	Department_ID	Student_Name
2	2	Marie

Botany

Student_ID	Department_ID	Student_Name
4	3	Sara

Vertical splitting

Add derived columns

Student_ID	Name
1	Alex
2	Marie

Student_ID	Assignment_ID	Mark
1	1	20
1	2	35.50
2	1	45
2	2	45

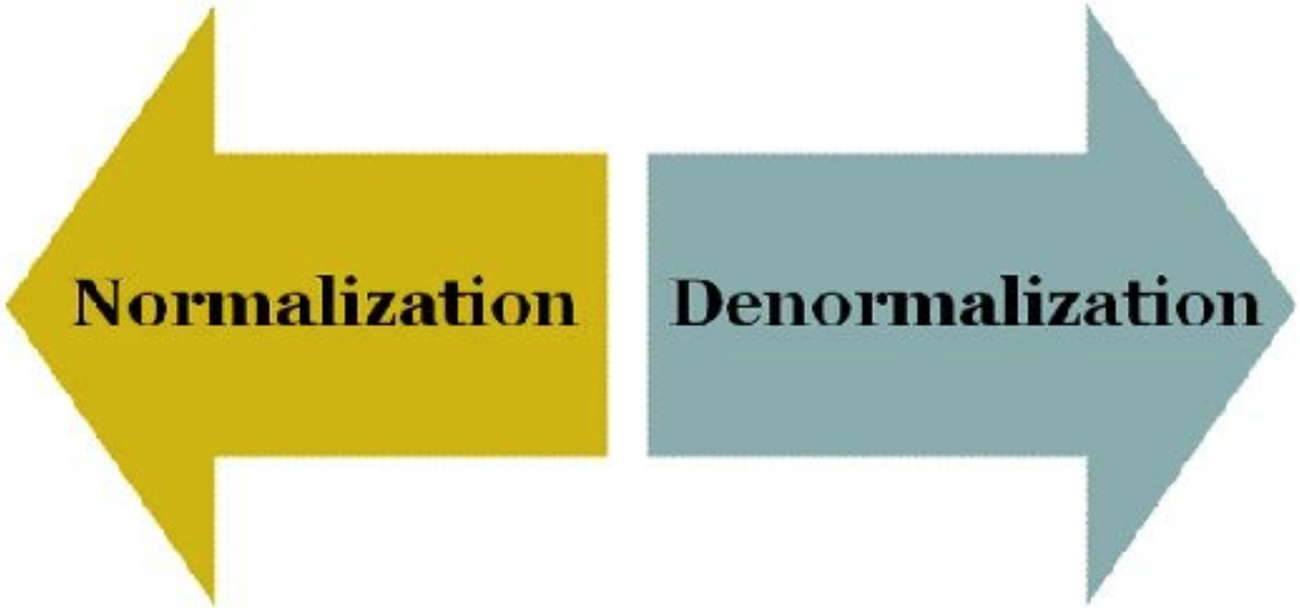
Student_ID	Name	Total_Marks
1	Alex	55.5
2	Marie	90

Pros of data denormalization

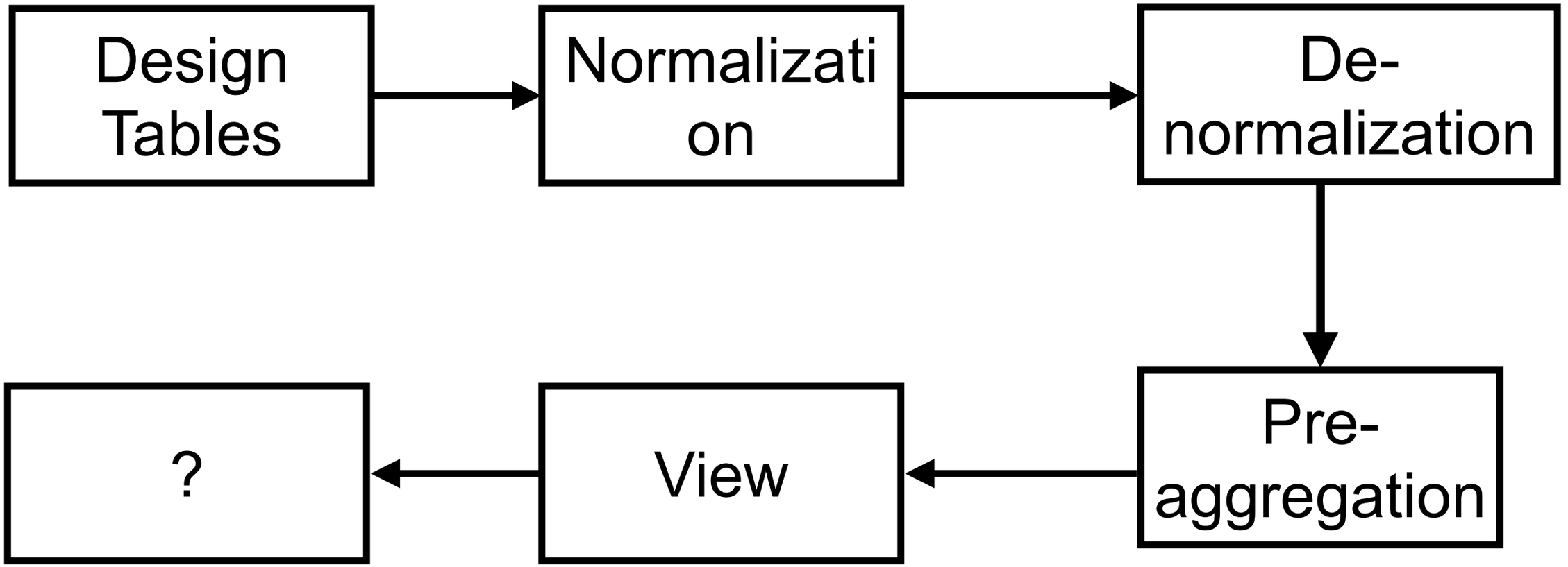
Improve query performance (UX)
Reduce complexity of data model
Improve application scalability
Generate data report faster

Cons of data denormalization

Increase data redundancy
Inconsistency between data sets
Require more disk space (split tables)
More data model (hard to maintain)
Difficult to insert and update data
Maintenance costs



Workshop



Q/A